THIRSTY'S CODEX

STANDARDS EDITION

VOLUME V / 05 OF 20

SOFTWARE SYSTEMS THROUGH SMART CONTRACTS

Lifecycle Sections 107-150

NO PROOF, NO CLAIM

CONTINUOUS MASTER FOLIO / SOURCE-PRESERVING EDITION

Edition Statement

THIRSTY'S CODEX / STANDARDS EDITION / SOURCE-PRESERVING SET

This volume is one part of a continuous 20-volume edition. Canonical lifecycle section numbers remain stable across the set; no volume restarts or renumbers them.

The visual treatment has been rebuilt as a unified standards library: coordinated covers, volume architecture, source dividers, running headers, continuous master folios, local page numbers, and source-identity marks.

The source corpus remains authoritative over the presentation layer. Status terms are preserved exactly. CONTENT-COMPLETE does not silently become source-reconciled, hostile-reviewed, owner-accepted, ratified, or published.

Every registered source witness is identified by path, byte count, and SHA-256 digest. The complete source archive is preserved as an embedded attachment in Volume XX, while the two supplied foundational PDFs are preserved as attachments in Volume I.

CONTINUITY RULE

Read the master folio and canonical section sequence as one uninterrupted work. Each succeeding volume begins where the preceding volume ends.

Standards Edition Set Map

VOLUME I
Foundations and Governance
Doctrine, prompts, front matter, shared governance, roles, and output contracts

VOLUME II
Research, Strategy, and Risk
Lifecycle Sections 001-018

VOLUME III
Requirements through Hardware Architecture
Lifecycle Sections 019-060

VOLUME IV
Electrical Engineering through User Validation
Lifecycle Sections 061-106

VOLUME V
Software Systems through Smart Contracts
Lifecycle Sections 107-150

VOLUME VI
Data, Intelligence, and Human Oversight
Lifecycle Sections 151-188

VOLUME VII
Cybersecurity through Privacy Engineering
Lifecycle Sections 189-218

VOLUME VIII
Law, Assurance, and Validation
Lifecycle Sections 219-257

VOLUME IX
Configuration through Delivery Infrastructure
Lifecycle Sections 258-300

VOLUME X
Production Operations and Command
Lifecycle Sections 301-332

VOLUME XI
Support, Enterprise, and Roadmaps
Lifecycle Sections 333-373

VOLUME XII
Maintenance, Decommissioning, and Retirement
Lifecycle Sections 374-406

VOLUME XIII
Reference Registers and Working Documentation
Indexes, cross-references, source registers, checkpoints, and operational guides

VOLUME XIV
Standards Source Library
Preserved standards, command editions, blueprints, and source artifacts

VOLUME XV
Canonical Authority and Assurance
Authority, constitutions, policies, schemas, provenance, evaluation, and verifica...

VOLUME XVI
Applied Corpus and V3Q
Agent-output corpus, V3Q implementation, fixtures, tests, and governed examples

VOLUME XVII
Training Views and Indexes
Continuous-pretraining, evaluation, retrieval, and training views with canonical ...

VOLUME XVIII
Agents and Owner Command Standards
Agents 101 and Owner's Cockpit source-standard facsimiles

VOLUME XIX
Static Analysis and Tooling Evidence
Codacy configuration, rulesets, analysis logs, and repository tooling

Volume Contents

186 Section 107 - Software Engineering 6
187 Section 108 - Computer Science 11
188 Section 109 - Algorithm Engineering 15
189 Section 110 - Programming Language Engineering 19
190 Section 111 - Compiler Engineering 23
191 Section 112 - Runtime Engineering 27
192 Section 113 - Operating Systems Engineering 31
193 Section 114 - Kernel Engineering 35
194 Section 115 - Driver Engineering 39
195 Section 116 - Middleware Engineering 43
196 Section 117 - Application Engineering 47
197 Section 118 - Frontend Engineering 51
198 Section 119 - Backend Engineering 55
199 Section 120 - Full Stack Engineering 59
200 Section 121 - Web Engineering 63
201 Section 122 - Desktop Engineering 67
202 Section 123 - Mobile Engineering 71
203 Section 124 - Cross Platform Engineering 75
204 Section 125 - Native Application Engineering 79
205 Section 126 - Browser Engineering 83
206 Section 127 - Extension Engineering 87
207 Section 128 - Api Engineering 91
208 Section 129 - Integration Engineering 95
209 Section 130 - Microservices Engineering 99
210 Section 131 - Distributed Systems Engineering 103
211 Section 132 - Real Time Systems Engineering 107
212 Section 133 - High Performance Computing 111
213 Section 134 - Parallel Computing 115
214 Section 135 - Scientific Computing 119
215 Section 136 - Simulation Engineering 123
216 Section 137 - Digital Twin Engineering 127

Volume Contents

217 Section 138 - Game Engineering 131
218 Section 139 - Graphics Engineering 135
219 Section 140 - Rendering Engineering 139
220 Section 141 - Animation Engineering 143
221 Section 142 - Audio Engineering 147
222 Section 143 - Video Engineering 151
223 Section 144 - Media Engineering 155
224 Section 145 - Streaming Engineering 159
225 Section 146 - Search Engineering 163
226 Section 147 - Recommendation Systems Engineering 167
227 Section 148 - Geographic Information Systems 171
228 Section 149 - Blockchain Engineering 175
229 Section 150 - Smart Contract Engineering 179

107

SECTION 107 - SOFTWARE ENGINEERING

CODEX/03_SECTIONS/107_SOFTWARE_ENGINEERING.txt

SHA-256 d13dbc6f183f4a84fa363bdc8e4a783b98845cff35165872e93fcb48dc069a46

12,033 BYTES / PRESERVED SOURCE IDENTITY

SECTION 107 — Software Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define the disciplined creation, modification, verification, delivery, operation, maintenance, and retirement of software within approved requirements, architecture, authority, quality, security, privacy, safety, accessibility, and lifecycle boundaries.

Software Engineering turns authorized intent into maintainable software artifacts and evidence. It covers construction practices and engineering controls, not product ownership, policy authority, or proof that deployed software is effective.

REQUIRED OUTCOME

- A bounded software scope, quality model, lifecycle, ownership, and engineering plan.
- Traceable requirements, design, source, dependencies, configuration, builds, tests, releases, runtime evidence, defects, and retirement records.
- Reproducible development and build conditions with review, secure supply chain, accessibility, observability, and recovery controls.
- Explicit treatment of human, AI-assisted, generated, third-party, and machine-produced code.
- Verification evidence that distinguishes code inspection, test results, build success, deployment, and user or operational acceptance.

WHY THIS MATTERS

Software is executable authority. A small change can alter data, security, user rights, service continuity, safety, cost, or auditability. Engineering controls make intent, change, provenance, defects, and recovery visible across the full software lifecycle.

AI-assisted development increases throughput but also increases the need for attribution, review, dependency scrutiny, test depth, and clear boundaries on generated code. A passing build is not evidence of correctness or authorization.

APPLICABILITY

- Software is designed, coded, configured, generated, integrated, tested, released, operated, maintained, migrated, or retired.
- A change affects behavior, interfaces, data, permissions, performance, reliability, accessibility, security, privacy, safety, or evidence.
- Source, dependencies, build tools, models, generated artifacts, or external services create supply-chain or reproducibility concerns.
- The work requires controlled collaboration, code review, branching, release, rollback, maintenance, or defect management.

NON-APPLICABILITY CONDITIONS

- The work is a read-only inspection or explanatory note with no alteration to software, configuration, dependency, evidence, or lifecycle state.
- An approved engineering baseline covers the exact artifact, version, environment, and change, and impact analysis finds no material effect.
- A prototype is explicitly isolated, non-production, and not represented as verified, secure, supported, or deployable.

The record must state the baseline, scope, owner, evidence date, limitations, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Requirements, functional and operational architecture, security, privacy, safety, accessibility, reliability, and risk constraints are available or marked unresolved.
- Source-control, build, dependency, artifact, environment, test, release, and incident ownership is assigned.
- The target runtime, support horizon, licensing obligations, data boundary, and acceptance authority are known.

REQUIRED INPUTS

- Approved requirements, interfaces, architecture, constraints, risks, acceptance criteria, and change request.
- Source tree, build instructions, dependency manifests and locks, toolchain versions, environment definitions, and artifact locations.
- Test strategy, quality thresholds, threat model, accessibility requirements, observability needs, release plan, support model, and recovery objectives.
- Existing defects, incidents, compatibility commitments, user research, operational evidence, and known assumptions.

HUMAN RESPONSIBILITIES AND AUTHORITY

The accountable human owner approves scope, priorities, risk, quality thresholds, technology choices, exceptions, release readiness, support commitments, and retirement. Humans must review material design and code changes, approve security and privacy consequences, and accept only the exact version and evidence examined.

AI RESPONSIBILITIES AND BOUNDARIES

AI may explain code, draft implementation, generate tests, identify defects, propose refactors, summarize changes, and analyze build or runtime evidence within an explicit assignment. It must identify generated or inferred content, preserve source context, expose uncertainty, and stop when requirements, authority, security, privacy, safety, or evidence are unclear. AI must not merge, release, alter protected environments, accept risk, invent dependencies, or treat generated code, test passage, or build success as acceptance without delegated authority.

APPLICABLE EXISTING AGENT ROLES

- Architect, Analyst, Researcher, Planner, Implementer, Refactorer, Tester, Verifier, Critic, Security Auditor, Governance Reviewer, Documentation Writer, Memory Curator, Coordinator, Optimizer, Red Team, and User Experience Reviewer.

Role selection is proportional to consequence; existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Establish scope, baseline, owners, environments, lifecycle, quality attributes, and non-goals.
2. Inspect requirements, architecture, interfaces, data, risks, controls, and acceptance conditions.
3. Design maintainable modules, contracts, error behavior, state, observability, security, accessibility, and recovery.
4. Plan source control, review, branching, dependency pinning, build reproducibility, artifact promotion, and release evidence.
5. Implement with tests, documentation, secure defaults, input validation, least privilege, and explicit failure handling.
6. Review human-written, AI-generated, copied, generated, and third-party code for correctness, license, security, quality, and provenance.
7. Run proportionate unit, integration, system, performance, security, accessibility, compatibility, and recovery verification.
8. Record defects, deviations, assumptions, waivers, evidence, and unresolved risks; never silently lower the acceptance bar.
9. Package, release, monitor, support, patch, migrate, deprecate, and retire through approved lifecycle controls.

HUMAN-AI INTERACTION REQUIREMENTS

- Show whether an artifact is authored, generated, transformed, copied, reviewed, tested, released, or verified.

- Preserve prompts or generation context where policy permits, tool and model identity, reviewer identity, and resulting diff.
- Require human review for security, privacy, safety, accessibility, data, authorization, external side effects, and release changes.
- Keep manual review, stop, rollback, diagnosis, evidence, and recovery paths available when AI tools fail.
- Do not let conversational convenience bypass code review, change control, provenance, or release authorization.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Keep source, configuration, generated code, dependencies, build outputs, deployment manifests, and runtime state distinct and versioned.
- Define module and interface contracts with identity, schema, compatibility, errors, timing, authorization, observability, and ownership.
- Make builds reproducible or record every unavoidable variance and its impact.
- Treat dependencies, compilers, package registries, models, prompts, tools, and credentials as governed supply-chain inputs.
- Design idempotent migration, safe rollout, rollback or compensation, feature isolation, graceful degradation, and retirement paths.
- Preserve traceability from requirement through commit, artifact, release, runtime observation, defect, and retirement.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show repository, baseline, commit, build, dependency, environment, release, test, risk, and evidence status.
- Expose changed files, affected requirements, reviewers, generated-code markers, findings, approvals, and invalidated evidence.
- Provide authorized controls to review, approve, reject, pause, quarantine, promote, rollback, restore, supersede, and retire.
- Distinguish build success, test success, deployment, runtime health, user acceptance, and operational effectiveness.
- Export the exact source, metadata, evidence, decisions, dissent, and handoff record.

REQUIRED OUTPUTS AND ARTIFACTS

- Software engineering plan, baseline, design, source, dependency, configuration, and build records.
- Reviewed changes, traceability, test plans and results, defect and risk registers, security and license findings.
- Reproducible build and artifact manifests, release package, deployment and rollback instructions, support and retirement records.
- AI-assistance provenance and human-review record where applicable.

EVIDENCE REQUIREMENTS

- Each claim identifies artifact, version, environment, method, actor, date, result, limitations, and exact scope.
- Build, test, review, scan, deployment, runtime, user, and operational evidence remain distinct.
- Dependency and generated-code evidence identifies source, version, license, tool or model, configuration, reviewer, and unresolved risk.
- Failed, stale, partial, contradictory, and missing evidence constrains status and remains visible.

VERIFICATION AND ACCEPTANCE TESTS

- Reproduce a clean build from the declared source and dependency inputs.
- Trace requirements to design, code, tests, artifact, release, runtime evidence, and acceptance.
- Introduce malformed input, unauthorized access, dependency failure, partial deployment, stale configuration, and unavailable build services; verify safe behavior.
- Compare AI-generated and human-authored changes under the same review, test, security, license, and release controls.
- Verify rollback or compensation, migration, observability, accessibility, support, and retirement procedures with recorded results.

FAILURE STATES AND ANTI-PATTERNS

- Treating compilation, code coverage, a green pipeline, or deployment as proof of outcome.
- Merging unreviewed generated code, floating dependencies, unverifiable binaries, or undocumented environment changes.
- Hiding defects, accessibility failures, security findings, license conflicts, or operational burden.
- Using rollback to erase evidence or reverse external effects without a compensation record.

RECOVERY AND RECONSIDERATION

Stop affected promotion on material uncertainty, failed controls, unsafe behavior, provenance loss, or invalid evidence. Preserve source, artifacts, logs, decisions, and user impact. Use approved isolation, rollback, restore, hotfix, substitution, or compensation; reassess requirements and reopen the baseline when assumptions or dependencies change.

COMPLETION CRITERIA

Scope, ownership, design, source, dependencies, build, review, tests, evidence, release, support, recovery, and retirement are explicit. Material findings are resolved, accepted by authorized humans, or recorded as blockers. This draft is not proof of implementation or production readiness.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve baseline, commit, artifact, dependency, environment, decisions, evidence, dissent, defects, open risks, and next safe action. Hand off to Application, Software, Platform, Build, Testing, Security, Operations, Maintenance, and Retirement sections. Record conditions that require reopening.

RELATED CODEX SECTIONS

- 019 Requirements Engineering; 020 Systems Engineering; 027 Process Architecture; 028 Functional Architecture; 029 Operational Architecture.
- 033-044 Software through Runtime Architecture; 045-053 Security through Accessibility Architecture.
- 231-270 Testing, Build, CI, Delivery, DevSecOps; 301-406 Operations, Maintenance, Governance, and Retirement.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-032, Owner Cockpit Requirements, Status and Evidence Lexicon, and lifecycle conventions.
- Sector-specific engineering standards, language policies, licensing rules, toolchain constraints, support commitments, and acceptance thresholds remain unresolved until supplied for the actual system.

108

SECTION 108 - COMPUTER SCIENCE

CODEX/03_SECTIONS/108_COMPUTER_SCIENCE.txt

SHA-256 50075e22762e9734c69fe4c98263e0f9092701fb8144505c368ef52438c04162

6,482 BYTES / PRESERVED SOURCE IDENTITY

SECTION 108 — Computer Science

STATUS: CONTENT-COMPLETE

PURPOSE

Define the governed use of computational models, abstractions, theories, and methods to understand, design, analyze, and verify systems without confusing a mathematical result with an implemented or acceptable system.

REQUIRED OUTCOME

The result is a bounded computational question, model or proof basis, assumptions, complexity analysis, reproducible method, evidence, limitations, and handoff to engineering or architecture.

WHY THIS MATTERS

Computer science provides the abstractions used to reason about computation, information, algorithms, languages, systems, and limits. Unstated assumptions can make a formally valid result irrelevant to real users, data, resources, security, or lifecycle conditions.

APPLICABILITY

Applies when computational complexity, correctness, formal models, algorithms, data structures, languages, concurrency, distributed behavior, information representation, or theoretical limits affect a design or decision.

NON-APPLICABILITY CONDITIONS

Not applicable to a purely descriptive or bounded editorial change with no computational claim, model, method, requirement, implementation, or evidence consequence. Record the exact scope and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires an authorized question, domain and requirements context, declared assumptions, relevant data or formal definitions, qualified review, and a reproducible method. Related architecture, security, privacy, safety, and performance constraints must be available or marked unresolved.

REQUIRED INPUTS

Question, definitions, inputs, constraints, model, proof or experiment method, expected properties, complexity target, resource limits, threat assumptions, evidence standard, and acceptance authority.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans set the question, acceptable abstraction, values, constraints, interpretation, risk, and acceptance. They approve claims that affect architecture, policy, safety, rights, or investment and ensure qualified independent review.

AI RESPONSIBILITIES AND BOUNDARIES

AI may explain concepts, propose models, derive examples, check reasoning, generate experiments, and identify counterexamples. It must label conjecture, proof, simulation, and empirical result separately; it must not invent a proof, hide assumptions, or approve consequential conclusions.

APPLICABLE EXISTING AGENT ROLES

Architect, Researcher, Analyst, Critic, Verifier, Tester, Security Auditor, Governance Reviewer, Documentation Writer, Memory Curator, Planner, and Teacher. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define the problem, abstraction, scope, assumptions, and non-goals.
2. Select notation, models, data structures, computational resources, and correctness properties.
3. Analyze feasibility, complexity, termination, determinism, concurrency, information loss, and adversarial conditions.
4. Prove, model-check, simulate, implement a reference, or experimentally evaluate as appropriate.
5. Seek counterexamples, boundary conditions, sensitivity, and independent review.
6. Record results, limitations, unresolved gaps, and implications for implementation and operations.

HUMAN-AI INTERACTION REQUIREMENTS

Human review is required for model selection, assumption acceptance, proof claims, security or safety conclusions, and decisions affecting people or resources. AI output remains advisory until independently checked.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Keep formal model, reference implementation, production implementation, data, experiment, and evidence separate. Record computational resource limits, nondeterminism, precision, reproducibility, and invalidation conditions. Preserve fallback and human interpretation for consequential results.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show question, model version, assumptions, method, proof or experiment status, evidence freshness, limitations, reviewer, and actions to reproduce or challenge the result. Provide controls to quarantine, supersede, or reopen claims.

REQUIRED OUTPUTS AND ARTIFACTS

Problem statement, formal definitions, model, proof or analysis, reference code, experiment record, complexity and resource analysis, counterexamples, review record, limitations, and engineering handoff.

EVIDENCE REQUIREMENTS

Evidence identifies definitions, model version, inputs, method, environment, seed or nondeterminism, resource limits, reviewer, result, and excluded claims. Missing or inconclusive evidence is not success.

VERIFICATION AND ACCEPTANCE TESTS

Reproduce the proof or experiment; vary assumptions; test boundary and adversarial inputs; compare model and implementation; measure resource use; independently challenge the conclusion; verify handoff requirements and invalidation triggers.

FAILURE STATES AND ANTI-PATTERNS

Unstated assumptions, asymptotic claims treated as production performance, simulation treated as proof, proof treated as user acceptance, ignored resource limits, and AI-generated reasoning accepted without checking.

RECOVERY AND RECONSIDERATION

Preserve the prior claim and evidence when a counterexample, assumption change, or reproducibility failure appears. Narrow use, rerun analysis, obtain qualified review, or supersede through an attributable decision; do not erase the failed result.

COMPLETION CRITERIA

The question, model, assumptions, method, result, limits, review, acceptance authority, and engineering implications are explicit, reproducible within declared conditions, and free of unrecorded material gaps.

HANDOFF AND CONTINUITY REQUIREMENTS

Hand off model identity, definitions, assumptions, results, counterexamples, evidence, decisions, dissent, open work, and reopening conditions to Algorithm, Software, Data, AI, Security, Testing, and Architecture sections.

RELATED CODEX SECTIONS

019 Requirements Engineering; 020 Systems Engineering; 028 Functional Architecture; 031 Data Architecture; 033 Software Architecture; 056 AI Architecture; 109 Algorithm Engineering; 111 Compiler Engineering; 132-135 Real-Time, High-Performance, Parallel, and Scientific Computing; 180 Model Evaluation.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract and shared governance conventions. The Codex does not select a universal formalism, proof assistant, computational model, curriculum, or domain-specific acceptance threshold; those remain system-specific gaps.

109

SECTION 109 - ALGORITHM ENGINEERING

CODEX/03_SECTIONS/109_ALGORITHM_ENGINEERING.txt

SHA-256 4f1d9f77273b4c2833eb8d321ba6872c417e1ee7dda3e28fee17acb8ea51ffaf

6,587 BYTES / PRESERVED SOURCE IDENTITY

SECTION 109 — Algorithm Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define the selection, design, implementation, analysis, testing, tuning, deployment, monitoring, and retirement of algorithms that transform inputs into outputs or decisions under explicit correctness, complexity, resource, security, fairness, safety, and human-control constraints.

REQUIRED OUTCOME

A specified algorithm with contract, assumptions, complexity, resource limits, implementation, test evidence, alternatives, failure behavior, monitoring, and acceptance boundaries.

WHY THIS MATTERS

Algorithmic errors can be systematic, invisible, amplified by scale, or mistaken for neutral automation. Engineering must expose tradeoffs, edge cases, distribution changes, adversarial inputs, and the difference between technical output and authorized decision.

APPLICABILITY

Applies to deterministic, randomized, heuristic, optimization, search, ranking, scheduling, control, cryptographic, statistical, or learned algorithms with material system effects.

NON-APPLICABILITY CONDITIONS

A trivial explanatory example or bounded refactoring with no algorithmic behavior, contract, performance, data, risk, or evidence effect may be exempt. Record the algorithm baseline and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires requirements, objective function, input and output contracts, data characteristics, constraints, threat model, resource budget, quality measures, and acceptance authority. Domain, privacy, security, safety, and accessibility obligations must be identified.

REQUIRED INPUTS

Problem definition, inputs, outputs, invariants, objective, constraints, complexity target, workload, data, error tolerance, adversary model, baseline, alternatives, test oracle, evidence method, and lifecycle conditions.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve the objective, tradeoffs, protected constraints, acceptable error, risk, deployment, and use of outputs in consequential decisions. They ensure affected parties and qualified reviewers are represented.

AI RESPONSIBILITIES AND BOUNDARIES

AI may propose algorithms, explain tradeoffs, generate reference implementations and tests, find counterexamples, and analyze traces. It must expose assumptions and uncertainty, not fabricate complexity or benchmark results, and must not select or deploy a consequential algorithm without authority.

APPLICABLE EXISTING AGENT ROLES

Architect, Analyst, Researcher, Implementer, Optimizer, Tester, Verifier, Critic, Red Team, Security Auditor, User Experience Reviewer, Governance Reviewer, Documentation Writer, and Memory Curator. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define the problem, objective, output use, constraints, and non-goals.
2. Characterize inputs, distributions, missingness, adversarial conditions, and workload.
3. Specify invariants, contracts, complexity, accuracy, latency, resource, fairness, safety, and failure requirements.
4. Compare alternatives and implement a reference and production candidate.
5. Test ordinary, boundary, malformed, adversarial, shifted, unavailable, duplicate, and worst-case inputs.
6. Measure, review, monitor, tune, version, release, and retire with evidence and rollback or compensation paths.

HUMAN-AI INTERACTION REQUIREMENTS

Expose algorithm identity, version, data, configuration, objective, authority, reviewer, and result. Human review is required for protected outcomes, ranking, denial, allocation, safety, privacy, or external effects. Preserve explanation, appeal, and manual fallback.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Version algorithm, parameters, data assumptions, random seeds, dependencies, test oracle, and evaluation environment. Separate reference, benchmark, production, and observed behavior. Design idempotency, resource limits, monitoring, drift detection, safe fallback, and retirement.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show algorithm version, objective, input scope, output use, quality, resource and evidence status, drift, incidents, approvals, and available stop, quarantine, fallback, supersede, and rollback or compensation controls.

REQUIRED OUTPUTS AND ARTIFACTS

Algorithm specification, contract, rationale, alternatives, reference implementation, production implementation, complexity analysis, test suite, benchmark, threat and fairness review, monitoring, release, and retirement records.

EVIDENCE REQUIREMENTS

Record data, workload, environment, configuration, seed, metrics, uncertainty, baselines, excluded populations, reviewer, and limitations. Benchmark results do not prove field performance or acceptable outcomes.

VERIFICATION AND ACCEPTANCE TESTS

Verify invariants, correctness, complexity, resource limits, reproducibility, boundary and adversarial behavior, data shift, accessibility, security, fairness where applicable, fallback, monitoring, and independent acceptance.

FAILURE STATES AND ANTI-PATTERNS

Optimizing a proxy while harming the outcome, ignoring worst-case behavior, leaking sensitive inputs, treating ranking as neutral, using stale benchmarks, hiding error distributions, and deploying without drift or fallback controls.

RECOVERY AND RECONSIDERATION

On defect, drift, unsafe output, resource exhaustion, or evidence loss, stop or narrow use, preserve inputs and outputs, notify the owner, use an approved prior version or manual path, and record compensation for external effects. Reopen when objective, data, constraints, or population changes.

COMPLETION CRITERIA

The algorithm and its use are bounded, specified, implemented, independently tested, evidenced, accepted for the exact scope, monitored, and supported by a truthful recovery and retirement plan.

HANDOFF AND CONTINUITY REQUIREMENTS

Hand off algorithm identity, contracts, data assumptions, tests, benchmarks, risks, incidents, decisions, evidence, dissent, and next actions to Software, Data, AI, Security, Operations, Testing, and Maintenance sections.

RELATED CODEX SECTIONS

108 Computer Science; 019 Requirements Engineering; 028 Functional Architecture; 031 Data Architecture; 056-057 AI and Machine Learning Architecture; 132-135 Computing sections; 167-180 AI Engineering through Model Evaluation; 208 Threat Modeling; 239-245 Performance and Reliability Testing.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract and shared governance conventions. Domain-specific fairness, safety, statistical, cryptographic, and regulatory thresholds remain unresolved until the system owner supplies them.

110

SECTION 110 - PROGRAMMING LANGUAGE ENGINEERING

CODEX/03_SECTIONS/110_PROGRAMMING_LANGUAGE_ENGINEERING.txt

SHA-256 ede9904af1912009190599577df44bd4c00ea40bf14bb9cdc1bceb74632a8441

6,584 BYTES / PRESERVED SOURCE IDENTITY

SECTION 110 — Programming Language Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define the governed design, implementation, evolution, tooling, verification, distribution, and retirement of programming languages and their ecosystems, including syntax, semantics, type systems, compilers, runtimes, libraries, diagnostics, compatibility, and migration.

REQUIRED OUTCOME

A versioned language and toolchain contract with intended users, semantics, safety and security properties, implementation status, conformance evidence, compatibility policy, migration path, and retirement conditions.

WHY THIS MATTERS

Language behavior is infrastructure for every program written in it. Ambiguous semantics, unsound typing, compiler divergence, runtime incompatibility, or poor diagnostics can create failures far beyond the language project.

APPLICABILITY

Applies when a language, dialect, DSL, compiler, interpreter, virtual machine, standard library, formatter, linter, debugger, package ecosystem, or compatibility guarantee is designed or changed.

NON-APPLICABILITY CONDITIONS

A project merely using an established language without changing its language, toolchain, compatibility, or conformance claims is not itself language engineering. Record the consumed baseline and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires language goals, semantic model, grammar, type and memory model, runtime assumptions, target platforms, threat model, compatibility horizon, conformance suite, licensing, and human acceptance authority.

REQUIRED INPUTS

Use cases, syntax, semantics, grammar, type rules, evaluation order, error model, resource and safety limits, ABI or interface rules, implementation strategy, tests, benchmarks, compatibility and migration requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve semantics, safety and security tradeoffs, compatibility breaks, ecosystem commitments, licensing, release, deprecation, and retirement. Qualified language and systems reviewers must challenge material claims.

AI RESPONSIBILITIES AND BOUNDARIES

AI may draft grammar, examples, tests, documentation, diagnostics, and implementation proposals. It must not invent semantic authority, declare conformance, hide incompatibility, or release a compiler or runtime without delegated review and evidence.

APPLICABLE EXISTING AGENT ROLES

Architect, Researcher, Analyst, Implementer, Tester, Verifier, Critic, Security Auditor, Optimizer, Documentation Writer, Governance Reviewer, Memory Curator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define language purpose, users, scope, non-goals, and compatibility horizon.
2. Specify grammar, semantics, types, effects, memory, concurrency, errors, modules, tooling, and runtime behavior.
3. Analyze safety, security, performance, portability, accessibility of tooling, licensing, and migration.
4. Build reference and production implementations with diagnostics and conformance tests.
5. Test valid, invalid, ambiguous, adversarial, resource-bound, concurrent, upgrade, and recovery cases.
6. Publish versioned documentation, compatibility statements, migration tools, support policy, and retirement evidence.

HUMAN-AI INTERACTION REQUIREMENTS

AI-generated language features and compiler code remain proposals until human review. Show source specification, implementation, generated tests, tool and model identity, and known gaps. Preserve a stable escape hatch, diagnostics, and manual recovery for tooling failure.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate specification, reference interpreter, compiler, runtime, standard library, tools, packages, generated artifacts, and target binaries. Define conformance, determinism, reproducibility, sandboxing, resource limits, ABI, versioning, migration, and deprecation.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show language and toolchain version, specification status, conformance, target platforms, compatibility, security findings, release authority, evidence freshness, migration, and retirement controls.

REQUIRED OUTPUTS AND ARTIFACTS

Language specification, grammar and semantic model, compatibility matrix, reference implementation, compiler or runtime artifacts, conformance suite, diagnostics guide, benchmarks, threat review, migration guide, release record, and retirement plan.

EVIDENCE REQUIREMENTS

Evidence identifies specification version, implementation, target, test corpus, environment, result, deviations, reviewer, and limitations. Passing examples do not prove complete conformance, safety, or ecosystem compatibility.

VERIFICATION AND ACCEPTANCE TESTS

Compare reference and production semantics; test parser, type checker, optimizer, runtime, diagnostics, concurrency, memory, errors, resource exhaustion, portability, version compatibility, security, and migration; independently reproduce conformance results.

FAILURE STATES AND ANTI-PATTERNS

Undocumented semantics, compiler-specific behavior presented as standard, unsound optimization, silent breaking change, unsafe runtime default, incomplete diagnostics, unbounded resource use, and a migration plan that depends on the retired toolchain.

RECOVERY AND RECONSIDERATION

Quarantine the affected release, preserve source and artifacts, route users to a supported version or compatibility mode, publish the defect and impact, and use compensation for external effects. Reopen the specification when evidence or intended semantics change.

COMPLETION CRITERIA

Semantics, implementation, conformance, compatibility, security, support, migration, evidence, ownership, and retirement are explicit for the declared version and targets.

HANDOFF AND CONTINUITY REQUIREMENTS

Hand off specifications, test corpus, artifacts, compatibility, decisions, evidence, defects, migration, and open gaps to Compiler, Runtime, Software, Platform, Security, Operations, Testing, and Maintenance sections.

RELATED CODEX SECTIONS

108 Computer Science; 109 Algorithm Engineering; 019 Requirements Engineering; 020 Systems Engineering; 033 Software Architecture; 044 Runtime Architecture; 045 Security Architecture; 111 Compiler Engineering; 112 Runtime Engineering; 260-261 Dependency and Package Management; 388-389 Version and Deprecation Management.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract and shared governance conventions. No universal language specification, memory model, conformance level, target platform, licensing model, or compatibility promise is supplied; these remain system-specific gaps.

111

SECTION 111 - COMPILER ENGINEERING

CODEX/03_SECTIONS/111_COMPILER_ENGINEERING.txt

SHA-256 261110a28a59e01cb6d2ff334d5fea304fdabccb638a56b626d39c2e099031b8

5,783 BYTES / PRESERVED SOURCE IDENTITY

SECTION 111 — Compiler Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define the governed construction and evolution of compilers, interpreters, linkers, optimizers, assemblers, and associated tooling that translate source programs into executable or intermediate forms.

REQUIRED OUTCOME

A versioned compiler contract, implementation, target matrix, conformance suite, reproducible build, diagnostics, security model, compatibility statement, and evidence-backed release boundary.

WHY THIS MATTERS

Translation defects can change program meaning, weaken security, corrupt data, or create platform-specific failures. Compiler success is not proof that the source intent or target behavior is correct.

APPLICABILITY

Applies to compiler, interpreter, assembler, linker, optimizer, code generator, transpiler, static analyzer, debugger, build backend, or language-toolchain change.

NON-APPLICABILITY CONDITIONS

Using an unchanged, supported toolchain without changing its behavior or trust boundary is not compiler engineering. Record the consumed version and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires language specification, source and target model, ABI, optimization constraints, threat model, supported platforms, build inputs, conformance tests, licensing, and release authority.

REQUIRED INPUTS

Grammar and semantics, intermediate representation, passes, diagnostics, target architecture, runtime and ABI, optimization invariants, reproducibility needs, test corpus, performance and security limits.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve semantics, optimization risk, target support, security posture, compatibility breaks, release, and retirement; independent qualified review is required for safety or security-sensitive claims.

AI RESPONSIBILITIES AND BOUNDARIES

AI may draft passes, tests, diagnostics, documentation, and code review findings. It must not invent language semantics, claim conformance, hide miscompilation, or publish artifacts without human authorization and evidence.

APPLICABLE EXISTING AGENT ROLES

Architect, Analyst, Researcher, Implementer, Tester, Verifier, Critic, Security Auditor, Optimizer, Documentation Writer, Governance Reviewer, Red Team, and Memory Curator. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Establish specification, targets, ABI, trust boundary, scope, and non-goals.

2. Design front end, intermediate representation, passes, back end, diagnostics, and runtime interactions.
3. Preserve semantic invariants through optimization and code generation.
4. Build reproducibly and test valid, invalid, adversarial, resource-bound, cross-target, and regression cases.
5. Record deviations, release evidence, migration, fallback, support, and retirement conditions.

HUMAN-AI INTERACTION REQUIREMENTS

Generated compiler code and tests require human review. Show source, specification, toolchain, model or tool identity, and evidence. Keep diagnostic, manual build, rollback, and recovery paths available.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate specification, compiler stages, generated code, runtime, binaries, symbols, build inputs, caches, and signing. Record deterministic settings, provenance, sandboxing, resource limits, ABI, compatibility, and artifact promotion.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show compiler version, source hash, target, pass configuration, conformance, diagnostics, security findings, build reproducibility, evidence freshness, approval, and quarantine or rollback controls.

REQUIRED OUTPUTS AND ARTIFACTS

Compiler specification, architecture, source, test corpus, conformance matrix, reproducible build record, binaries, diagnostics, benchmark, security review, release, compatibility, and retirement artifacts.

EVIDENCE REQUIREMENTS

Identify source, compiler, target, flags, dependencies, environment, corpus, expected and actual output, reviewer, deviations, and limits. Benchmarks and successful compilation do not prove semantic correctness.

VERIFICATION AND ACCEPTANCE TESTS

Differential-test reference and production tools; test optimization invariants, malformed input, undefined behavior, ABI, concurrency, resource exhaustion, security, reproducibility, portability, and migration; reproduce failures independently.

FAILURE STATES AND ANTI-PATTERNS

Unreviewed optimization, target-specific miscompilation, unsigned binaries, hidden build flags, stale test corpus, unsafe diagnostics, silent ABI break, and treating compiler speed as program correctness.

RECOVERY AND RECONSIDERATION

Quarantine affected versions, preserve source and binaries, notify users, provide a supported prior toolchain or compensation path, and publish impact. Reopen the specification when semantics, targets, or evidence change.

COMPLETION CRITERIA

The declared toolchain is specified, reproducible, tested, reviewed, evidenced, supported, and accepted for exact targets and versions with compatibility and retirement plans.

HANDOFF AND CONTINUITY REQUIREMENTS

Hand off specification, source, artifacts, corpus, deviations, evidence, decisions, defects, compatibility, migration, and open gaps to Language, Runtime, Software, Platform, Security, Testing, and Operations sections.

RELATED CODEX SECTIONS

108 Computer Science; 110 Programming Language Engineering; 019 Requirements Engineering; 033 Software Architecture; 044 Runtime Architecture; 045 Security Architecture; 264-268 Build, CI, Delivery, and Deployment; 388-389 Version and Deprecation Management.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract and shared governance conventions. Target-specific conformance levels, compiler standards, ABI commitments, toolchain policy, and licensing remain system-specific gaps.

112

SECTION 112 - RUNTIME ENGINEERING

CODEX/03_SECTIONS/112_RUNTIME_ENGINEERING.txt

SHA-256 0cd8e02420d9267227b52fc9e51eb02c64a6224e4c457b0209776a014f167b89

5,350 BYTES / PRESERVED SOURCE IDENTITY

SECTION 112 — Runtime Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define the design, implementation, verification, operation, and retirement of execution environments that load, schedule, isolate, monitor, and support software.

REQUIRED OUTCOME

A versioned runtime contract covering execution semantics, resources, isolation, compatibility, observability, failure, recovery, security, support, and evidence.

WHY THIS MATTERS

Runtime behavior determines what software actually does under load, failure, concurrency, and hostile input. A healthy process or passing unit test cannot establish runtime safety or correctness.

APPLICABILITY

Applies to virtual machines, interpreters, managed runtimes, containers, schedulers, garbage collectors, loaders, plugin hosts, sandboxes, and execution libraries.

NON-APPLICABILITY CONDITIONS

An application consuming an unchanged supported runtime is not runtime engineering; record the runtime baseline and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires language and functional contracts, resource and isolation model, target environment, threat model, workload, compatibility, observability, recovery, and acceptance authority.

REQUIRED INPUTS

Execution semantics, memory and concurrency model, scheduling, quotas, isolation, startup and shutdown, errors, signals, telemetry, upgrade, platform constraints, tests, and support horizon.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve isolation, resource, safety, privacy, security, compatibility, release, incident, and retirement decisions. Operators retain stop and recovery authority.

AI RESPONSIBILITIES AND BOUNDARIES

AI may analyze traces, draft runtime code or tests, and propose tuning. It must not change production scheduling, quotas, isolation, or recovery without delegation and evidence; model advice remains advisory.

APPLICABLE EXISTING AGENT ROLES

Architect, Analyst, Researcher, Implementer, Tester, Verifier, Optimizer, Critic, Security Auditor, Governance Reviewer, Documentation Writer, Memory Curator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define execution contract, resources, modes, isolation, dependencies, and non-goals.

2. Specify lifecycle, scheduling, memory, I/O, concurrency, failure, telemetry, and compatibility behavior.
3. Implement and test normal, boundary, adversarial, overloaded, interrupted, and degraded execution.
4. Verify observability, safe shutdown, recovery, upgrade, rollback, support, and retirement.

HUMAN-AI INTERACTION REQUIREMENTS

Expose runtime version, workload, data, authority, tool or model use, decision, and evidence. Require human review for consequential tuning or isolation changes, with manual stop and fallback.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate runtime definition, configuration, host, workload, process state, telemetry, and artifacts. Enforce quotas, least privilege, sandboxing, backpressure, cancellation, idempotency, and safe upgrade.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show runtime and host identity, mode, resource use, health, authority, dependency state, evidence freshness, active changes, and authorized stop, isolate, restore, and rollback controls.

REQUIRED OUTPUTS AND ARTIFACTS

Runtime specification, design, source, configuration, compatibility matrix, test and stress results, threat review, telemetry model, runbooks, release, recovery, and retirement records.

EVIDENCE REQUIREMENTS

Identify workload, host, runtime, configuration, resource limits, inputs, environment, method, output, reviewer, and limitations. Health metrics do not prove correct or safe execution.

VERIFICATION AND ACCEPTANCE TESTS

Exercise scheduling, memory, concurrency, isolation, signals, cancellation, overload, dependency loss, malformed input, upgrade, restore, observability, and safe shutdown; reproduce incidents independently.

FAILURE STATES AND ANTI-PATTERNS

Unbounded resource use, shared mutable state without control, hidden host dependence, silent data loss on shutdown, missing telemetry, unsafe plugin loading, and tuning that sacrifices isolation or recovery.

RECOVERY AND RECONSIDERATION

Stop or isolate the affected runtime, preserve state and evidence, use an approved fallback or prior version, restore or compensate external effects, and reopen after root-cause and evidence review.

COMPLETION CRITERIA

Execution semantics, resources, isolation, observability, recovery, compatibility, ownership, evidence, and operational support are explicit and verified for declared hosts and workloads.

HANDOFF AND CONTINUITY REQUIREMENTS

Hand off runtime contract, configuration, artifacts, host assumptions, telemetry, incidents, decisions, recovery, and open gaps to Software, Platform, Infrastructure, Operations, Security, Testing, and Maintenance sections.

RELATED CODEX SECTIONS

033-044 Software through Runtime Architecture; 045 Security Architecture; 051-052 Reliability and Resilience Architecture; 112 Runtime Engineering; 130-134 Microservices through Parallel Computing; 301-330 Operations and Recovery.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract and shared governance conventions. Host-specific runtime guarantees, resource budgets, isolation standards, support periods, and recovery objectives remain unresolved.

113

SECTION 113 - OPERATING SYSTEMS ENGINEERING

CODEX/03_SECTIONS/113_OPERATING_SYSTEMS_ENGINEERING.txt

SHA-256 7520d77ae970bd77cc084dd788e21a6e8ff0902d1eaa6c3627af0fdf709abd33

5,413 BYTES / PRESERVED SOURCE IDENTITY

SECTION 113 — Operating Systems Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define the engineering of operating-system services that manage hardware, processes, memory, files, devices, identity, isolation, scheduling, and system calls.

REQUIRED OUTCOME

A versioned OS contract, kernel and user-space boundaries, security model, driver and hardware assumptions, compatibility, test evidence, update, recovery, and retirement plan.

WHY THIS MATTERS

Operating systems mediate nearly every resource and authority. A defect can compromise confidentiality, integrity, availability, safety, or recoverability across all applications.

APPLICABILITY

Applies to kernels, system services, distributions, boot systems, filesystems, process models, security subsystems, device support, and OS-level configuration or update.

NON-APPLICABILITY CONDITIONS

Using an unchanged supported OS without altering OS behavior, configuration baseline, trust boundary, or support claim is not OS engineering; record the baseline.

PREREQUISITES AND DEPENDENCIES

Requires hardware and platform model, threat model, privilege model, resource goals, compatibility, update, backup, recovery, testing, support, and acceptance authority.

REQUIRED INPUTS

System calls, processes, memory, storage, scheduling, identity, permissions, boot, update, logging, hardware, drivers, failure, recovery, target platforms, and lifecycle obligations.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve privilege, security, safety, compatibility, update, release, incident, and retirement decisions. They retain authority to isolate, stop, restore, and accept residual risk.

AI RESPONSIBILITIES AND BOUNDARIES

AI may analyze source and telemetry, draft tests and documentation, and propose fixes. It must not alter kernel, privilege, boot, update, or recovery controls without explicit authority and independent review.

APPLICABLE EXISTING AGENT ROLES

Architect, Analyst, Researcher, Implementer, Tester, Verifier, Security Auditor, Red Team, Critic, Governance Reviewer, Documentation Writer, Memory Curator, and Optimizer. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Establish platform, hardware, scope, privilege, trust, lifecycle, and non-goals.

2. Specify kernel, service, device, storage, process, identity, update, telemetry, and recovery contracts.
3. Implement with least privilege, isolation, safe defaults, diagnostics, and reproducible builds.
4. Test faults, hostile inputs, upgrades, hardware loss, corruption, recovery, and compatibility.

HUMAN-AI INTERACTION REQUIREMENTS

AI-assisted OS changes remain proposed until human review. Show toolchain, source, target, authority, evidence, and affected assets. Keep offline administration, safe boot, recovery, and evidence access available.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate kernel, services, drivers, firmware, configuration, secrets, hardware, boot artifacts, and logs. Enforce privilege separation, secure update, rollback or recovery, integrity measurement, and compatibility.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show platform, version, boot state, privilege, patch, driver, resource, security, evidence, and recovery state with authorized isolate, restore, update, and rollback controls.

REQUIRED OUTPUTS AND ARTIFACTS

OS architecture and threat model, source, build and signing records, configuration baseline, compatibility matrix, driver inventory, tests, incident and recovery runbooks, update, support, and retirement records.

EVIDENCE REQUIREMENTS

Identify hardware, OS, configuration, privilege, test, environment, result, reviewer, and limitations. A booted host or patch result does not prove secure or reliable operation.

VERIFICATION AND ACCEPTANCE TESTS

Test privilege boundaries, system calls, memory, storage, scheduling, drivers, boot, update, corruption, outage, hostile input, isolation, logging, restore, and hardware compatibility.

FAILURE STATES AND ANTI-PATTERNS

Running as administrator, unsigned updates, shared secrets, hidden driver assumptions, missing recovery media, suppressed kernel errors, unbounded resource use, and destructive rollback that erases evidence.

RECOVERY AND RECONSIDERATION

Isolate affected hosts, preserve forensic and operational evidence, use safe boot or approved prior image, restore or replace hardware, compensate external effects, and reopen after independent review.

COMPLETION CRITERIA

The OS behavior, privilege, platform, security, update, compatibility, recovery, evidence, ownership, support, and retirement boundaries are explicit and verified.

HANDOFF AND CONTINUITY REQUIREMENTS

Hand off platform, source, signed artifacts, configuration, hardware assumptions, test evidence, incidents, decisions, recovery, and open gaps to Kernel, Driver, Infrastructure, Security, Operations, and Maintenance sections.

RELATED CODEX SECTIONS

020 Systems Engineering; 035 Infrastructure Architecture; 044 Runtime Architecture; 045 Security Architecture; 113-115 Kernel, Driver, and Middleware Engineering; 258-299 Configuration through Domain Management; 327-330 Disaster Recovery through Failover.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract and shared governance conventions. Hardware targets, OS family, privilege standards, support window, certification, and recovery objectives remain unresolved until supplied.

114

SECTION 114 - KERNEL ENGINEERING

CODEX/03_SECTIONS/114_KERNEL_ENGINEERING.txt

SHA-256 0100dce5683ef1d1bc11a5b3f32587d0fea4e06d103a21639a3eb0a881718baa

5,545 BYTES / PRESERVED SOURCE IDENTITY

SECTION 114 — Kernel Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define the design, implementation, verification, operation, and retirement of a kernel and its privileged mechanisms for scheduling, memory, interrupts, devices, isolation, and system calls.

REQUIRED OUTCOME

A bounded kernel contract with privilege and threat model, invariants, hardware assumptions, interfaces, performance limits, test evidence, update and recovery path, and explicit acceptance authority.

WHY THIS MATTERS

Kernel defects have system-wide blast radius and can defeat higher-level controls. Privileged code requires stronger evidence, isolation, review, and recovery than ordinary application code.

APPLICABILITY

Applies to kernel code, hypervisor-like privileged layers, schedulers, memory managers, interrupt paths, system-call interfaces, and kernel configuration or extensions.

NON-APPLICABILITY CONDITIONS

A project using an unchanged supported kernel without modifying its privileged behavior or trust boundary is not kernel engineering; record the baseline and trigger.

PREREQUISITES AND DEPENDENCIES

Requires hardware model, privilege and threat model, invariants, ABI, performance and safety goals, source and build provenance, fault model, test environment, update, recovery, and acceptance authority.

REQUIRED INPUTS

Scheduling, memory, interrupts, concurrency, system calls, isolation, devices, boot, fault handling, observability, resource budgets, security properties, tests, and lifecycle constraints.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve privileged behavior, security and safety tradeoffs, release, patch, emergency response, and retirement. Independent review and explicit residual-risk acceptance are required.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect code, draft tests, identify races, and propose changes. It must not modify or release privileged code, claim absence of vulnerabilities, or suppress findings without delegated authority and independent verification.

APPLICABLE EXISTING AGENT ROLES

Architect, Analyst, Researcher, Implementer, Tester, Verifier, Security Auditor, Red Team, Critic, Governance Reviewer, Documentation Writer, Memory Curator, and Optimizer. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define kernel boundary, hardware, privilege, invariants, lifecycle, and non-goals.

2. Specify concurrency, memory, scheduling, calls, isolation, fault, update, and recovery behavior.
3. Implement minimally with review, fuzzing, static analysis, instrumentation, and reproducible signed builds.
4. Test races, faults, corruption, privilege escalation, resource exhaustion, boot, update, rollback, and recovery.

HUMAN-AI INTERACTION REQUIREMENTS

Human approval is required for privilege, isolation, safety, boot, update, and recovery changes. Preserve safe boot, out-of-band control, logs, and manual recovery when AI or tooling fails.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate kernel source, modules, firmware, hardware, boot chain, configuration, symbols, crash data, and signed artifacts. Enforce least privilege, memory safety where possible, isolation, quotas, integrity, and safe update.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show kernel, hardware, boot, privilege, patch, module, fault, resource, evidence, and recovery status. Provide authorized quarantine, stop, capture, restore, and update controls.

REQUIRED OUTPUTS AND ARTIFACTS

Kernel specification, threat model, invariants, source, build and signing records, ABI, test and fuzz results, crash analysis, compatibility, incident, recovery, release, and retirement artifacts.

EVIDENCE REQUIREMENTS

Identify hardware, kernel, configuration, test, fault injection, environment, result, reviewer, and limitations. Passing boot or benchmark evidence does not prove privileged safety.

VERIFICATION AND ACCEPTANCE TESTS

Exercise system calls, races, interrupts, memory corruption, privilege boundaries, devices, resource exhaustion, boot, updates, faults, logging, recovery, and compatibility with independent reproduction.

FAILURE STATES AND ANTI-PATTERNS

Unreviewed privileged code, hidden race conditions, missing symbols, unsafe module loading, disabled security controls, untested recovery, and erasing crash or audit evidence during rollback.

RECOVERY AND RECONSIDERATION

Stop or isolate affected systems, preserve crash and audit evidence, use safe boot or trusted prior image, restore or replace hardware, compensate external effects, and reopen after root-cause review.

COMPLETION CRITERIA

The kernel contract, privilege boundaries, invariants, implementation, evidence, compatibility, update, recovery, support, and retirement are explicit and independently reviewed.

HANDOFF AND CONTINUITY REQUIREMENTS

Hand off kernel source, signed artifacts, ABI, hardware assumptions, crash evidence, decisions, vulnerabilities, recovery, and open gaps to OS, Driver, Security, Infrastructure, Testing, Operations, and Maintenance sections.

RELATED CODEX SECTIONS

108 Computer Science; 113 Operating Systems Engineering; 020 Systems Engineering; 035 Infrastructure Architecture; 045 Security Architecture; 051 Reliability Architecture; 115 Driver Engineering; 246 Security Testing; 327-330 Disaster Recovery through Failover.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract and shared governance conventions. Hardware, certification, memory-safety, real-time, privilege, support, and recovery requirements remain system-specific gaps.

115

SECTION 115 - DRIVER ENGINEERING

CODEX/03_SECTIONS/115_DRIVER_ENGINEERING.txt

SHA-256 6f0952390d466945e69078f5cc01060436bc997d1a80316f8f941db6f58d1691

5,092 BYTES / PRESERVED SOURCE IDENTITY

SECTION 115 — Driver Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define the design, implementation, verification, deployment, operation, update, and retirement of software that controls or exposes hardware devices within explicit safety, security, performance, compatibility, and recovery boundaries.

REQUIRED OUTCOME

A versioned driver contract covering hardware assumptions, discovery, permissions, I/O, timing, errors, isolation, compatibility, tests, update, recovery, and evidence.

WHY THIS MATTERS

Drivers cross the hardware and privileged software boundary. A fault can corrupt data, bypass isolation, damage equipment, or make recovery unavailable.

APPLICABILITY

Applies to kernel, user-space, firmware-adjacent, virtual, bus, device, sensor, actuator, and compatibility drivers.

NON-APPLICABILITY CONDITIONS

Using an unchanged supported driver without altering behavior, configuration, trust boundary, or support claim is not driver engineering; record the baseline.

PREREQUISITES AND DEPENDENCIES

Requires hardware specification, protocol, timing, privilege, threat, safety, lifecycle, test equipment, platform, update, and recovery requirements.

REQUIRED INPUTS

Device behavior, discovery, initialization, I/O, concurrency, errors, timeouts, power, permissions, data integrity, diagnostics, compatibility, and failure model.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve hardware access, privilege, safety, release, field update, residual risk, and retirement. Operators retain isolation and recovery authority.

AI RESPONSIBILITIES AND BOUNDARIES

AI may analyze protocols, draft code and tests, and inspect traces. It must not invent electrical behavior, claim hardware safety, or release a driver without qualified human review and evidence.

APPLICABLE EXISTING AGENT ROLES

Architect, Researcher, Analyst, Implementer, Tester, Verifier, Security Auditor, Red Team, Critic, Governance Reviewer, Documentation Writer, Memory Curator, and Optimizer. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define device, bus, platform, privilege, timing, safety, and recovery scope.

2. Specify discovery, commands, data, synchronization, errors, power, update, and compatibility contracts.
3. Implement isolation, validation, timeouts, diagnostics, and safe defaults.
4. Test normal, malformed, disconnected, noisy, overloaded, hostile, upgraded, and failed hardware.

HUMAN-AI INTERACTION REQUIREMENTS

Identify human, AI, automation, device, and provider actors. Require human review for safety, privilege, field update, and external effects; preserve manual and out-of-band recovery.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate driver, firmware, hardware, configuration, calibration, permissions, logs, and signed artifacts. Enforce least privilege, input validation, timeout, safe state, compatibility, and rollback or restore.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show device, driver, firmware, platform, permission, health, telemetry, evidence, update, and recovery state with authorized isolate, reset, restore, and quarantine controls.

REQUIRED OUTPUTS AND ARTIFACTS

Driver contract, hardware matrix, source, build and signing records, protocol tests, fault injection, compatibility, security and safety review, deployment, field recovery, and retirement records.

EVIDENCE REQUIREMENTS

Identify hardware revision, driver, firmware, configuration, test equipment, inputs, results, reviewer, and limitations. Emulation does not prove field behavior.

VERIFICATION AND ACCEPTANCE TESTS

Test discovery, permissions, I/O, timing, disconnect, corruption, concurrency, power loss, hostile input, hardware variance, update, recovery, and safe-state behavior.

FAILURE STATES AND ANTI-PATTERNS

Unsafe defaults, unchecked device data, privileged access, ignored disconnects, untested hardware revisions, field updates without recovery, and erasing diagnostics during reset.

RECOVERY AND RECONSIDERATION

Isolate device, preserve logs and state, enter approved safe state, use prior driver or manual control, replace hardware if needed, compensate effects, and reopen after independent review.

COMPLETION CRITERIA

Hardware and driver contracts, permissions, safety, compatibility, evidence, support, update, recovery, and retirement are explicit for declared devices and platforms.

HANDOFF AND CONTINUITY REQUIREMENTS

Hand off hardware matrix, source, signed artifacts, tests, calibration, incidents, decisions, recovery, and open gaps to Kernel, OS, Embedded, Hardware, Security, Testing, Operations, and Maintenance sections.

RELATED CODEX SECTIONS

060 Hardware Architecture; 062 Electronics Engineering; 068 Firmware Engineering; 070 Internet of Things Engineering; 113-114 OS and Kernel Engineering; 198 Embedded Security; 232 Hardware Testing; 355 Device Provisioning.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract and shared governance conventions. Hardware revisions, timing, safety certification, field support, update authority, and recovery objectives remain unresolved.

116

SECTION 116 - MIDDLEWARE ENGINEERING

CODEX/03_SECTIONS/116_MIDDLEWARE_ENGINEERING.txt

SHA-256 f9dcd4df884b74d53c1bbd85ef8b7bad58635e161c8e103997eed823ec32dcc1

4,993 BYTES / PRESERVED SOURCE IDENTITY

SECTION 116 — Middleware Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define the engineering of shared middleware that coordinates applications, services, messaging, transactions, identity, serialization, and cross-cutting controls.

REQUIRED OUTCOME

A versioned middleware contract with supported patterns, interfaces, compatibility, security, observability, failure, recovery, performance, and ownership.

WHY THIS MATTERS

Middleware concentrates hidden coupling. A default serializer, retry, transaction, or identity behavior can affect every consumer and make failures systemic.

APPLICABILITY

Applies to brokers, RPC layers, service meshes, transaction managers, serializers, workflow engines, caches, configuration services, and shared libraries.

NON-APPLICABILITY CONDITIONS

An application using an unchanged approved middleware component is not middleware engineering; record the consumed baseline and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires consumer contracts, workload, data, trust model, delivery semantics, failure model, compatibility, capacity, observability, support, and acceptance authority.

REQUIRED INPUTS

Interfaces, schemas, identity, ordering, delivery, retries, idempotency, transactions, timeouts, backpressure, quotas, telemetry, upgrades, and recovery.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve shared defaults, security, compatibility, service commitments, risk, release, and deprecation. Consumer owners remain accountable for local consequences.

AI RESPONSIBILITIES AND BOUNDARIES

AI may draft adapters, schemas, tests, and analyses. It must not change shared semantics, retry or identity policy, or production configuration without delegated approval and evidence.

APPLICABLE EXISTING AGENT ROLES

Architect, Analyst, Researcher, Implementer, Tester, Verifier, Critic, Security Auditor, Governance Reviewer, Optimizer, Documentation Writer, Memory Curator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Inventory consumers and define shared contracts and ownership.

2. Specify delivery, identity, schema, error, retry, transaction, ordering, and compatibility behavior.
3. Implement observability, backpressure, quotas, isolation, and safe degradation.
4. Test failures, duplicates, reordering, partial work, upgrades, and recovery.

HUMAN-AI INTERACTION REQUIREMENTS

Expose middleware and consumer identity, data, policy, authority, version, and outcome. Require human review for shared behavior and preserve direct or manual fallback.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate middleware definition, configuration, runtime state, consumer contract, message or transaction, and evidence. Design idempotency, dead-letter, replay, compatibility, and compensation.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show service, version, consumers, queues, dependency, capacity, failure, evidence, and recovery state with authorized pause, drain, replay, isolate, and rollback controls.

REQUIRED OUTPUTS AND ARTIFACTS

Middleware contracts, schemas, ownership matrix, configuration, compatibility, load and failure tests, security review, observability, migration, runbooks, and retirement records.

EVIDENCE REQUIREMENTS

Identify consumer, message or transaction, version, configuration, workload, environment, result, reviewer, and limitations. Nominal delivery does not prove end-to-end outcome.

VERIFICATION AND ACCEPTANCE TESTS

Test duplicate, loss, delay, reorder, poison data, timeout, retry storm, partition, schema change, unauthorized access, overload, replay, drain, restore, and compensation.

FAILURE STATES AND ANTI-PATTERNS

Unbounded retries, hidden coupling, incompatible schema change, shared credentials, lost messages, unowned dead letters, and rollback that duplicates external effects.

RECOVERY AND RECONSIDERATION

Pause or isolate, preserve messages and evidence, drain or quarantine, use compatible prior behavior, replay or compensate idempotently, and reopen affected consumer contracts.

COMPLETION CRITERIA

Contracts, semantics, ownership, capacity, security, observability, compatibility, recovery, evidence, support, and retirement are explicit and verified for consumers.

HANDOFF AND CONTINUITY REQUIREMENTS

Hand off contracts, schemas, configuration, consumer inventory, evidence, incidents, decisions, migration, recovery, and open gaps to Integration, API, Event, Application, Platform, Security, Testing, and Operations sections.

RELATED CODEX SECTIONS

041-044 Integration, API, Event-Driven, and Runtime Architecture; 045-046 Security and Identity Architecture; 128-131 API through Distributed Systems Engineering; 266-270 CI through DevSecOps; 319-330 Incident through Failover.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract and shared governance conventions. Delivery guarantees, transaction model, broker, capacity, compatibility window, and recovery objectives remain system-specific gaps.

117

SECTION 117 - APPLICATION ENGINEERING

CODEX/03_SECTIONS/117_APPLICATION_ENGINEERING.txt

SHA-256 bcd6e57670e9b6cc21ff0cf19fcbaf7d677a46590f736462ab3fc188714d1573

5,269 BYTES / PRESERVED SOURCE IDENTITY

SECTION 117 — Application Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define the engineering of applications that realize approved user, operator, business, domain, security, data, and operational outcomes through maintainable software.

REQUIRED OUTCOME

An application baseline with behavior, interfaces, data, configuration, dependencies, tests, accessibility, security, deployment, support, recovery, and retirement evidence.

WHY THIS MATTERS

Applications are where abstract requirements meet real users, data, operations, and external effects. Local shortcuts can create systemic harm or unowned lifecycle burden.

APPLICABILITY

Applies to application design, implementation, configuration, integration, testing, release, support, modernization, migration, and retirement.

NON-APPLICABILITY CONDITIONS

Read-only analysis or use of an unchanged application may be exempt when no behavior, data, access, evidence, or lifecycle claim changes; record the baseline.

PREREQUISITES AND DEPENDENCIES

Requires approved requirements, process and functional context, domain rules, architecture, data and interface contracts, risks, users, support, and acceptance authority.

REQUIRED INPUTS

User and operator journeys, use cases, data, integrations, configurations, quality attributes, accessibility, security, tests, deployment, support, incident, and recovery needs.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans own outcome, user impact, risk, privacy, security, accessibility, release, commitments, support, and retirement. They approve exceptions and exact version acceptance.

AI RESPONSIBILITIES AND BOUNDARIES

AI may draft code, tests, documentation, and diagnostics. It must preserve requirements and provenance, expose uncertainty, and never release, alter data, or accept risk without authority.

APPLICABLE EXISTING AGENT ROLES

Architect, Analyst, Researcher, Implementer, Refactorer, Tester, Verifier, Critic, Security Auditor, Governance Reviewer, User Experience Reviewer, Documentation Writer, Memory Curator, Optimizer, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Establish application scope, users, outcomes, environments, lifecycle, and non-goals.
2. Allocate requirements to journeys, modules, data, interfaces, controls, and evidence.
3. Implement accessible, secure, observable behavior with explicit error and recovery paths.
4. Test, release, monitor, support, migrate, patch, deprecate, and retire through controlled change.

HUMAN-AI INTERACTION REQUIREMENTS

Mark human, AI, automated, and external actions. Require review for consequential behavior, permissions, data, communication, and external effects; preserve correction, appeal, manual fallback, and status.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate application source, configuration, data, dependencies, runtime, external integrations, and user state. Enforce contracts, least privilege, migrations, observability, safe rollout, and compensation.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show application version, environment, user impact, dependencies, changes, tests, incidents, evidence, release state, and authorized stop, rollback, restore, and retire controls.

REQUIRED OUTPUTS AND ARTIFACTS

Application plan, architecture, source and configuration, data and interface contracts, test and scan evidence, release, support, runbooks, incidents, recovery, and retirement records.

EVIDENCE REQUIREMENTS

Identify version, configuration, data, environment, actor, method, expected and actual result, reviewer, date, and limitations. Deployment and uptime do not prove user outcome.

VERIFICATION AND ACCEPTANCE TESTS

Trace requirements through user journey, code, data, interface, tests, release, runtime, accessibility, security, recovery, and acceptance; exercise invalid, denied, degraded, duplicate, and migration paths.

FAILURE STATES AND ANTI-PATTERNS

Feature-first delivery without support, hidden data mutation, inaccessible workflows, unowned integrations, weak authorization, missing observability, and rollback that ignores external effects.

RECOVERY AND RECONSIDERATION

Stop or narrow the affected feature, preserve data and evidence, use approved fallback or prior version, restore or compensate, communicate impact, and reopen requirements or architecture when assumptions change.

COMPLETION CRITERIA

Exact application behavior, data, interfaces, controls, quality, evidence, support, recovery, acceptance, and retirement are explicit for the declared release.

HANDOFF AND CONTINUITY REQUIREMENTS

Hand off source, configuration, contracts, tests, release, telemetry, incidents, decisions, support, recovery, and open gaps to Frontend, Backend, Full Stack, Security, Testing, Operations, and Maintenance sections.

RELATED CODEX SECTIONS

019-032 Requirements through Architecture; 033-053 Software through Accessibility Architecture; 117-129 Application through Integration Engineering; 231-270 Testing and Delivery; 301-406 Operations and Lifecycle.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract and shared governance conventions. Domain-specific acceptance, support, privacy, accessibility, security, and service commitments remain unresolved until supplied.

118

SECTION 118 - FRONTEND ENGINEERING

CODEX/03_SECTIONS/118_FRONTEND_ENGINEERING.txt

SHA-256 8e46d4638665a8761252c603b83d1082233ccc5015031171f67a0cf4a9337838

5,610 BYTES / PRESERVED SOURCE IDENTITY

SECTION 118 — Frontend Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define the engineering of user-facing interfaces and client-side behavior that enables accessible, secure, understandable, resilient, and supportable interaction with a product or service.

REQUIRED OUTCOME

A versioned frontend contract covering journeys, state, rendering, input, accessibility, localization, permissions, performance, errors, telemetry, offline or degraded behavior, tests, and release evidence.

WHY THIS MATTERS

The frontend is a control surface, not decoration. A visually correct screen can still mislead, exclude, leak data, lose work, or make recovery impossible.

APPLICABILITY

Applies to web, client, mobile, embedded, terminal, and rendered user or operator interfaces, including state, interaction, accessibility, and client-side integration.

NON-APPLICABILITY CONDITIONS

An unchanged interface consumed without behavior, content, accessibility, permission, data, or evidence change is not frontend engineering; record the baseline.

PREREQUISITES AND DEPENDENCIES

Requires user and operator journeys, functional and content requirements, design system, API contracts, data and privacy rules, accessibility, localization, performance, security, and acceptance authority.

REQUIRED INPUTS

Personas and contexts, journeys, states, content, input and output, APIs, permissions, devices, browsers, languages, assistive technology, performance, errors, telemetry, and recovery.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans own user outcomes, accessibility, content meaning, privacy, security, release, and acceptance. Affected users and accessibility specialists review material changes.

AI RESPONSIBILITIES AND BOUNDARIES

AI may draft components, tests, content, and diagnostics. It must not invent user needs, claim accessibility, expose data, or release changes without human review and evidence.

APPLICABLE EXISTING AGENT ROLES

Architect, Researcher, Analyst, Implementer, Refactorer, Tester, Verifier, User Experience Reviewer, Accessibility specialist through existing role contracts, Security Auditor, Governance Reviewer, Documentation Writer, Memory Curator, and Critic.

REQUIRED ACTIVITIES

1. Define journeys, contexts, states, content, devices, accessibility, security, and non-goals.
2. Implement stable layout, semantic interaction, validation, permissions, loading, errors, empty, offline, and recovery states.
3. Verify keyboard, screen reader, contrast, localization, responsive, performance, security, compatibility, and analytics behavior.
4. Release, monitor, support, correct, and retire through controlled change.

HUMAN-AI INTERACTION REQUIREMENTS

Show human, AI, client, service, and external actions. Require review for consent, permissions, data, denial, communication, and irreversible action; keep non-AI, status, correction, and recovery paths.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate design, content, client state, server state, cache, credentials, telemetry, and rendered output. Enforce authorization server-side, safe caching, progressive failure, and no data loss on refresh or reconnect.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show version, route, state, user context, permission, data freshness, device, accessibility, performance, errors, dependency, evidence, and authorized pause, rollback, and disable controls.

REQUIRED OUTPUTS AND ARTIFACTS

Frontend contract, component and state model, content and accessibility decisions, source, dependency and build records, visual and behavioral tests, performance, security, localization, release, and recovery artifacts.

EVIDENCE REQUIREMENTS

Identify browser or device, version, data, user context, assistive technology, viewport, method, expected and actual result, reviewer, and limitations. Screenshots alone do not prove accessibility or usability.

VERIFICATION AND ACCEPTANCE TESTS

Exercise keyboard and assistive technology, focus, semantics, responsive layout, localization, slow network, offline, stale data, denied permission, malformed input, script failure, reconnect, and destructive confirmation.

FAILURE STATES AND ANTI-PATTERNS

Happy-path-only design, inaccessible custom controls, client-only authorization, hidden loading or error state, lost user input, misleading copy, unsafe cache, and analytics treated as comprehension.

RECOVERY AND RECONSIDERATION

Preserve user input and evidence, degrade to status or manual support, disable unsafe action, restore prior client state, communicate impact, and reopen design or requirements after material accessibility or behavior failure.

COMPLETION CRITERIA

Journeys, states, content, accessibility, permissions, performance, evidence, support, recovery, and acceptance are explicit for declared devices, browsers, users, and release.

HANDOFF AND CONTINUITY REQUIREMENTS

Hand off journeys, components, content, contracts, tests, accessibility evidence, telemetry, incidents, decisions, support, and open gaps to UX, UI, Application, API, Security, Testing, and Operations sections.

RELATED CODEX SECTIONS

075-106 Human-Computer Interaction through User Validation; 117-121 Application through Web Engineering; 128 API Engineering; 103 Accessibility Design; 247 Accessibility Testing; 250 Localization Testing; 314-318 Monitoring and Alerting.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract and shared governance conventions. Target users, devices, browsers, assistive technologies, localization, performance budgets, and accessibility acceptance thresholds remain unresolved until supplied.

119

SECTION 119 - BACKEND ENGINEERING

CODEX/03_SECTIONS/119_BACKEND_ENGINEERING.txt

SHA-256 6a9589be1eaa100b6d5adbedad985c8b1e71c152c058c6a9ce5a9e15a126ed1a

5,028 BYTES / PRESERVED SOURCE IDENTITY

SECTION 119 — Backend Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define engineering of server-side software that provides authorized services, data processing, business rules, interfaces, security controls, and operational behavior.

REQUIRED OUTCOME

A versioned backend contract covering APIs, processes, data, authorization, validation, transactions, observability, scaling, failure, recovery, tests, release, and retirement.

WHY THIS MATTERS

Backend defects can corrupt shared data, expose protected information, or create inconsistent external effects even when the client appears correct.

APPLICABILITY

Applies to services, server applications, workers, jobs, APIs, domain logic, data access, and backend configuration.

NON-APPLICABILITY CONDITIONS

Using an unchanged supported backend without changing behavior, data, access, or lifecycle claims is not backend engineering; record the baseline.

PREREQUISITES AND DEPENDENCIES

Requires requirements, domain and process models, API and data contracts, identity, threat model, workload, recovery, support, and acceptance authority.

REQUIRED INPUTS

Service behavior, endpoints, schemas, authorization, transactions, consistency, jobs, queues, errors, idempotency, migrations, telemetry, capacity, and dependencies.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans own business outcomes, data protection, authorization, risk, release, support, commitments, and retirement. They approve changes with external or irreversible effects.

AI RESPONSIBILITIES AND BOUNDARIES

AI may draft handlers, queries, tests, and diagnostics. It must not invent business rules, access permissions, data meaning, production writes, or acceptance without authority.

APPLICABLE EXISTING AGENT ROLES

Architect, Analyst, Researcher, Implementer, Refactorer, Tester, Verifier, Security Auditor, Governance Reviewer, Critic, Optimizer, Documentation Writer, Memory Curator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define service boundaries, contracts, data, authority, lifecycle, and non-goals.
2. Implement validation, authorization, transactions, idempotency, observability, and failure handling.

3. Test normal, invalid, unauthorized, concurrent, duplicate, degraded, migration, and recovery paths.
4. Release, monitor, support, patch, migrate, and retire with evidence.

HUMAN-AI INTERACTION REQUIREMENTS

Identify human, AI, service, data, and external actors. Require human review for permission, data mutation, policy, and external effects; preserve status, correction, and compensation paths.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate service code, configuration, secrets, data, queues, caches, jobs, runtime state, and external effects. Enforce server-side authorization, idempotency, transaction boundaries, and safe migration.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show service version, endpoint, data, authority, dependency, queue, deployment, evidence, incident, and recovery state with authorized pause, drain, restore, and rollback controls.

REQUIRED OUTPUTS AND ARTIFACTS

Service contracts, domain logic, schemas, migrations, source, tests, scans, deployment, telemetry, runbooks, incident, recovery, support, and retirement artifacts.

EVIDENCE REQUIREMENTS

Identify request, identity, data, version, environment, method, result, side effects, reviewer, and limits. Successful HTTP responses do not prove correct outcome.

VERIFICATION AND ACCEPTANCE TESTS

Trace requirements through endpoint, authorization, data, transaction, tests, release, runtime, and acceptance; exercise retries, duplicates, partial work, dependency loss, and restore.

FAILURE STATES AND ANTI-PATTERNS

Client-only authorization, unbounded retries, non-idempotent jobs, hidden data mutation, unsafe migrations, leaked errors, and rollback that ignores committed external effects.

RECOVERY AND RECONSIDERATION

Stop affected writes, preserve logs and data evidence, use approved read-only or prior behavior, restore or compensate, communicate impact, and reopen contracts when assumptions change.

COMPLETION CRITERIA

Backend behavior, data, access, contracts, evidence, support, recovery, and retirement are explicit and verified for the declared release and environment.

HANDOFF AND CONTINUITY REQUIREMENTS

Hand off source, contracts, schemas, migrations, telemetry, incidents, decisions, runbooks, recovery, and gaps to Application, API, Integration, Database, Security, Testing, and Operations sections.

RELATED CODEX SECTIONS

117 Application Engineering; 119-131 Application through Distributed Systems Engineering; 031 Data Architecture; 042 API Architecture; 045-047 Security, Identity, and Privacy Architecture; 151-161 Data Engineering; 301-330 Operations and Recovery.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract and shared governance conventions. Domain rules, data classifications, service levels, migration limits, and acceptance thresholds remain unresolved until supplied.

120

SECTION 120 - FULL STACK ENGINEERING

CODEX/03_SECTIONS/120_FULL_STACK_ENGINEERING.txt

SHA-256 ed1aee6ec93593afaa826fe1b0b50c1ccac09968a2394e041fb0e4769181fac3

4,894 BYTES / PRESERVED SOURCE IDENTITY

SECTION 120 — Full-Stack Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define coordinated engineering across user interface, client, service, integration, data, deployment, and operational boundaries for an end-to-end product or service.

REQUIRED OUTCOME

A traceable end-to-end baseline connecting journey, client, backend, data, dependencies, security, tests, release, runtime, recovery, and retirement.

WHY THIS MATTERS

Cross-layer failures often hide at boundaries: stale state, mismatched contracts, unauthorized calls, inaccessible recovery, or data changes that no single team sees.

APPLICABILITY

Applies when one work item or team owns material behavior across two or more application layers or when an end-to-end outcome crosses frontend, backend, data, or operations.

NON-APPLICABILITY CONDITIONS

A single-layer change with no cross-layer contract, data, access, behavior, or lifecycle effect may be exempt; record impact and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires end-to-end requirements, journeys, architecture, layer contracts, data, identity, environments, support, recovery, and acceptance authority.

REQUIRED INPUTS

User outcomes, client and server behavior, APIs, schemas, dependencies, configuration, tests, telemetry, accessibility, security, deployment, and recovery.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans own end-to-end outcome, user impact, risk, data, security, release, commitments, and retirement. Layer owners must coordinate rather than transfer unresolved responsibility.

AI RESPONSIBILITIES AND BOUNDARIES

AI may draft cross-layer changes and tests, but must show affected layers, contracts, data, authority, and evidence; it must not release or migrate across boundaries without approval.

APPLICABLE EXISTING AGENT ROLES

Architect, Analyst, Researcher, Implementer, Refactorer, Tester, Verifier, User Experience Reviewer, Security Auditor, Governance Reviewer, Coordinator, Critic, Documentation Writer, and Memory Curator. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Establish end-to-end scope and trace each outcome across layers.
2. Align contracts, state, identity, data, errors, observability, and recovery.
3. Implement and verify normal, boundary, denied, degraded, migration, and retirement paths.
4. Release and operate with coordinated evidence and ownership.

HUMAN-AI INTERACTION REQUIREMENTS

Show layer and actor boundaries, generated content, authority, data, and side effects. Human review is required for cross-layer permissions, migrations, external effects, and acceptance.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Represent each layer separately with explicit contracts, shared identifiers, correlation, version compatibility, data ownership, migrations, tracing, compensation, and safe degradation.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show end-to-end version, layer status, contract compatibility, user impact, data, incidents, evidence, and authorized controls to pause, narrow, rollback, restore, or retire.

REQUIRED OUTPUTS AND ARTIFACTS

End-to-end architecture, contract matrix, source and configuration, test evidence, deployment, telemetry, support, incident, recovery, and handoff package.

EVIDENCE REQUIREMENTS

Identify every layer, version, environment, request, state, data, actor, method, result, and limitation. A layer-local green test is not end-to-end evidence.

VERIFICATION AND ACCEPTANCE TESTS

Exercise contract mismatch, stale client, partial request, duplicate submission, auth failure, data migration, dependency loss, offline recovery, accessibility, and release rollback.

FAILURE STATES AND ANTI-PATTERNS

Layer ownership gaps, incompatible releases, duplicated business rules, client trust, missing correlation, inconsistent transactions, and rollback that breaks another layer.

RECOVERY AND RECONSIDERATION

Freeze affected boundary, preserve correlation and evidence, use compatible versions or manual path, restore or compensate, and reopen impacted architecture and contracts.

COMPLETION CRITERIA

The end-to-end outcome, layer contracts, ownership, evidence, release, support, recovery, and retirement are explicit and accepted for scope.

HANDOFF AND CONTINUITY REQUIREMENTS

Hand off layer contracts, dependencies, tests, telemetry, decisions, incidents, runbooks, recovery, and gaps to Frontend, Backend, API, Integration, Data, Security, Testing, and Operations.

RELATED CODEX SECTIONS

117-119 Application through Backend Engineering; 019-053 Requirements through Architecture; 128-131 API through Distributed Systems; 231-270 Testing and Delivery; 301-406 Operations and Lifecycle.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract and shared governance conventions. Actual layer ownership, deployment topology, service levels, data authority, and recovery objectives remain unresolved.

121

SECTION 121 - WEB ENGINEERING

CODEX/03_SECTIONS/121_WEB_ENGINEERING.txt

SHA-256 455d7430b4730e60e6b078093bd32b7d11e70dafabfd48e8a29b38f2df09373c

5,116 BYTES / PRESERVED SOURCE IDENTITY

SECTION 121 — Web Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define engineering of web applications and services across browser, server, network, content, identity, accessibility, performance, security, and delivery boundaries.

REQUIRED OUTCOME

A versioned web baseline with routes, rendering, content, APIs, state, security, accessibility, compatibility, performance, observability, deployment, recovery, and retirement evidence.

WHY THIS MATTERS

Web systems expose behavior to hostile and diverse environments. Routing, caching, browser state, origin rules, and accessibility failures can undermine otherwise sound services.

APPLICABILITY

Applies to websites, web apps, browser APIs, server-rendered or client-rendered applications, web delivery, and web-specific configuration.

NON-APPLICABILITY CONDITIONS

Unchanged web consumption without a behavior, content, access, compatibility, or lifecycle change may be exempt; record the baseline and trigger.

PREREQUISITES AND DEPENDENCIES

Requires journeys, web and API contracts, browsers, devices, content, identity, privacy, security, accessibility, localization, performance, deployment, and acceptance authority.

REQUIRED INPUTS

Routes, origins, sessions, cookies, forms, rendering, APIs, cache, content, browser support, assistive technology, telemetry, and failure behavior.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans own user outcome, content, consent, privacy, security, accessibility, compatibility, release, and support. They approve material disclosure or destructive interactions.

AI RESPONSIBILITIES AND BOUNDARIES

AI may draft web code, tests, content, and diagnostics; it must not claim security or accessibility, expose secrets, change routes, or release without human review.

APPLICABLE EXISTING AGENT ROLES

Architect, Analyst, Researcher, Implementer, Refactorer, Tester, Verifier, User Experience Reviewer, Security Auditor, Governance Reviewer, Critic, Documentation Writer, and Memory Curator. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define web scope, routes, audiences, browsers, data, trust, and non-goals.
2. Implement semantic accessible interaction, secure sessions, validation, caching, error, offline, and recovery states.
3. Test compatibility, security, performance, localization, accessibility, failure, and deployment paths.
4. Release, monitor, support, and retire with evidence.

HUMAN-AI INTERACTION REQUIREMENTS

Identify browser, human, AI, server, provider, and external actions. Require review for consent, identity, data, external effects, and accessibility; preserve status and non-web support paths.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate browser state, server state, cache, origin, credentials, content, route, API, and deployment. Enforce server authorization, secure cookies, CSP or equivalent controls, cache safety, and graceful failure.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show route, release, browser, session, permission, content, cache, dependency, evidence, incidents, and recovery controls.

REQUIRED OUTPUTS AND ARTIFACTS

Web architecture, route and API contracts, source, dependency and build records, accessibility and security tests, compatibility matrix, performance, deployment, support, and recovery artifacts.

EVIDENCE REQUIREMENTS

Identify browser, route, release, user context, data, method, result, reviewer, and limits. Visual rendering or uptime does not prove secure, accessible, or understandable use.

VERIFICATION AND ACCEPTANCE TESTS

Test origin, session, cookies, authorization, injection, caching, keyboard and assistive technology, browsers, slow network, offline, stale state, localization, and deployment recovery.

FAILURE STATES AND ANTI-PATTERNS

Client-only access control, unsafe caching, inaccessible forms, missing error state, origin confusion, secret exposure, unsupported browsers, and deployment rollback without cache or session handling.

RECOVERY AND RECONSIDERATION

Restrict routes, preserve evidence, invalidate unsafe cache, use maintenance or prior release, restore session and data safely, communicate impact, and reopen after material failure.

COMPLETION CRITERIA

Routes, content, state, APIs, security, accessibility, compatibility, evidence, operations, recovery, and retirement are explicit and accepted for declared web environments.

HANDOFF AND CONTINUITY REQUIREMENTS

Hand off routes, contracts, content, source, tests, security and accessibility evidence, telemetry, incidents, recovery, and gaps to Frontend, Backend, API, Security, Testing, and Operations sections.

RELATED CODEX SECTIONS

118-120 Frontend through Full Stack Engineering; 042 API Architecture; 045-047 Security, Identity, and Privacy Architecture; 103 Accessibility Design; 247 Accessibility Testing; 299 Domain and DNS Management; 300 CDN; 314-318 Observability.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract and shared governance conventions. Browser matrix, web security standard, accessibility level, localization, traffic, cache, and recovery targets remain unresolved.

122

SECTION 122 - DESKTOP ENGINEERING

CODEX/03_SECTIONS/122_DESKTOP_ENGINEERING.txt

SHA-256 68f27c02ff5e37d1862aa86dcf2ca594d0536854a9ffd9ab52e9ecb52e2ed06f

5,076 BYTES / PRESERVED SOURCE IDENTITY

SECTION 122 — Desktop Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define engineering of desktop applications and agents across local OS, filesystem, device, network, identity, update, accessibility, and user-control boundaries.

REQUIRED OUTCOME

A versioned desktop contract covering supported hosts, local data, permissions, offline behavior, updates, compatibility, security, accessibility, diagnostics, recovery, and retirement.

WHY THIS MATTERS

Desktop software operates with local privileges, intermittent connectivity, user-owned data, and diverse host configurations. A failed update or unsafe local action can strand users or expose data.

APPLICABILITY

Applies to desktop GUI, tray, background, command-line, workstation, and local agent software.

NON-APPLICABILITY CONDITIONS

Using an unchanged supported desktop application without altering behavior, permissions, data, update, or support claims may be exempt; record the baseline.

PREREQUISITES AND DEPENDENCIES

Requires host matrix, user journeys, local data, permissions, network, device, update, accessibility, security, support, offline, recovery, and acceptance authority.

REQUIRED INPUTS

Window and input behavior, local storage, files, identity, IPC, devices, network, offline, update, crash recovery, telemetry, accessibility, and uninstall or retirement.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans own local data, consent, permissions, user outcome, support, security, release, and retirement. Users must be able to decline or recover material actions.

AI RESPONSIBILITIES AND BOUNDARIES

AI may draft code, tests, and diagnostics. It must not access local files, execute commands, alter settings, or claim host compatibility without explicit authority and evidence.

APPLICABLE EXISTING AGENT ROLES

Architect, Analyst, Researcher, Implementer, Tester, Verifier, User Experience Reviewer, Security Auditor, Governance Reviewer, Critic, Documentation Writer, and Memory Curator. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define hosts, users, local data, permissions, devices, modes, and non-goals.

2. Implement accessible interaction, least privilege, safe local storage, offline, update, error, and crash recovery.
3. Test host variance, permissions, file loss, network loss, update, uninstall, accessibility, and restore.
4. Release, support, patch, migrate, and retire with evidence.

HUMAN-AI INTERACTION REQUIREMENTS

Show local and remote actors, AI actions, permissions, data scope, and side effects. Require confirmation for destructive local actions and preserve offline status, backup, and recovery.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate application, installer, updater, local data, credentials, OS integration, devices, caches, logs, and recovery media. Use signed updates, atomic migration, least privilege, and safe uninstall.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show host, app, version, permission, update, local data, connectivity, evidence, crash, and recovery status with authorized disable, restore, and rollback controls.

REQUIRED OUTPUTS AND ARTIFACTS

Desktop architecture, host matrix, source and signing records, installer and updater, local data model, accessibility and security tests, diagnostics, support, recovery, and retirement artifacts.

EVIDENCE REQUIREMENTS

Identify host, OS, version, user, data, permissions, device, method, result, reviewer, and limitations. A successful install does not prove safe update or recovery.

VERIFICATION AND ACCEPTANCE TESTS

Test permissions, file corruption, offline, network loss, host variance, update interruption, crash, restore, accessibility, secure storage, IPC, uninstall, and rollback.

FAILURE STATES AND ANTI-PATTERNS

Over-privileged install, hidden local writes, unsafe updater, lost user data, inaccessible recovery, unverified plugins, silent background action, and uninstall that destroys records.

RECOVERY AND RECONSIDERATION

Stop update or unsafe action, preserve local data and logs, use backup or prior version, restore or compensate, provide support path, and reopen compatibility or recovery claims.

COMPLETION CRITERIA

Host support, permissions, local data, updates, accessibility, security, evidence, support, recovery, and retirement are explicit and verified for declared environments.

HANDOFF AND CONTINUITY REQUIREMENTS

Hand off host matrix, installers, source, signing, local data, tests, incidents, decisions, support, recovery, and gaps to Native, Cross-Platform, Security, Testing, Operations, and Maintenance sections.

RELATED CODEX SECTIONS

122-125 Desktop through Native Application Engineering; 045-047 Security through Privacy Architecture; 053 Accessibility Architecture; 247-248 Accessibility and Compatibility Testing; 293 Installation; 378-381 Updates and Patching.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract and shared governance conventions. Supported hosts, local data policy, updater authority, accessibility level, support window, and recovery objectives remain unresolved.

123

SECTION 123 - MOBILE ENGINEERING

CODEX/03_SECTIONS/123_MOBILE_ENGINEERING.txt

SHA-256 7696c4fe1c7e018f9cc3c1853f00a87ace702548e811fa56b8065ee180368cdb

7,217 BYTES / PRESERVED SOURCE IDENTITY

SECTION 123 - Mobile Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define the governed engineering of applications and services used on mobile devices, including device constraints, mobile operating systems, connectivity variation, local state, distribution, accessibility, privacy, security, and lifecycle behavior.

REQUIRED OUTCOME

An approved mobile engineering baseline with platform scope, supported devices and versions, user journeys, offline and recovery behavior, release controls, evidence, ownership, and handoff conditions.

WHY THIS MATTERS

Mobile use occurs in variable networks, physical contexts, battery states, screen sizes, permissions, and device lifecycles. Ignoring those conditions can cause data loss, unsafe interaction, exclusion, privacy leakage, or unmaintainable releases.

APPLICABILITY

Applies when software runs on phones, tablets, wearables, mobile-connected devices, or mobile-specific services and the design depends on mobile hardware, operating-system APIs, app stores, sensors, notifications, intermittent connectivity, or mobile user journeys.

NON-APPLICABILITY CONDITIONS

May be recorded as non-applicable for a server-only or desktop-only artifact with no mobile client, mobile integration, mobile data, or mobile operational consequence. The record must name the scope, owner, evidence date, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires requirements, supported platforms, device and OS policy, threat model, privacy and accessibility obligations, identity and API contracts, data lifecycle, release authority, test environments, and support and retirement ownership.

REQUIRED INPUTS

Provide journeys, device classes, OS versions, network conditions, permissions, data classifications, offline expectations, performance targets, localization, accessibility, store policy, telemetry, support, incident, and recovery requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

The human owner approves supported populations, risk, permissions, data retention, release gates, store distribution, emergency controls, accessibility, privacy, and acceptance. Humans retain authority to pause distribution, revoke access, and retire a version.

AI RESPONSIBILITIES AND BOUNDARIES

AI may propose mobile flows, compatibility matrices, test cases, diagnostics, and implementation changes. It must label assumptions and evidence, avoid inventing device behavior, and never publish, collect sensitive data, change permissions, or release without delegated approval.

APPLICABLE EXISTING AGENT ROLES

Architect, Researcher, Analyst, Security Auditor, Privacy and Governance Reviewer, User Experience Reviewer, Accessibility specialist, Implementer, Tester, Verifier, Documentation Writer, Memory Curator, Coordinator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Establish scope, platforms, devices, environments, users, operating contexts, and lifecycle.
2. Define mobile journeys, permissions, identity, data, connectivity, local storage, notifications, sensors, and recovery.
3. Allocate functions to client, service, device, operating system, and human actors with explicit authority.
4. Design compatibility, accessibility, localization, performance, battery, privacy, security, observability, and support behavior.
5. Build, test, verify, pilot, release, monitor, patch, support, deprecate, and retire under approved gates.

HUMAN-AI INTERACTION REQUIREMENTS

The human must see whether a recommendation, permission request, data operation, release, or incident action is proposed, approved, executed, or verified. AI suggestions remain proposals; consent, denial, correction, stop, appeal, offline status, and recovery must remain available without AI.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Use explicit client, service, device, OS, network, identity, storage, notification, and telemetry boundaries. Treat local data as an authorized cache with expiry and protection. Model sync conflicts, retries, idempotency, upgrades, rollback, deep links, and revoked sessions.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show release identity, platform, device coverage, rollout state, crash and performance evidence, permission status, data freshness, incidents, available pause or rollback actions, and the authority required for each consequential control.

REQUIRED OUTPUTS AND ARTIFACTS

Mobile architecture baseline, device and OS matrix, journey and state models, permission and data-flow records, accessibility and localization plan, security and privacy assessment, release package, test evidence, telemetry plan, support runbook, and retirement record.

EVIDENCE REQUIREMENTS

Evidence identifies build, signing identity, platform, device, OS, network, configuration, data, test method, result, date, and limitations. Crashes, battery, accessibility, security, privacy, store, offline, sync, and recovery evidence remain distinct and failed evidence constrains status.

VERIFICATION AND ACCEPTANCE TESTS

Verify installation, upgrade, downgrade policy, permissions, authentication, offline and reconnect behavior, sync conflicts, interrupted writes, deep links, notifications, accessibility, localization, battery, performance, security, crash recovery, telemetry, rollout, and revocation across declared device classes.

FAILURE STATES AND ANTI-PATTERNS

Unsupported devices presented as supported; assuming connectivity; storing secrets or excessive personal data locally; unbounded background work; inaccessible controls; silent permission escalation; destructive sync; store release without rollback; and treating crash-free rate as user success.

RECOVERY AND RECONSIDERATION

On crash, loss, stale data, revoked authority, unsafe permission, provider failure, or compatibility defect, preserve local evidence, stop risky actions, quarantine affected data, use approved fallback or compensation, communicate impact, and reopen the baseline before resuming.

COMPLETION CRITERIA

Scope, support matrix, contracts, permissions, controls, journeys, failure paths, evidence, release gates, owner acceptance, support, and retirement conditions are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve build and signing identity, source revision, device matrix, decisions, evidence, incidents, open gaps, and next safe action. Hand off to Native, Cross-Platform, Backend, API, Mobile Security, Accessibility, Testing, Operations, Maintenance, and Retirement sections.

RELATED CODEX SECTIONS

118 Frontend Engineering; 119 Backend Engineering; 121 Web Engineering; 124 Cross-Platform Engineering; 125 Native Application Engineering; 197 Mobile Security; 247 Accessibility Testing; 248 Compatibility Testing; 378 Software Updates; 388 Version Management.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 019-020, Sections 107-122, Owner Cockpit Requirements, and Status and Evidence Lexicon. Device inventories, jurisdictional privacy rules, store obligations, accessibility standards, support commitments, and recovery objectives remain unresolved until supplied and verified.

124

SECTION 124 - CROSS PLATFORM ENGINEERING

CODEX/03_SECTIONS/124_CROSS_PLATFORM_ENGINEERING.txt

SHA-256 baa5b24b3ed4b6b6ef386a1629ff699a9affbb44c2ffb01ae4a819faedc801f5

6,377 BYTES / PRESERVED SOURCE IDENTITY

SECTION 124 - Cross-Platform Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering for one product or capability delivered across multiple operating systems, form factors, runtimes, or device classes while preserving approved behavior, accessibility, security, performance, and platform-appropriate interaction.

REQUIRED OUTCOME

A cross-platform baseline with shared contracts, explicit platform variations, parity decisions, build reproducibility, compatibility evidence, ownership, release sequencing, and safe divergence rules.

WHY THIS MATTERS

Shared code reduces duplication but can hide platform differences, inaccessible defaults, lowest-common-denominator design, or a failure that affects every target simultaneously. Explicit variation prevents false parity.

APPLICABILITY

Applies when a shared codebase, framework, component system, generated client, shared service, or common release serves two or more platforms or device families.

NON-APPLICABILITY CONDITIONS

Non-applicability may be recorded for a single-platform implementation with no shared target contract or portability claim. Record the controlling scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires product requirements, platform inventory, supported versions, design and accessibility contracts, shared API and data contracts, build tooling, dependency policy, security model, test matrix, release authority, and support ownership.

REQUIRED INPUTS

Provide target platforms, capability map, parity requirements, allowed variations, performance and accessibility thresholds, localization, permissions, network, data, telemetry, packaging, rollout, and retirement constraints.

HUMAN RESPONSIBILITIES AND AUTHORITY

The human owner decides which behavior must be equal, where platform variation is required, acceptable residual risk, support scope, release order, and acceptance. Humans approve exceptions and may stop a common release.

AI RESPONSIBILITIES AND BOUNDARIES

AI may compare implementations, find divergence, draft adapters and tests, and analyze compatibility. It must not equate compilation with parity, invent platform capabilities, or merge, release, or weaken controls without authorization.

APPLICABLE EXISTING AGENT ROLES

Architect, Analyst, Researcher, User Experience Reviewer, Accessibility specialist, Security Auditor, Implementer, Refactorer, Tester, Verifier, Governance Reviewer, Documentation Writer, Memory Curator, Coordinator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Declare targets, shared scope, platform boundaries, and parity obligations.
2. Classify shared behavior, platform services, adapters, variations, and unsupported capabilities.
3. Define common contracts, feature flags, compatibility rules, accessibility, security, performance, and release sequencing.
4. Test each target and cross-target behavior, including divergence and failure containment.
5. Baseline, release, monitor, correct, and retire with traceable exceptions.

HUMAN-AI INTERACTION REQUIREMENTS

Expose target platform, shared or native path, evidence, confidence, and unresolved differences. Human review is required for parity exceptions, permissions, user harm, security, accessibility, and rollout. AI failure must not remove native status or recovery controls.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate shared domain logic from platform adapters, UI, storage, identity, notifications, sensors, and lifecycle services. Version contracts and feature flags; isolate platform failures; prevent shared dependency defects from silently propagating.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show target matrix, parity state, known differences, build identity, rollout, failed targets, evidence freshness, and controls to pause a target, narrow scope, approve an exception, or roll back within authority.

REQUIRED OUTPUTS AND ARTIFACTS

Cross-platform charter, target matrix, capability and parity map, shared contracts, adapter design, variation register, build records, compatibility and accessibility evidence, release plan, exception register, and continuity handoff.

EVIDENCE REQUIREMENTS

Identify target, version, device, build, dependency, configuration, test path, result, and limitations. Parity claims require comparable scenarios and must separate behavioral, visual, performance, accessibility, and security evidence.

VERIFICATION AND ACCEPTANCE TESTS

Verify shared and platform-specific behavior, feature flags, permissions, storage, networking, accessibility, localization, performance, upgrade, offline, failure isolation, packaging, rollout, rollback, and target-specific recovery.

FAILURE STATES AND ANTI-PATTERNS

Treating shared code as identical behavior; testing only one target; hiding native differences; coupling release timing unnecessarily; using incompatible defaults; common dependency single points of failure; and accepting the weakest platform as universal design.

RECOVERY AND RECONSIDERATION

Quarantine a failing target or shared dependency, preserve prior stable versions, disable affected capabilities, use platform-specific fallback, communicate scope, and reopen parity and risk decisions before restoring common rollout.

COMPLETION CRITERIA

Targets, contracts, variations, controls, evidence, test coverage, release authority, and fallback are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve target matrix, build graph, shared revision, exceptions, evidence, decisions, and next safe action. Hand off to Mobile, Native, Browser, Frontend, API, Testing, Security, Accessibility, Release, Operations, and Maintenance sections.

RELATED CODEX SECTIONS

118 Frontend Engineering; 121 Web Engineering; 123 Mobile Engineering; 125 Native Application Engineering; 126 Browser Engineering; 248 Compatibility Testing; 264 Build Engineering; 287 Release Engineering; 296 Feature Management.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 019-020, Sections 107-123, Owner Cockpit Requirements, and Status and Evidence Lexicon. Exact platform populations, framework support lifetimes, accessibility baselines, vendor constraints, and parity thresholds remain unresolved until supplied.

125

SECTION 125 - NATIVE APPLICATION ENGINEERING

CODEX/03_SECTIONS/125_NATIVE_APPLICATION_ENGINEERING.txt

SHA-256 e7a78a143f655024ab0ac71428baede99b10c3913f28425ad7a8b1f20f4564e8

6,391 BYTES / PRESERVED SOURCE IDENTITY

SECTION 125 - Native Application Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of applications built for a specific operating system or hardware platform, using its native lifecycle, APIs, security model, accessibility services, performance characteristics, and distribution controls.

REQUIRED OUTCOME

An approved native baseline with platform contracts, lifecycle states, permissions, hardware use, accessibility, security, performance, build and signing provenance, release evidence, support, and retirement conditions.

WHY THIS MATTERS

Native APIs provide capability and integration but also create privileged access, vendor coupling, lifecycle drift, and device-specific failure. Treating platform behavior as permanent can compromise users and continuity.

APPLICABILITY

Applies to applications, agents, services, extensions, or device software intentionally implemented against a specific OS, runtime, hardware, or native distribution channel.

NON-APPLICABILITY CONDITIONS

May be non-applicable to platform-neutral services with no native process, API, binary, permission, signing, or hardware dependency. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires platform SDK policy, supported OS and hardware, native API contracts, signing and identity, threat model, permissions, privacy, accessibility, performance, store, update, support, and retirement authority.

REQUIRED INPUTS

Provide platform behaviors, user journeys, hardware and sensor needs, lifecycle callbacks, data and permission flows, offline requirements, performance and battery targets, test devices, release and rollback constraints.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve privileged capability use, signing, distribution, support scope, risk, store claims, update policy, emergency disablement, and acceptance. Humans retain authority over irreversible device or user effects.

AI RESPONSIBILITIES AND BOUNDARIES

AI may draft native code and tests, inspect API changes, and identify misuse. It must not infer permission consent, claim OS behavior without evidence, expose secrets, or sign, publish, or modify device state without authorization.

APPLICABLE EXISTING AGENT ROLES

Architect, Researcher, Analyst, Security Auditor, Privacy Reviewer, User Experience Reviewer, Accessibility specialist, Implementer, Tester, Verifier, Governance Reviewer, Documentation Writer, Memory Curator, Coordinator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Establish platform and hardware scope, users, contexts, lifecycle, and authority.
2. Map native APIs, permissions, data, identity, hardware, lifecycle, and external contracts.
3. Design safe states, background behavior, upgrades, revocation, accessibility, performance, and recovery.
4. Implement with reproducible builds, signed artifacts, review, testing, staged release, monitoring, and support.
5. Requalify on SDK, OS, hardware, dependency, policy, or permission change and retire safely.

HUMAN-AI INTERACTION REQUIREMENTS

Show native API, permission, build, signing, and device context for suggestions and actions. Human approval is required for privileged operations, release, data collection, and user-impacting changes. Preserve manual status and recovery if tools fail.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Use explicit native lifecycle, permission, storage, identity, background, sensor, notification, IPC, signing, update, and hardware boundaries. Validate API availability and failure behavior; isolate vendor services and maintain migration paths.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show platform, OS, device, binary, signing, permission, rollout, crash, performance, security, and update status. Provide authorized pause, disable, revoke, rollback, restore, and retirement controls with clear consequences.

REQUIRED OUTPUTS AND ARTIFACTS

Native architecture baseline, API and permission inventory, data-flow and threat records, lifecycle model, device matrix, reproducible build and signing records, test evidence, release package, support runbook, and retirement plan.

EVIDENCE REQUIREMENTS

Evidence identifies SDK, OS, device, binary hash, signing identity, permissions, configuration, test method, result, date, and limitations. Separate API inspection, device testing, security review, accessibility, performance, crash, and user outcome evidence.

VERIFICATION AND ACCEPTANCE TESTS

Verify installation, signing, permission denial, lifecycle interruption, background limits, storage protection, IPC, sensors, notifications, accessibility, localization, performance, battery, upgrade, rollback, revocation, and device loss.

FAILURE STATES AND ANTI-PATTERNS

Using undocumented APIs; broad permissions; hard-coded vendor assumptions; unbounded background work; trusting local storage; signing from an untracked environment; failing offline; and treating native API availability as user benefit.

RECOVERY AND RECONSIDERATION

Stop affected privileged actions, revoke or narrow permissions, preserve evidence, disable the feature, use approved fallback or prior binary, compensate external effects where needed, communicate impact, and reopen platform qualification.

COMPLETION CRITERIA

Platform scope, APIs, permissions, lifecycle, controls, evidence, build provenance, release, support, and retirement are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve source revision, SDK, signing, binary identity, device evidence, decisions, incidents, exceptions, and next safe action. Hand off to Mobile, Cross-Platform, Security, Accessibility, Testing, Release, Operations, Updates, and Retirement sections.

RELATED CODEX SECTIONS

123 Mobile Engineering; 124 Cross-Platform Engineering; 197 Mobile Security; 203 Cryptography; 204 Key Management; 247 Accessibility Testing; 264 Build Engineering; 287 Release Engineering; 378 Software Updates.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 019-020, Sections 107-124, Owner Cockpit Requirements, and Status and Evidence Lexicon. Platform vendor policies, API lifetimes, hardware populations, support commitments, jurisdictional privacy rules, and recovery objectives remain unresolved until verified.

126

SECTION 126 - BROWSER ENGINEERING

CODEX/03_SECTIONS/126_BROWSER_ENGINEERING.txt

SHA-256 7a8e93087d49c506ef02391d9b6eb8a35a545a2aabcc1a01c97d831f151b02ef

6,417 BYTES / PRESERVED SOURCE IDENTITY

SECTION 126 - Browser Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of browser-executed applications, pages, extensions, and client capabilities across browser engines, web standards, security boundaries, accessibility, storage, networking, and user-controlled execution.

REQUIRED OUTCOME

A browser engineering baseline with supported engines and versions, web contracts, progressive behavior, security and privacy controls, accessibility, performance, compatibility evidence, deployment identity, and recovery paths.

WHY THIS MATTERS

Browsers mediate execution, permissions, storage, rendering, isolation, caching, and network trust. Engine differences, blocked APIs, stale assets, or unsafe client assumptions can harm users and expose systems.

APPLICABILITY

Applies to web applications, browser clients, extensions, workers, WebAssembly, browser APIs, rendering, client storage, service workers, and browser-specific user or operator journeys.

NON-APPLICABILITY CONDITIONS

May be non-applicable to non-browser software with no web client, browser extension, browser API, rendered interface, or browser security consequence. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires web and API requirements, supported engines, standards policy, threat model, identity and session model, content and data rules, accessibility, performance, caching, deployment, telemetry, and support authority.

REQUIRED INPUTS

Provide journeys, browser and device matrix, standards and API use, CSP and origin rules, storage, cookies, workers, offline, localization, accessibility, performance, security, privacy, and rollback requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve security and privacy boundaries, permissions, third-party content, browser support, accessibility, release, incident response, and acceptance. Humans retain authority to disable features or withdraw content.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect compatibility, draft markup and tests, and identify security or accessibility gaps. It must not claim browser support without evidence, weaken origin or content controls, collect data, or deploy changes without authorization.

APPLICABLE EXISTING AGENT ROLES

Architect, Researcher, Analyst, Security Auditor, Privacy and Governance Reviewer, User Experience Reviewer, Accessibility specialist, Implementer, Tester, Verifier, Documentation Writer, Memory Curator, Coordinator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define engines, versions, contexts, users, security origins, and lifecycle.
2. Model navigation, rendering, identity, storage, network, workers, permissions, data, and failure paths.
3. Implement standards-based progressive behavior with controlled dependencies and accessible interaction.
4. Verify security, privacy, compatibility, performance, localization, offline, caching, deployment, and recovery.
5. Stage, monitor, correct, and retire browser assets and capabilities.

HUMAN-AI INTERACTION REQUIREMENTS

Expose browser, engine, origin, permission, asset, cache, and evidence context. Human review is required for trust boundaries, data collection, third-party code, accessibility, and release. Preserve direct status and recovery paths when scripts fail.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate server authority from untrusted client execution. Model origins, CSP, cookies, storage, workers, service-worker cache, rendering, navigation, API, and third-party boundaries. Version assets and support cache invalidation and rollback.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show deployment and asset identity, browser coverage, security headers, permission state, cache freshness, errors, performance, accessibility, incidents, and controls to disable, purge, roll back, or narrow behavior.

REQUIRED OUTPUTS AND ARTIFACTS

Browser baseline, compatibility matrix, standards and origin model, threat and privacy records, accessibility plan, asset and cache strategy, test evidence, deployment manifest, telemetry, support, and retirement package.

EVIDENCE REQUIREMENTS

Identify browser, engine, OS, device, build, asset, network, origin, configuration, test method, result, date, and limits. Separate rendering, interaction, accessibility, security, privacy, performance, cache, and user outcome evidence.

VERIFICATION AND ACCEPTANCE TESTS

Verify engines, keyboard and assistive access, navigation, focus, permissions, origin isolation, CSP, cookies, storage, offline and stale cache, malformed input, network loss, third-party failure, performance, localization, and rollback.

FAILURE STATES AND ANTI-PATTERNS

Assuming one engine; trusting client validation; unsafe inner HTML or third-party code; cache poisoning; inaccessible custom controls; hidden script dependency; leaking tokens to storage; and treating page load or analytics as task success.

RECOVERY AND RECONSIDERATION

Freeze or purge affected assets, revoke sessions or permissions, disable risky client paths, preserve logs and prior versions, use server-side or manual fallback, compensate external effects, and reopen the browser and security baseline.

COMPLETION CRITERIA

Supported engines, standards, security, privacy, accessibility, performance, evidence, deployment, rollback, support, and retirement are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve source revision, asset hashes, browser matrix, headers, decisions, evidence, incidents, open gaps, and next safe action. Hand off to Web, Frontend, API, Security, Privacy, Accessibility, Testing, CDN, Release, and Operations sections.

RELATED CODEX SECTIONS

118 Frontend Engineering; 121 Web Engineering; 127 Extension Engineering; 128 API Engineering; 042 API Architecture; 053 Accessibility Architecture; 190 Application Security; 247 Accessibility Testing; 248 Compatibility Testing; 300 Content Delivery Networks.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 019-020, Sections 107-124, Owner Cockpit Requirements, and Status and Evidence Lexicon. Exact browser populations, standards support, third-party obligations, privacy regimes, accessibility criteria, and incident objectives remain unresolved until supplied and verified.

127

SECTION 127 - EXTENSION ENGINEERING

CODEX/03_SECTIONS/127_EXTENSION_ENGINEERING.txt

SHA-256 8803bf138ed8a7d328099ea3a1238a3bd6ac92e6f8f61023cc1e74395346462f

5,621 BYTES / PRESERVED SOURCE IDENTITY

SECTION 127 - Extension Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of extensions that augment a host application, browser, editor, platform, or device through declared APIs, permissions, lifecycle, isolation, distribution, and user-controlled activation.

REQUIRED OUTCOME

An extension baseline with host contract, permissions, trust boundary, activation, compatibility, update, uninstall, evidence, owner, and recovery conditions.

WHY THIS MATTERS

Extensions inherit host privileges and user trust while depending on changing APIs. Weak permission or lifecycle design can create covert access, persistence, data leakage, or host instability.

APPLICABILITY

Applies to plugins, browser extensions, editor extensions, add-ons, modules, integrations, and package-based augmentations loaded by a host.

NON-APPLICABILITY CONDITIONS

May be non-applicable to a standalone artifact with no host loading, extension API, permission, distribution, or compatibility claim. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires host API and version policy, permission model, threat model, data and privacy rules, signing, packaging, review, sandboxing, compatibility matrix, update, support, and uninstall authority.

REQUIRED INPUTS

Provide extension capabilities, activation events, data flows, host permissions, user journeys, target hosts, versions, resource limits, telemetry, update, disablement, and retirement requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve permissions, host trust, distribution, data collection, release, emergency disablement, and acceptance. The owner can revoke or quarantine an extension without requiring the extension itself.

AI RESPONSIBILITIES AND BOUNDARIES

AI may draft manifests, adapters, tests, and compatibility analysis. It must not broaden permissions, access secrets, publish packages, or imply host safety without evidence and delegated approval.

APPLICABLE EXISTING AGENT ROLES

Architect, Security Auditor, Privacy Reviewer, Analyst, Implementer, Tester, Verifier, Governance Reviewer, User Experience Reviewer, Documentation Writer, Memory Curator, Coordinator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define host, extension, permissions, data, lifecycle, and authority boundaries.
2. Review APIs, activation, isolation, resource use, compatibility, accessibility, and failure behavior.
3. Build signed reproducible packages and test install, update, disablement, uninstall, and host failure.
4. Stage distribution, monitor behavior, respond to incidents, and retire safely.

HUMAN-AI INTERACTION REQUIREMENTS

Show extension identity, host, permissions, activation, version, evidence, and status. Human approval is required for privileged or user-impacting actions; host status and disable controls remain available if AI fails.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Isolate extension code and data, validate host messages, constrain permissions, define activation and shutdown, cap resources, protect secrets, version APIs, and preserve uninstall and rollback paths.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show host and extension versions, permissions, signing, activation, health, data access, rollout, incidents, and controls to approve, disable, revoke, quarantine, update, or uninstall.

REQUIRED OUTPUTS AND ARTIFACTS

Extension charter, host contract, manifest and permission record, threat and privacy assessment, compatibility matrix, signed build evidence, tests, release plan, support runbook, and retirement record.

EVIDENCE REQUIREMENTS

Identify host, extension version, permissions, package hash, signing identity, environment, test method, result, and limitations. Separate API, isolation, privacy, security, usability, performance, and uninstall evidence.

VERIFICATION AND ACCEPTANCE TESTS

Verify least privilege, host validation, activation, shutdown, isolation, data access, update, rollback, disablement, uninstall, host upgrade, offline, malformed input, and accessibility behavior.

FAILURE STATES AND ANTI-PATTERNS

Overbroad permissions; hidden activation; host trust by name alone; persistent secrets; unbounded resource use; unsigned packages; update without rollback; and treating marketplace approval as runtime safety.

RECOVERY AND RECONSIDERATION

Disable or quarantine the extension, revoke permissions, preserve evidence, restore the host, use a prior package or manual path, compensate external effects, and reopen the trust and compatibility baseline.

COMPLETION CRITERIA

Host contract, permissions, lifecycle, evidence, controls, release, support, and retirement are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve host and package identity, manifests, permissions, evidence, incidents, decisions, and next safe action. Hand off to Browser, API, Security, Privacy, Testing, Release, Operations, and Maintenance sections.

RELATED CODEX SECTIONS

126 Browser Engineering; 128 API Engineering; 190 Application Security; 194 Cloud Security; 203 Cryptography; 248 Compatibility Testing; 260 Dependency Management; 287 Release Engineering; 389 Deprecation Management.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 019-020, Sections 107-126, Owner Cockpit Requirements, and Status and Evidence Lexicon. Host-specific permission systems, store rules, support lifetimes, user populations, and incident objectives remain unresolved until verified.

128

SECTION 128 - API ENGINEERING

CODEX/03_SECTIONS/128_API_ENGINEERING.txt

SHA-256 3762db3a227e5ce936ad884f0eb9c4654b53eab001114ec79e59e10eaed63c37

5,792 BYTES / PRESERVED SOURCE IDENTITY

SECTION 128 - API Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of application programming interfaces that expose capabilities, data, commands, events, or decisions to authorized consumers with explicit contracts, identity, authority, compatibility, evidence, and lifecycle.

REQUIRED OUTCOME

A versioned API baseline with purpose, consumers, schemas, errors, security, limits, observability, compatibility, test evidence, owner, change policy, and retirement path.

WHY THIS MATTERS

An API is a durable boundary. Ambiguous semantics, weak authorization, unsafe retries, undocumented limits, or unmanaged versions turn local defects into systemic impact.

APPLICABILITY

Applies to REST, GraphQL, RPC, command, query, event, file, SDK, internal, partner, or machine interfaces that cross a responsibility or trust boundary.

NON-APPLICABILITY CONDITIONS

May be non-applicable to purely internal computation with no caller, boundary, contract, external effect, or integration claim. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires requirements, domain and data models, identity, authorization, threat model, schemas, ownership, SLO or quality expectations, compatibility policy, observability, testing, and lifecycle authority.

REQUIRED INPUTS

Provide use cases, consumers, resources, operations, schemas, identity, permissions, idempotency, pagination, limits, errors, versioning, privacy, audit, availability, recovery, and deprecation requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve semantics, access, data exposure, commitments, compatibility, rate limits, changes, exceptions, and retirement. Humans retain authority over consequential commands and production promotion.

AI RESPONSIBILITIES AND BOUNDARIES

AI may draft schemas, clients, tests, documentation, and diagnostics. It must not invent contracts, expose data, broaden authorization, execute commands, or declare compatibility without evidence and approval.

APPLICABLE EXISTING AGENT ROLES

Architect, Analyst, Security Auditor, Privacy Reviewer, Implementer, Tester, Verifier, Governance Reviewer, Documentation Writer, Coordinator, Memory Curator, Observer, Critic, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define purpose, consumers, authority, data, operations, errors, and lifecycle.
2. Specify schemas, invariants, idempotency, ordering, limits, compatibility, observability, and recovery.
3. Implement contract-first with secure validation, versioned tests, and staged deployment.
4. Monitor use and failures, manage change, deprecate consumers, and retire with notice and fallback.

HUMAN-AI INTERACTION REQUIREMENTS

Show operation intent, identity, authorization, data scope, dry-run status, evidence, and consequence. Human approval is required for material mutations; no conversational route bypasses API controls.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate transport, contract, policy, domain, data, and side-effect boundaries. Define authentication, authorization, replay protection, idempotency, timeout, retry, rate limits, audit, tracing, and compensation.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show API version, owner, consumers, health, error rate, latency, limits, schema changes, authorization decisions, incidents, evidence freshness, and controls for pause, throttle, revoke, rollback, and deprecate.

REQUIRED OUTPUTS AND ARTIFACTS

API charter, contract and schema, security and privacy record, consumer catalog, version and compatibility matrix, test suite, observability plan, change and deprecation records, and runbook.

EVIDENCE REQUIREMENTS

Identify endpoint or operation, version, caller, data, configuration, environment, method, expected and actual result, trace, date, and limits. Keep contract, security, performance, and outcome evidence distinct.

VERIFICATION AND ACCEPTANCE TESTS

Verify authentication, authorization, validation, replay, idempotency, ordering, pagination, limits, errors, retries, timeouts, schema compatibility, audit, tracing, abuse resistance, and compensation for partial effects.

FAILURE STATES AND ANTI-PATTERNS

Undocumented side effects; authorization by UI; unsafe retries; leaking errors; unbounded queries; silent schema breaking; missing tenant isolation; and treating a successful response as successful business outcome.

RECOVERY AND RECONSIDERATION

Throttle or disable affected operations, preserve evidence, revoke credentials, use prior compatible version or manual process, compensate completed side effects, notify consumers, and reopen the contract.

COMPLETION CRITERIA

Purpose, ownership, contract, security, evidence, compatibility, observability, consumers, change, and retirement are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve contract version, schemas, consumers, decisions, traces, test evidence, incidents, open gaps, and next safe action. Hand off to API Architecture, Integration, Security, Data, Testing, Operations, Release, and Deprecation sections.

RELATED CODEX SECTIONS

042 API Architecture; 128 API Engineering; 129 Integration Engineering; 041 Integration Architecture; 019 Requirements Engineering; 045 Security Architecture; 200 Identity and Access Management; 234 Integration Testing; 389 Deprecation Management.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 019-020, Sections 041-042 and 107-127, Owner Cockpit Requirements, and Status and Evidence Lexicon. Sector-specific API standards, legal data constraints, consumer commitments, latency objectives, and retention rules remain unresolved until supplied.

129

SECTION 129 - INTEGRATION ENGINEERING

CODEX/03_SECTIONS/129_INTEGRATION_ENGINEERING.txt

SHA-256 8330e7b350fd7ce362c306c9cc9bd18e1eba3473dabf90c6d5972a51b03d9433

5,637 BYTES / PRESERVED SOURCE IDENTITY

SECTION 129 - Integration Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of connections among systems, services, people, devices, data, organizations, and suppliers, including transformation, orchestration, synchronization, identity, failure, evidence, and lifecycle.

REQUIRED OUTCOME

An integration baseline with participants, contracts, mappings, ownership, ordering, reliability, security, observability, reconciliation, migration, and retirement conditions.

WHY THIS MATTERS

Integration defects cross boundaries and amplify quickly. Missing ownership or reconciliation can create duplicate effects, inconsistent records, silent loss, and impossible recovery.

APPLICABILITY

Applies whenever two or more systems, services, teams, suppliers, devices, data stores, or processes exchange information, commands, events, or responsibility.

NON-APPLICABILITY CONDITIONS

May be non-applicable to isolated computation with no boundary exchange or shared lifecycle consequence. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires participant inventory, domain and data contracts, identity, trust boundaries, mappings, timing, failure and reconciliation policy, ownership, tests, observability, and change authority.

REQUIRED INPUTS

Provide source and target, purpose, data and commands, schemas, transformations, frequency, ordering, duplicates, retries, security, privacy, reconciliation, recovery, and retirement needs.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve cross-boundary authority, data use, side effects, supplier commitments, risk, release, exceptions, and retirement. Humans decide reconciliation outcomes and material compensation.

AI RESPONSIBILITIES AND BOUNDARIES

AI may map schemas, identify dependency gaps, draft adapters, analyze traces, and suggest reconciliation. It must not execute external mutations, approve mappings, or suppress mismatches without authority.

APPLICABLE EXISTING AGENT ROLES

Architect, Analyst, Researcher, Security Auditor, Privacy Reviewer, Implementer, Tester, Verifier, Governance Reviewer, Observer, Coordinator, Documentation Writer, Memory Curator, Critic, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Inventory participants, ownership, trust, data, commands, events, and boundaries.
2. Define contracts, mappings, transformation, ordering, delivery, retry, idempotency, reconciliation, and evidence.
3. Implement with isolation, observability, staged migration, and tested compensation.
4. Reconcile, monitor, change, deprecate, and retire integrations with consumer notice.

HUMAN-AI INTERACTION REQUIREMENTS

Show source, target, identity, authority, payload, transformation, side effect, status, evidence, and reconciliation state. Human review is required for mappings and consequential writes; chat cannot bypass boundary controls.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Model synchronous and asynchronous paths, queues, schemas, adapters, identity, secrets, transactions, retries, deduplication, ordering, dead letters, reconciliation, tracing, and compensation.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show integration health, lag, failures, duplicates, dead letters, mismatches, credentials, versions, data freshness, incidents, and controls to pause, replay, reconcile, isolate, or compensate.

REQUIRED OUTPUTS AND ARTIFACTS

Integration catalog, context and sequence models, contracts and mappings, trust and data-flow assessment, runbooks, reconciliation design, test evidence, migration plan, and retirement record.

EVIDENCE REQUIREMENTS

Identify participants, contract versions, message or request identity, data, transformation, environment, trace, result, date, and limitations. Preserve successful, failed, duplicate, delayed, and unmatched evidence.

VERIFICATION AND ACCEPTANCE TESTS

Verify delivery, ordering, duplicate and replay handling, partial failure, timeout, dead letter, schema change, credential loss, reconciliation, backfill, migration, audit, and compensation.

FAILURE STATES AND ANTI-PATTERNS

Point-to-point sprawl; silent transformation; no reconciliation; retrying non-idempotent effects; shared credentials; ignoring dead letters; and treating message delivery as business completion.

RECOVERY AND RECONSIDERATION

Pause affected flow, preserve queues and evidence, isolate bad data, revoke credentials, reconcile against authoritative records, compensate external effects, and reopen mappings and contracts.

COMPLETION CRITERIA

Participants, contracts, mappings, controls, evidence, reconciliation, recovery, ownership, change, and retirement are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve participant and contract identity, mapping versions, traces, queues, incidents, decisions, evidence, and next safe action. Hand off to Integration Architecture, API, Event, Data, Security, Testing, Operations, and Migration sections.

RELATED CODEX SECTIONS

041 Integration Architecture; 042 API Architecture; 043 Event-Driven Architecture; 128 API Engineering; 130 Microservices Engineering; 157 Data Integration; 234 Integration Testing; 382 Data Migration Operations.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 019-020, Sections 041-043 and 107-128, Owner Cockpit Requirements, and Status and Evidence Lexicon. Actual supplier contracts, data authorities, reconciliation tolerances, recovery objectives, and jurisdictional obligations remain unresolved.

130

SECTION 130 - MICROSERVICES ENGINEERING

CODEX/03_SECTIONS/130_MICROSERVICES_ENGINEERING.txt

SHA-256 f032ac23b119fdb97cbde2c5c845bf448fb80f2e69c65e4eae6dececc68a65e9a

5,770 BYTES / PRESERVED SOURCE IDENTITY

SECTION 130 - Microservices Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of independently deployable services organized around bounded responsibilities, explicit contracts, data ownership, operational authority, resilience, security, observability, and lifecycle.

REQUIRED OUTCOME

A service baseline with bounded contexts, ownership, APIs and events, data authority, deployment, failure, scaling, security, observability, compatibility, and retirement evidence.

WHY THIS MATTERS

Microservices distribute failure and decision-making. Poor boundaries create chatty coupling, inconsistent data, operational overload, and unclear responsibility disguised as flexibility.

APPLICABILITY

Applies when a solution uses multiple independently deployed services, service teams, containers, queues, or provider components with separate lifecycles.

NON-APPLICABILITY CONDITIONS

May be non-applicable to a bounded monolith or single process with no service boundary, independent deployment, or distributed contract. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires domain and capability boundaries, requirements, service ownership, APIs and events, data authority, identity, threat model, deployment, observability, reliability, support, and retirement policy.

REQUIRED INPUTS

Provide service goals, boundaries, dependencies, data, commands, events, SLOs, failure domains, scaling, security, tenancy, migration, test, operational, and compensation requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve service boundaries, ownership, data authority, risk, operational burden, release, exceptions, and retirement. Humans retain authority over cross-service business decisions and external commitments.

AI RESPONSIBILITIES AND BOUNDARIES

AI may propose boundaries, contracts, tests, capacity analysis, and diagnostics. It must not create services for fashion, alter data authority, deploy changes, or hide distributed failure without authorization.

APPLICABLE EXISTING AGENT ROLES

Architect, Analyst, Researcher, Security Auditor, Governance Reviewer, Implementer, Tester, Verifier, Observer, Coordinator, Documentation Writer, Memory Curator, Optimizer, Critic, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define bounded contexts, outcomes, owners, data authority, and contracts.
2. Model calls, events, dependencies, consistency, failure, retries, tracing, deployment, and recovery.
3. Build and release with independent evidence, operational readiness, and migration control.
4. Monitor service and outcome health, manage change, and retire consumers and services safely.

HUMAN-AI INTERACTION REQUIREMENTS

Show service identity, owner, version, dependencies, authority, data, health, evidence, and consequence. Human review is required for boundary, data, and production decisions; stop and recovery controls must survive model failure.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Use explicit bounded contexts, contract versions, data ownership, synchronous and event paths, timeouts, circuit breaking, idempotency, tracing, tenancy, secrets, deployment, and compensation. Avoid distributed transactions unless justified.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show service map, versions, owners, dependencies, SLOs, capacity, incidents, data freshness, deployment state, contract drift, and controls to throttle, isolate, roll back, restore, or retire.

REQUIRED OUTPUTS AND ARTIFACTS

Service catalog, boundary and dependency model, contracts, data ownership, threat assessment, deployment and observability design, resilience tests, runbooks, migration, and retirement package.

EVIDENCE REQUIREMENTS

Identify service, revision, dependency, request or event, data, environment, trace, workload, method, result, and limits. Distinguish service health from end-to-end outcome and preserve partial-failure evidence.

VERIFICATION AND ACCEPTANCE TESTS

Verify contracts, authorization, tenancy, timeout, retry, idempotency, ordering, consistency, dependency loss, scaling, deployment, rollback, tracing, recovery, migration, and retirement.

FAILURE STATES AND ANTI-PATTERNS

Service per table; shared database authority; chatty synchronous chains; unbounded retries; missing timeouts; independent deploy without compatibility; and green service health masking failed outcomes.

RECOVERY AND RECONSIDERATION

Isolate failing services, stop unsafe writes, preserve queues and traces, use approved fallback or prior version, reconcile data, compensate external effects, and reopen boundaries and readiness evidence.

COMPLETION CRITERIA

Boundaries, ownership, contracts, data, controls, operations, evidence, recovery, change, and retirement are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve service and revision identity, maps, contracts, traces, incidents, evidence, decisions, open gaps, and next safe action. Hand off to Distributed Systems, API, Integration, Data, Platform, Reliability, Testing, and Operations sections.

RELATED CODEX SECTIONS

038 Distributed Systems Architecture; 041 Integration Architecture; 042 API Architecture; 130 Microservices Engineering; 131 Distributed Systems Engineering; 243 Reliability Testing; 301 Site Reliability Engineering; 389 Deprecation Management.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 019-020, Sections 038, 041-044 and 107-129, Owner Cockpit Requirements, and Status and Evidence Lexicon. Actual domain boundaries, team ownership, consistency needs, traffic, cost, supplier limits, and recovery objectives remain unresolved.

131

SECTION 131 - DISTRIBUTED SYSTEMS ENGINEERING

CODEX/03_SECTIONS/131_DISTRIBUTED_SYSTEMS_ENGINEERING.txt

SHA-256 bf6f8f5bb61246e7af897148debedf0714c8609ae33c46a1d30d1f73217a4ad7

5,336 BYTES / PRESERVED SOURCE IDENTITY

SECTION 131 - Distributed Systems Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of systems whose state, computation, authority, or outcome spans multiple processes, machines, regions, networks, or organizations.

REQUIRED OUTCOME

A distributed-systems baseline with ownership, consistency, timing, partition, identity, coordination, observability, recovery, and evidence boundaries.

WHY THIS MATTERS

Networks fail, clocks disagree, messages duplicate, and nodes diverge. Designs that assume a single reliable state create data loss, split authority, and unreproducible incidents.

APPLICABILITY

Applies to distributed storage, services, actors, queues, clusters, replicated state, multi-region systems, and federated operations.

NON-APPLICABILITY CONDITIONS

May be non-applicable to a single-process, single-state artifact with no distributed execution or external coordination claim. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires consistency and availability objectives, failure model, topology, clocks, identity, data ownership, contracts, capacity, observability, recovery, security, and operational authority.

REQUIRED INPUTS

Provide nodes, regions, messages, state, ordering, quorum, latency, partitions, retries, duplication, tenancy, security, data, recovery, and user-impact requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve consistency tradeoffs, quorum, data authority, risk, recovery objectives, regions, cost, and acceptance. Humans decide conflict resolution and external communication.

AI RESPONSIBILITIES AND BOUNDARIES

AI may model failures, inspect traces, propose tests, and compare designs. It must not invent guarantees, change consensus or data authority, or execute recovery without delegation.

APPLICABLE EXISTING AGENT ROLES

Architect, Analyst, Observer, Security Auditor, Implementer, Tester, Verifier, Governance Reviewer, Coordinator, Documentation Writer, Memory Curator, Optimizer, Critic, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define topology, state, actors, trust, authority, consistency, timing, and failure model.

2. Design communication, coordination, partition behavior, idempotency, reconciliation, and recovery.
3. Implement with fault injection, tracing, staged rollout, and operational readiness.
4. Measure outcome, repair divergence, migrate, and retire safely.

HUMAN-AI INTERACTION REQUIREMENTS

Expose node, region, leader, quorum, message, state, authority, and evidence context. Human approval is required for failover and conflict decisions; status and safe halt remain available during automation failure.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Model partial failure, clock uncertainty, retries, deduplication, ordering, leader election, quorum, replication, isolation, backpressure, tracing, and compensation. Separate local health from global outcome.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show topology, role, state freshness, quorum, lag, partitions, conflicts, capacity, traces, incidents, and controls to isolate, fail over, reconcile, restore, or narrow operation.

REQUIRED OUTPUTS AND ARTIFACTS

Topology and failure model, consistency contract, data authority, protocol records, test and fault-injection evidence, observability plan, recovery runbook, migration, and retirement package.

EVIDENCE REQUIREMENTS

Identify topology, revision, node, message, clock, workload, configuration, method, trace, result, and limitation. Preserve partition, duplicate, stale, divergent, and recovered evidence.

VERIFICATION AND ACCEPTANCE TESTS

Verify partition, duplication, reordering, clock skew, node loss, leader loss, quorum loss, replay, failover, reconciliation, recovery, migration, security, and user outcome.

FAILURE STATES AND ANTI-PATTERNS

Assuming reliable delivery or clocks; shared mutable state without authority; unbounded retries; split-brain; opaque failover; and treating all replicas healthy as correct outcome.

RECOVERY AND RECONSIDERATION

Freeze risky writes, isolate divergent nodes, preserve logs and state, select approved authority, reconcile and compensate effects, communicate impact, and reopen guarantees before resuming.

COMPLETION CRITERIA

Failure model, guarantees, ownership, controls, evidence, recovery, operations, and retirement are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve topology, protocol, state, traces, incidents, decisions, evidence, and next safe action. Hand off to Distributed Architecture, Microservices, Runtime, Network, Data, Reliability, Resilience, Testing, and Operations.

RELATED CODEX SECTIONS

038 Distributed Systems Architecture; 043 Event-Driven Architecture; 130 Microservices Engineering; 131 Distributed Systems Engineering; 243 Reliability Testing; 244 Resilience Testing; 316 Tracing; 330 Failover Engineering.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 019-020, Sections 038, 043-044 and 107-130, Owner Cockpit Requirements, and Status and Evidence Lexicon. Actual failure assumptions, consistency needs, network domains, capacity, jurisdictions, and recovery objectives remain unresolved.

132

SECTION 132 - REAL TIME SYSTEMS ENGINEERING

CODEX/03_SECTIONS/132_REAL_TIME_SYSTEMS_ENGINEERING.txt

SHA-256 596efcee8c611aefe38ac5d788b26dbb53f2c3feb6746fc46b1436c94f8575bb

5,331 BYTES / PRESERVED SOURCE IDENTITY

SECTION 132 - Real-Time Systems Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of systems whose correctness depends on bounded response time, scheduling, deadlines, jitter, synchronization, or timely interaction with physical or operational processes.

REQUIRED OUTCOME

A real-time baseline with timing classes, workloads, clocks, scheduling, resource budgets, safety margins, evidence, degradation, and recovery ownership.

WHY THIS MATTERS

Late output can be wrong output. Average latency hides deadline misses, priority inversion, overload, clock drift, and unsafe degraded behavior.

APPLICABILITY

Applies to hard, firm, or soft real-time software, control, communications, trading, telemetry, interactive, embedded, or operational systems.

NON-APPLICABILITY CONDITIONS

May be non-applicable where timing has no material correctness, safety, service, or user consequence and no deadline claim is made. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires timing requirements, clock and synchronization model, workload, hardware, scheduler, resource budgets, priority policy, safety and security constraints, measurement, and recovery authority.

REQUIRED INPUTS

Provide deadlines, periods, jitter, latency, load, clock, priority, resource, data, failure, safety, user, observability, and degraded-mode requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve timing class, risk, priority, overload policy, safety margin, release, and acceptance. Humans decide whether a missed deadline is tolerable and when to stop operation.

AI RESPONSIBILITIES AND BOUNDARIES

AI may analyze traces, schedules, workloads, and test scenarios. It must not infer safety from averages, alter priority, claim deadline compliance, or change live scheduling without authority.

APPLICABLE EXISTING AGENT ROLES

Architect, Analyst, Observer, Safety Auditor, Security Auditor, Implementer, Tester, Verifier, Governance Reviewer, Optimizer, Documentation Writer, Coordinator, Memory Curator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Classify timing and define deadlines, clocks, workloads, resources, and failure modes.
2. Design scheduling, synchronization, admission, overload, degradation, monitoring, and recovery.
3. Measure distributions and worst cases under representative and adversarial load.
4. Gate release, monitor deadlines, and retire obsolete timing assumptions.

HUMAN-AI INTERACTION REQUIREMENTS

Show timing class, deadline, current load, clock, authority, evidence, and consequence. Human review is required for safety or material deadline exceptions; a status path remains available when automation fails.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Define clock source, synchronization, scheduling, priority, budget, admission, backpressure, timeout, safe state, and evidence capture. Keep control and observability paths bounded and independent.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show deadline distributions, misses, jitter, clock health, queue depth, CPU and memory, mode, evidence freshness, incidents, and controls to shed work, degrade, pause, or stop.

REQUIRED OUTPUTS AND ARTIFACTS

Timing model, workload and resource budget, scheduler design, measurement plan, deadline evidence, overload and recovery runbook, safety review, release record, and retirement assumptions.

EVIDENCE REQUIREMENTS

Identify hardware, OS, build, clock, workload, configuration, sampling, percentile and worst-case method, result, and limits. Average latency never replaces deadline and miss evidence.

VERIFICATION AND ACCEPTANCE TESTS

Verify clocks, scheduling, priority inversion, overload, queueing, jitter, deadline misses, resource exhaustion, dependency delay, fail-safe mode, recovery, and representative worst-case workload.

FAILURE STATES AND ANTI-PATTERNS

Using averages only; unbounded queues; priority inversion; synchronized timeouts; ignoring clock drift; overload without safe mode; and declaring timing correct from a lab run.

RECOVERY AND RECONSIDERATION

Reduce load, shed noncritical work, enter approved safe mode, preserve timing evidence, stop hazardous outputs, compensate effects where possible, and reopen budgets and acceptance.

COMPLETION CRITERIA

Timing classes, budgets, evidence, controls, safe mode, ownership, and recovery are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve timing baseline, workload, clock, build, traces, misses, decisions, evidence, and next safe action. Hand off to Runtime, Embedded, Control, Network, Performance, Safety, Reliability, Testing, and Operations.

RELATED CODEX SECTIONS

CODEX/03_SECTIONS/132_REAL_TIME_SYSTEMS_ENGINEERING.txt

044 Runtime Architecture; 059 Embedded Systems Architecture; 073 Control Systems Engineering; 132 Real-Time Systems Engineering; 239 Performance Testing; 252 Safety Testing; 323 Performance Management; 324 Reliability Management.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 019-020, Sections 044, 059, 073, and 107-131, Owner Cockpit Requirements, and Status and Evidence Lexicon. Actual deadline classes, hardware, workload, safety regimes, clock sources, and environmental conditions remain unresolved.

133

SECTION 133 - HIGH PERFORMANCE COMPUTING

CODEX/03_SECTIONS/133_HIGH_PERFORMANCE_COMPUTING.txt

SHA-256 e2d275a69dc26384026bd61cf22fcb4817a09aa611514209c9669ea1ef6d3a7a

5,462 BYTES / PRESERVED SOURCE IDENTITY

SECTION 133 - High-Performance Computing

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering for computation requiring substantial throughput, latency, memory, accelerator, energy, or scale optimization while preserving correctness, reproducibility, safety, and authorized resource use.

REQUIRED OUTCOME

An HPC baseline with workload, correctness, resource allocation, scaling, performance evidence, cost and energy limits, reproducibility, operations, and recovery.

WHY THIS MATTERS

Optimization can change numerical results, portability, cost, and fairness of resource access. A fast answer that cannot be reproduced or trusted is not an accepted outcome.

APPLICABILITY

Applies to clusters, supercomputers, accelerators, batch systems, large simulations, analytics, rendering, training, and throughput-critical services.

NON-APPLICABILITY CONDITIONS

May be non-applicable where workload and resource demand are bounded and no performance, scaling, accelerator, or capacity claim is made. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires workload definition, correctness criteria, hardware, scheduler, data, algorithms, numerical tolerance, resource and cost policy, security, reproducibility, and operations authority.

REQUIRED INPUTS

Provide problem size, input data, baseline, target, hardware, parallelism, memory, I/O, energy, cost, precision, failure, checkpoint, and acceptance requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve workload priority, resource allocation, correctness tolerance, cost, energy, data, and acceptance. Humans decide tradeoffs between speed, fidelity, access, and impact.

AI RESPONSIBILITIES AND BOUNDARIES

AI may profile, propose algorithms, optimize code, and draft experiments. It must not trade correctness for speed silently, consume scarce resources, or claim reproducibility without captured evidence.

APPLICABLE EXISTING AGENT ROLES

Architect, Analyst, Researcher, Implementer, Optimizer, Tester, Verifier, Security Auditor, Governance Reviewer, Observer, Documentation Writer, Coordinator, Memory Curator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define workload, correctness, resources, constraints, and baseline.
2. Profile computation, memory, I/O, communication, energy, cost, and bottlenecks.
3. Optimize with controlled experiments, reproducible environments, validation, and resource gates.
4. Operate, checkpoint, recover, compare, and retire workloads and hardware assumptions.

HUMAN-AI INTERACTION REQUIREMENTS

Expose workload, code, compiler, hardware, allocation, data, method, result, and confidence. Human review is required for correctness and scarce-resource decisions; stop and checkpoint controls remain independent of AI.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Represent workload, scheduler, nodes, accelerators, memory, storage, network, checkpoint, result, and evidence boundaries. Isolate jobs, capture environment, and plan for node loss and partial output.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show job, owner, allocation, queue, progress, resource use, performance, cost, energy, checkpoint, evidence, and controls to cancel, suspend, requeue, recover, or release resources.

REQUIRED OUTPUTS AND ARTIFACTS

Workload specification, baseline profile, resource and cost plan, reproducible environment, optimization experiments, correctness evidence, checkpoint design, runbook, and retirement record.

EVIDENCE REQUIREMENTS

Identify code, compiler, hardware, input, seed, precision, scheduler, configuration, measurement, result, uncertainty, and limitations. Separate benchmark, correctness, scalability, energy, and outcome evidence.

VERIFICATION AND ACCEPTANCE TESTS

Verify correctness, numerical tolerance, reproducibility, scaling, resource limits, node loss, checkpoint, restart, data integrity, security isolation, cost, energy, and output acceptance.

FAILURE STATES AND ANTI-PATTERNS

Benchmarking unlike workloads; optimizing a proxy; nonreproducible environments; hidden precision loss; monopolizing resources; ignoring I/O; and treating throughput as useful outcome.

RECOVERY AND RECONSIDERATION

Stop or checkpoint jobs, preserve partial results and environment, requeue safely, validate outputs, release resources, compensate external effects, and reopen performance and correctness claims.

COMPLETION CRITERIA

Workload, correctness, resource, evidence, reproducibility, recovery, ownership, and retirement are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve job and environment identity, inputs, seeds, profiles, outputs, decisions, evidence, incidents, and next safe action. Hand off to Parallel, Scientific, Simulation, Platform, Performance, Capacity, Testing, and Operations.

RELATED CODEX SECTIONS

133 High Performance Computing; 134 Parallel Computing; 135 Scientific Computing; 161 Analytics Engineering; 239 Performance Testing; 242 Scalability Testing; 264 Build Engineering; 321 Capacity Management.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 019-020, Sections 107-132, Owner Cockpit Requirements, and Status and Evidence Lexicon. Actual workloads, hardware, scheduler policy, numerical tolerances, energy limits, data governance, and cost authority remain unresolved.

134

SECTION 134 - PARALLEL COMPUTING

CODEX/03_SECTIONS/134_PARALLEL_COMPUTING.txt

SHA-256 0d986f4aae8820c1925e520063ff1968be98a6f6fc0c646d2c7efd72a591d220

5,244 BYTES / PRESERVED SOURCE IDENTITY

SECTION 134 - Parallel Computing

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of computation divided across concurrent workers, threads, processes, devices, or nodes with explicit decomposition, synchronization, memory, correctness, and failure behavior.

REQUIRED OUTCOME

A parallel-computing baseline with decomposition, data ownership, synchronization, race controls, scalability, reproducibility, resource limits, and recovery evidence.

WHY THIS MATTERS

Concurrency creates races, deadlocks, nondeterminism, contention, and numerical variation. Parallel speedup without correctness or diagnosability is an unsafe claim.

APPLICABILITY

Applies to multithreaded, vectorized, GPU, actor, task, map-reduce, distributed, and concurrent algorithms or services.

NON-APPLICABILITY CONDITIONS

May be non-applicable to sequential computation with no concurrency, parallelism, or scaling claim. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires workload, decomposition, memory and data ownership, synchronization, ordering, determinism, hardware, scheduler, correctness, performance, and failure requirements.

REQUIRED INPUTS

Provide parallel units, dependencies, shared state, synchronization, partitioning, scheduling, memory, precision, resource, performance, failure, and reproducibility constraints.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve determinism, correctness, resource allocation, performance tradeoffs, and acceptance. Humans decide whether nondeterminism or approximate results are permissible.

AI RESPONSIBILITIES AND BOUNDARIES

AI may propose decomposition, identify races, generate tests, and analyze profiles. It must not claim race freedom or alter synchronization and resource policy without review.

APPLICABLE EXISTING AGENT ROLES

Architect, Analyst, Implementer, Optimizer, Tester, Verifier, Observer, Security Auditor, Governance Reviewer, Critic, Documentation Writer, Memory Curator, Coordinator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define dependencies, ownership, ordering, determinism, workload, and correctness.
2. Select decomposition and synchronization; analyze races, deadlocks, contention, and failure.
3. Measure scaling and correctness under controlled and adversarial schedules.
4. Release, monitor, checkpoint, recover, and retire safely.

HUMAN-AI INTERACTION REQUIREMENTS

Expose worker, partition, synchronization, seed, hardware, result, evidence, and consequence. Human review is required for correctness and safety exceptions; status, cancel, and recovery paths remain available.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Make ownership, synchronization, memory visibility, ordering, cancellation, backpressure, checkpoint, and failure domains explicit. Capture schedules and versions sufficient for replay or bounded diagnosis.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show workers, queues, contention, lag, deadlocks, errors, nondeterminism, resource use, output validity, and controls to cancel, drain, checkpoint, isolate, or retry.

REQUIRED OUTPUTS AND ARTIFACTS

Parallel decomposition, ownership, synchronization design, correctness invariants, race and deadlock analysis, reproducible build, performance evidence, fault tests, and recovery runbook.

EVIDENCE REQUIREMENTS

Identify code, runtime, hardware, schedule, seed, inputs, partition, configuration, method, result, and limits. Preserve race, deadlock, nondeterminism, and corrected-run evidence.

VERIFICATION AND ACCEPTANCE TESTS

Verify race, deadlock, ordering, cancellation, failure, replay, scaling, determinism, numerical tolerance, resource isolation, checkpoint, and recovery across schedules and worker counts.

FAILURE STATES AND ANTI-PATTERNS

Assuming sequential reasoning; shared mutable state; lock-free code without proof; unbounded queues; timing-based correctness; hiding nondeterminism; and accepting speedup with changed results.

RECOVERY AND RECONSIDERATION

Stop affected workers, preserve schedules and partial output, restore a known-good synchronization path, validate results, compensate effects, and reopen the parallel correctness baseline.

COMPLETION CRITERIA

Decomposition, ownership, synchronization, correctness, evidence, performance, recovery, and authority are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve code, runtime, schedule, seed, hardware, traces, failures, decisions, evidence, and next safe action. Hand off to HPC, Distributed, Real-Time, Runtime, Testing, Reliability, and Operations.

RELATED CODEX SECTIONS

109 Algorithm Engineering; 112 Runtime Engineering; 131 Distributed Systems Engineering; 133 High Performance Computing; 239 Performance Testing; 242 Scalability Testing; 243 Reliability Testing; 264 Build Engineering.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 019-020, Sections 107-133, Owner Cockpit Requirements, and Status and Evidence Lexicon. Actual hardware, workloads, determinism requirements, numerical tolerances, scheduler behavior, and recovery objectives remain unresolved.

135

SECTION 135 - SCIENTIFIC COMPUTING

CODEX/03_SECTIONS/135_SCIENTIFIC_COMPUTING.txt

SHA-256 665636c2ab2f203c5f1192a23a687796defc522c78c6e96fd291d311447f7d28

5,416 BYTES / PRESERVED SOURCE IDENTITY

SECTION 135 - Scientific Computing

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of computational methods used to represent, analyze, or reproduce scientific phenomena, with explicit assumptions, numerical validity, provenance, uncertainty, and reproducibility.

REQUIRED OUTCOME

A scientific-computing baseline with model purpose, equations or algorithms, inputs, units, numerical methods, uncertainty, validation, reproducible environment, and decision limits.

WHY THIS MATTERS

Scientific code can produce precise-looking but invalid results when assumptions, units, discretization, uncertainty, or provenance are hidden.

APPLICABILITY

Applies to computational science, numerical analysis, research software, scientific workflows, experiment analysis, and domain simulations.

NON-APPLICABILITY CONDITIONS

May be non-applicable to ordinary business computation with no scientific model, numerical claim, experimental inference, or research reproducibility requirement. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires scientific objective, domain authority, model assumptions, data and units, numerical method, precision, validation data, uncertainty, compute environment, provenance, and publication authority.

REQUIRED INPUTS

Provide equations, algorithms, parameters, units, boundary conditions, data sources, tolerances, seeds, workload, validation cases, uncertainty, and intended decisions.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve scientific meaning, assumptions, data use, uncertainty, publication, and decision fitness. Domain experts retain authority over interpretation and limitations.

AI RESPONSIBILITIES AND BOUNDARIES

AI may draft code, summarize literature supplied by the user, propose experiments, and inspect results. It must not invent citations, hide uncertainty, or present a numerical result as scientific truth without review.

APPLICABLE EXISTING AGENT ROLES

Researcher, Analyst, Architect, Implementer, Tester, Verifier, Critic, Governance Reviewer, Documentation Writer, Memory Curator, Optimizer, Security Auditor, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define scientific question, scope, model, data, units, assumptions, and acceptance.

2. Implement and profile numerical methods with provenance and reproducibility.
3. Validate against analytical, experimental, observational, or benchmark evidence.
4. Quantify uncertainty, publish limits, preserve artifacts, and retire invalid methods.

HUMAN-AI INTERACTION REQUIREMENTS

Show model, version, input data, units, method, uncertainty, evidence, and interpretation state. Human review is required for scientific claims and consequential use; AI output remains proposed until verified.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate model, solver, data, parameter, execution, result, visualization, and decision boundaries. Capture units, precision, seeds, environment, provenance, and uncertainty through the pipeline.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show run identity, model, inputs, parameters, code, environment, convergence, uncertainty, validation, result state, and controls to stop, reproduce, compare, or withdraw.

REQUIRED OUTPUTS AND ARTIFACTS

Scientific objective, model record, data and units catalog, code and environment manifest, experiment plan, validation and uncertainty evidence, results, limitations, and reproducibility package.

EVIDENCE REQUIREMENTS

Identify model, code, data, units, parameters, solver, hardware, seed, method, result, uncertainty, and limitations. Separate numerical convergence from empirical validity and scientific interpretation.

VERIFICATION AND ACCEPTANCE TESTS

Verify units, conservation or invariants, convergence, sensitivity, reproducibility, analytical cases, benchmark or experimental comparison, uncertainty, malformed data, and failure recovery.

FAILURE STATES AND ANTI-PATTERNS

False precision; unit mismatch; cherry-picked runs; hidden parameter tuning; invalid extrapolation; unreproducible environments; and treating convergence or visualization as validation.

RECOVERY AND RECONSIDERATION

Preserve inputs and environment, mark results invalid or provisional, rerun with corrected assumptions, withdraw affected claims, compensate decisions where needed, and reopen the scientific baseline.

COMPLETION CRITERIA

Question, model, data, assumptions, method, evidence, uncertainty, interpretation, provenance, and limits are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve code, data, environment, seeds, runs, results, decisions, dissent, evidence, and next safe action. Hand off to Simulation, HPC, Data, Statistics, Research, Testing, and Records sections.

RELATED CODEX SECTIONS

001 Research; 016 Domain Analysis; 109 Algorithm Engineering; 133 High Performance Computing; 135 Scientific Computing; 136 Simulation Engineering; 163 Data Analysis; 165 Statistics; 393 Digital Preservation.

CODEX/03_SECTIONS/135_SCIENTIFIC_COMPUTING.txt

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 001-020, Sections 107-134, Owner Cockpit Requirements, and Status and Evidence Lexicon. Domain-specific standards, datasets, publication rules, uncertainty tolerances, compute environments, and owner acceptance remain unresolved.

136

SECTION 136 - SIMULATION ENGINEERING

CODEX/03_SECTIONS/136_SIMULATION_ENGINEERING.txt

SHA-256 51f8a1e18b8dc95f7daf4d9a18571b88dc4fad03c91f965e20c5775df4ee1bff

5,261 BYTES / PRESERVED SOURCE IDENTITY

SECTION 136 - Simulation Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of simulations that represent systems, environments, processes, or scenarios for analysis, training, prediction, design, or decision support without confusing simulated behavior with reality.

REQUIRED OUTCOME

A simulation baseline with purpose, fidelity, boundaries, assumptions, parameters, scenarios, calibration, validation, uncertainty, controls, and decision-use limits.

WHY THIS MATTERS

A simulation may be internally consistent yet externally wrong. Overconfidence can transfer model error into safety, finance, policy, training, or operational decisions.

APPLICABILITY

Applies to discrete-event, continuous, agent-based, Monte Carlo, training, operational, design, and scenario simulations.

NON-APPLICABILITY CONDITIONS

May be non-applicable to a static illustration or explanatory example that makes no predictive, training, decision, performance, or system-behavior claim. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires simulation purpose, system boundary, fidelity, source data, assumptions, equations or rules, calibration, validation targets, uncertainty, runtime, and decision authority.

REQUIRED INPUTS

Provide scenarios, entities, events, time model, parameters, distributions, initial conditions, outputs, validation data, sensitivity, reproducibility, and use restrictions.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve fitness for use, fidelity, scenarios, assumptions, thresholds, and consequential decisions. Domain owners decide when simulation evidence is admissible.

AI RESPONSIBILITIES AND BOUNDARIES

AI may generate scenarios, explore sensitivity, and summarize runs. It must not invent calibration evidence, suppress outliers, or recommend real-world action beyond delegated scope.

APPLICABLE EXISTING AGENT ROLES

Architect, Researcher, Analyst, Implementer, Tester, Verifier, Critic, Governance Reviewer, Documentation Writer, Memory Curator, Optimizer, Observer, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define purpose, boundary, fidelity, time, entities, scenarios, and decision limits.

2. Build model, calibrate against evidence, conduct sensitivity and uncertainty analysis.
3. Verify implementation, validate behavior, document mismatch, and preserve runs.
4. Review use, monitor drift, update under change control, and retire safely.

HUMAN-AI INTERACTION REQUIREMENTS

Show simulation versus reality, model version, scenario, inputs, assumptions, uncertainty, and use status. Human approval is required before consequential use; AI cannot silently convert a run into an operational command.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate model, simulator, scenario, data, calibration, execution, output, visualization, and decision boundaries. Support deterministic replay, seeds, time controls, provenance, and safe co-simulation.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show model identity, scenario, progress, fidelity, calibration, uncertainty, validation state, output status, and controls to stop, branch, compare, quarantine, or withdraw a run.

REQUIRED OUTPUTS AND ARTIFACTS

Simulation charter, model and scenario catalog, calibration and validation records, experiment plan, uncertainty and sensitivity analysis, run manifest, results, limitations, and continuity package.

EVIDENCE REQUIREMENTS

Identify model, simulator, version, scenario, data, seed, parameters, environment, method, output, uncertainty, and limitations. Simulation evidence remains distinct from observed-world evidence.

VERIFICATION AND ACCEPTANCE TESTS

Verify deterministic replay, conservation or invariants, calibration, validation cases, sensitivity, rare events, time-step effects, malformed inputs, model drift, and safe interpretation.

FAILURE STATES AND ANTI-PATTERNS

Treating a simulation as reality; validating only typical cases; hiding uncertainty; tuning to one dataset; untracked seeds; and using attractive visuals as evidence.

RECOVERY AND RECONSIDERATION

Mark outputs provisional, preserve runs, restrict affected decisions, recalibrate or select an approved prior model, compensate external decisions, and reopen fitness-for-use approval.

COMPLETION CRITERIA

Purpose, fidelity, scenarios, evidence, uncertainty, limits, authority, and recovery are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve models, scenarios, seeds, runs, evidence, decisions, mismatches, and next safe action. Hand off to Scientific, Digital Twin, Game, HPC, Testing, Data, and Operations sections.

RELATED CODEX SECTIONS

135 Scientific Computing; 136 Simulation Engineering; 137 Digital Twin Engineering; 094 Game Design; 239 Performance Testing; 253 Model Testing; 256 Verification; 257 Validation.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 016-020, Sections 107-135, Owner Cockpit Requirements, and Status and Evidence Lexicon. Domain calibration data, fidelity targets, scenario authority, validation populations, and real-world decision rules remain unresolved.

137

SECTION 137 - DIGITAL TWIN ENGINEERING

CODEX/03_SECTIONS/137_DIGITAL_TWIN_ENGINEERING.txt

SHA-256 f817f19751f62e797cab925d6ca2437bc82afb03365ef8c2180a29b754e1a87a

5,478 BYTES / PRESERVED SOURCE IDENTITY

SECTION 137 - Digital Twin Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of a digital representation linked to a physical, organizational, or operational counterpart through identity, telemetry, state, models, controls, and lifecycle evidence.

REQUIRED OUTCOME

A digital-twin baseline with counterpart identity, synchronization, provenance, fidelity, authority, safety boundaries, commands, model drift, and retirement conditions.

WHY THIS MATTERS

A twin can be stale, incomplete, or wrong while appearing live. Confusing representation with authority can cause unsafe commands, privacy exposure, or bad maintenance decisions.

APPLICABILITY

Applies to physical assets, facilities, processes, infrastructure, devices, organizations, or environments represented and linked to a digital system.

NON-APPLICABILITY CONDITIONS

May be non-applicable to an unlinked simulation, dashboard, or static model with no counterpart identity, synchronization, operational use, or command path. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires counterpart authority, identity, telemetry, data ownership, time synchronization, model purpose, command permissions, safety, privacy, security, calibration, operations, and retirement authority.

REQUIRED INPUTS

Provide asset identity, sensors, states, telemetry, models, commands, latency, freshness, calibration, users, decisions, safety limits, and failure or offline requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve counterpart identity, data collection, command authority, safety, privacy, calibration, and acceptance. Humans retain authority over physical or material actions.

AI RESPONSIBILITIES AND BOUNDARIES

AI may detect anomalies, estimate state, and propose maintenance or model updates. It must distinguish telemetry from inference, never issue unapproved commands, and expose stale or missing state.

APPLICABLE EXISTING AGENT ROLES

Architect, Domain Expert, Researcher, Analyst, Safety Auditor, Security Auditor, Implementer, Tester, Verifier, Governance Reviewer, Observer, Documentation Writer, Memory Curator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Identify counterpart, scope, identity, authority, sensors, models, and lifecycle.
2. Define synchronization, provenance, state, calibration, command, safety, privacy, and offline behavior.
3. Validate representation against observed counterpart evidence and test command boundaries.
4. Operate, monitor drift, update, quarantine, and retire with physical owner approval.

HUMAN-AI INTERACTION REQUIREMENTS

Show counterpart identity, live or stale state, source, timestamp, model versus measurement, authority, command consequence, and evidence. Human approval is required for actuation and material maintenance decisions.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate counterpart, telemetry, state estimation, simulation, command, policy, and physical effect. Bind data to identity and time; make freshness, uncertainty, calibration, and safe offline state explicit.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show asset, sync health, freshness, data quality, model drift, alerts, commands, approvals, incidents, and controls to pause, isolate, revoke, revert, or retire.

REQUIRED OUTPUTS AND ARTIFACTS

Twin charter, counterpart catalog, identity and telemetry map, model and calibration records, command authority, safety and privacy assessment, validation evidence, runbook, and retirement plan.

EVIDENCE REQUIREMENTS

Identify counterpart, sensor, timestamp, model, data, configuration, command, authority, environment, result, uncertainty, and limit. Preserve stale, missing, contradictory, and corrected observations.

VERIFICATION AND ACCEPTANCE TESTS

Verify identity, synchronization, freshness, calibration, sensor loss, model drift, unauthorized command, stale state, network loss, safe mode, physical comparison, recovery, and retirement.

FAILURE STATES AND ANTI-PATTERNS

Treating a dashboard as a twin; stale telemetry as live; inferred state as measurement; shared identity; unapproved actuation; and hiding model drift or sensor gaps.

RECOVERY AND RECONSIDERATION

Stop commands, isolate twin and counterpart, preserve telemetry, use local safe controls, reconcile state, compensate effects, notify owners, and reopen calibration and authority decisions.

COMPLETION CRITERIA

Identity, sync, freshness, model, authority, evidence, safety, recovery, and retirement are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve counterpart identity, telemetry, models, commands, approvals, incidents, evidence, decisions, and next safe action. Hand off to IoT, Embedded, Simulation, Control, Safety, Security, Data, and Operations.

RELATED CODEX SECTIONS

070 Internet of Things Engineering; 073 Control Systems Engineering; 135 Scientific Computing; 136 Simulation Engineering; 137 Digital Twin Engineering; 198 Embedded Security; 252 Safety Testing; 355 Device Provisioning.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 016-020, Sections 059, 070, 073, and 107-136, Owner Cockpit Requirements, and Status and Evidence Lexicon. Counterpart ownership, sensor quality, physical safety rules, latency, calibration data, and jurisdictional privacy remain unresolved.

138

SECTION 138 - GAME ENGINEERING

CODEX/03_SECTIONS/138_GAME_ENGINEERING.txt

SHA-256 d700e74bb61652731f90f2a7197f8aa33c95b389ef0057c289a25b6acad2e771

5,688 BYTES / PRESERVED SOURCE IDENTITY

SECTION 138 - Game Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of interactive games and game-like simulations, including deterministic or networked state, input, rendering, audio, progression, safety, accessibility, integrity, distribution, and live operation.

REQUIRED OUTCOME

A game-engineering baseline with authoritative state, client behavior, content and asset provenance, performance, accessibility, privacy, anti-abuse, release, telemetry, recovery, and retirement rules.

WHY THIS MATTERS

Players experience the runtime directly, and small state, timing, network, or accessibility errors can destroy fairness, safety, trust, or continuity.

APPLICABILITY

Applies to games, interactive simulations, game services, engines, clients, servers, multiplayer systems, tools, and live content platforms.

NON-APPLICABILITY CONDITIONS

May be non-applicable to noninteractive media or a static prototype with no gameplay, state, input, player, network, or distribution claim. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires game design, platform targets, player populations, authoritative state model, content rights, privacy, accessibility, safety, networking, moderation, monetization, performance, release, and support authority.

REQUIRED INPUTS

Provide mechanics, inputs, state, physics, timing, content, assets, progression, network, accounts, moderation, accessibility, telemetry, monetization, performance, and recovery requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve player safety, content, fairness, privacy, monetization, moderation, release, and acceptance. Humans retain authority over bans, purchases, resets, and live-service commitments.

AI RESPONSIBILITIES AND BOUNDARIES

AI may generate or test content, diagnose performance, and suggest balancing. It must preserve provenance, avoid unsafe or discriminatory content, and never alter live economy, bans, or player state without authority.

APPLICABLE EXISTING AGENT ROLES

Architect, Game Designer, User Experience Reviewer, Accessibility specialist, Security Auditor, Analyst, Implementer, Tester, Verifier, Governance Reviewer, Documentation Writer, Memory Curator, Coordinator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define player, platform, game state, input, content, network, authority, and lifecycle.
2. Implement deterministic or authoritative behavior, accessible interaction, secure accounts, and recoverable persistence.
3. Test mechanics, performance, compatibility, networking, abuse, content, economy, and recovery.
4. Operate live changes with review, telemetry, rollback, support, and retirement.

HUMAN-AI INTERACTION REQUIREMENTS

Show player, client, server, state, latency, content source, AI involvement, evidence, and consequence. Human approval is required for live content, moderation, purchases, and player-state mutation; status and appeal remain available.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate authoritative state, client prediction, rendering, input, physics, assets, accounts, economy, moderation, telemetry, and platform services. Define replay, rollback, anti-cheat, rate limits, and compensation for interrupted effects.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show build, server state, session, latency, errors, content version, economy, moderation, incidents, accessibility, and controls to pause events, restore state, compensate, or retire content.

REQUIRED OUTPUTS AND ARTIFACTS

Game-engineering charter, state and network model, platform matrix, content and asset provenance, accessibility and safety records, security and abuse model, test evidence, live-ops runbook, and retirement plan.

EVIDENCE REQUIREMENTS

Identify build, client, server, platform, session, inputs, content, configuration, test method, result, and limits. Separate frame rate, network health, mechanic correctness, player outcome, safety, and engagement evidence.

VERIFICATION AND ACCEPTANCE TESTS

Verify input, state authority, save and restore, reconnect, prediction, cheating, moderation, accessibility, content, performance, purchases, economy, migration, and safe shutdown.

FAILURE STATES AND ANTI-PATTERNS

Client-authoritative rewards; inaccessible controls; pay-to-fail recovery; untested netcode; opaque bans; unproven generated content; and treating engagement or frame rate as player wellbeing or fairness.

RECOVERY AND RECONSIDERATION

Pause affected systems, preserve sessions and ledgers, restore authoritative state, compensate valid effects, appeal moderation, communicate impact, and reopen live-change approval.

COMPLETION CRITERIA

State, content, authority, safety, accessibility, evidence, recovery, live change, and retirement are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve build, content, asset rights, state, sessions, decisions, evidence, incidents, and next safe action. Hand off to Game Design, Graphics, Audio, Network, Security, Testing, Operations, Payments, and Retirement.

RELATED CODEX SECTIONS

094 Game Design; 105 Prototyping; 138 Game Engineering; 139 Graphics Engineering; 142 Audio Engineering; 197 Mobile Security; 217 Trust and Safety; 239-249 Testing sections.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 075-106 and 107-137, Owner Cockpit Requirements, and Status and Evidence Lexicon. Game platform rules, player protection obligations, content rights, monetization law, moderation policy, and service objectives remain unresolved.

139

SECTION 139 - GRAPHICS ENGINEERING

CODEX/03_SECTIONS/139_GRAPHICS_ENGINEERING.txt

SHA-256 cd68dca2a322c0578213da8e5e765de0e2dd6f9189d572be016403e6d5055f56

5,184 BYTES / PRESERVED SOURCE IDENTITY

SECTION 139 - Graphics Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of visual computation, assets, pipelines, shaders, geometry, lighting, display, and interaction with explicit correctness, performance, accessibility, provenance, and platform boundaries.

REQUIRED OUTCOME

A graphics baseline with scene, asset, shader, rendering, device, color, performance, accessibility, security, build, evidence, and recovery contracts.

WHY THIS MATTERS

Visual output can be wrong, inaccessible, misleading, or unsafe while appearing polished. GPU and asset failures also create portability, privacy, and resource risks.

APPLICABILITY

Applies to 2D or 3D graphics, real-time visualization, game graphics, UI rendering, GPU compute, asset pipelines, and display output.

NON-APPLICABILITY CONDITIONS

May be non-applicable to systems with no generated, rendered, displayed, or graphics-dependent output. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires visual purpose, target devices, GPU and API support, assets, color and accessibility rules, performance, security, licensing, build, testing, and release authority.

REQUIRED INPUTS

Provide scenes, geometry, materials, shaders, assets, color, resolution, input, frame and memory budgets, accessibility, privacy, and failure requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve visual meaning, accessibility, asset rights, safety, performance tradeoffs, and release. Humans retain authority over public or consequential visual claims.

AI RESPONSIBILITIES AND BOUNDARIES

AI may draft shaders, assets, tests, and profiles. It must preserve provenance and licenses, avoid unsafe or deceptive output, and never ship generated content without review.

APPLICABLE EXISTING AGENT ROLES

Architect, Graphics Engineer, Designer, Researcher, Security Auditor, Accessibility specialist, Implementer, Tester, Verifier, Governance Reviewer, Documentation Writer, Memory Curator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define visual purpose, targets, assets, APIs, color, accessibility, and budgets.
2. Build scene and asset pipelines with provenance, determinism, and fallback.
3. Test rendering, correctness, performance, compatibility, accessibility, and device loss.
4. Release, monitor, optimize, and retire assets and rendering paths.

HUMAN-AI INTERACTION REQUIREMENTS

Show source or generated asset, shader, device, API, frame state, evidence, and confidence. Human review is required for rights, accessibility, and consequential visuals; fallback remains available when GPU or AI fails.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate assets, scene, materials, shader, renderer, display, input, GPU, cache, and content trust boundaries. Version pipelines and preserve software fallback and device-loss recovery.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show build, renderer, device, frame time, memory, visual errors, asset provenance, accessibility, incidents, and controls to disable effects, fall back, purge, or roll back.

REQUIRED OUTPUTS AND ARTIFACTS

Graphics charter, asset catalog and rights, render pipeline, shader records, device matrix, color and accessibility plan, performance evidence, build manifest, and recovery runbook.

EVIDENCE REQUIREMENTS

Identify asset, shader, build, device, API, configuration, test method, result, date, and limits. Separate pixel correctness, visual interpretation, accessibility, performance, security, and provenance evidence.

VERIFICATION AND ACCEPTANCE TESTS

Verify rendering, color, scaling, device loss, shader failure, malformed assets, memory pressure, performance, accessibility, localization, screenshots, and software fallback.

FAILURE STATES AND ANTI-PATTERNS

Unlicensed assets; shader-only correctness; inaccessible contrast; device-specific assumptions; unbounded texture memory; hidden tracking in assets; and treating visual fidelity as semantic or user success.

RECOVERY AND RECONSIDERATION

Disable affected effects, preserve assets and logs, use fallback renderer, remove unsafe or unlicensed content, compensate external use, and reopen the graphics baseline.

COMPLETION CRITERIA

Visual purpose, assets, rights, accessibility, performance, evidence, fallback, and lifecycle are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve assets, licenses, shaders, build, device evidence, decisions, incidents, and next safe action. Hand off to Rendering, Animation, Game, Media, Accessibility, Testing, Build, and Operations.

RELATED CODEX SECTIONS

083 UI Design; 084 Visual Design; 094 Game Design; 138 Game Engineering; 139 Graphics Engineering; 140 Rendering Engineering; 247 Accessibility Testing; 292 Packaging.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 075-106 and 107-138, Owner Cockpit Requirements, and Status and Evidence Lexicon. Device capability, asset rights, color standards, accessibility criteria, performance budgets, and safety obligations remain unresolved.

140

SECTION 140 - RENDERING ENGINEERING

CODEX/03_SECTIONS/140_RENDERING_ENGINEERING.txt

SHA-256 041be8db53533b41da35e058a0aa8cd8bb8b90351275ba0338ae9e7f45367c45

4,918 BYTES / PRESERVED SOURCE IDENTITY

SECTION 140 - Rendering Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of pipelines that transform scene, geometry, data, or content into pixels or other visual output under declared device, timing, color, accessibility, and correctness constraints.

REQUIRED OUTCOME

A rendering baseline with pipeline stages, resource ownership, output contract, device support, performance, fallback, evidence, and recovery.

WHY THIS MATTERS

Rendering failures include wrong ordering, stale resources, color errors, dropped frames, leaks, and nonreproducible GPU behavior. Output must be judged beyond appearance.

APPLICABILITY

Applies to raster, vector, 3D, offline, real-time, server-side, browser, GPU, display, and visualization renderers.

NON-APPLICABILITY CONDITIONS

May be non-applicable where no renderer or generated visual output exists. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires output purpose, renderer and device APIs, scene and asset contracts, color, frame, memory, accessibility, security, test, build, and fallback requirements.

REQUIRED INPUTS

Provide pipeline, resources, inputs, output, resolution, color, timing, memory, device, failure, determinism, accessibility, and comparison requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve output meaning, quality, accessibility, budgets, device support, and release. Humans decide whether degraded rendering is acceptable.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect frames, generate tests, and suggest optimization. It must not claim visual correctness from similarity alone or ship rendering changes without review.

APPLICABLE EXISTING AGENT ROLES

Architect, Graphics Engineer, Analyst, Implementer, Optimizer, Tester, Verifier, Accessibility specialist, Security Auditor, Documentation Writer, Memory Curator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define output and pipeline stages, resources, targets, and budgets.
2. Implement deterministic, instrumented paths with fallback and device-loss handling.

3. Verify pixels, semantics, accessibility, performance, compatibility, and recovery.
4. Release and retire pipeline versions with evidence.

HUMAN-AI INTERACTION REQUIREMENTS

Show pipeline, stage, resource, device, output, evidence, and degraded state. Human approval is required for material visual or accessibility changes; status and software fallback remain available.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate scene submission, scheduling, resource, shader, raster, display, cache, and capture boundaries. Version output contracts and isolate GPU faults.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show renderer, frame timing, memory, queue, device, errors, output quality, accessibility, and controls to lower fidelity, switch path, capture, stop, or roll back.

REQUIRED OUTPUTS AND ARTIFACTS

Renderer contract, pipeline model, device matrix, resource budget, visual and accessibility tests, performance evidence, capture fixtures, build record, and fallback runbook.

EVIDENCE REQUIREMENTS

Identify renderer, stage, device, build, scene, assets, settings, method, result, and limits. Preserve captures, metrics, accessibility findings, and device-specific failures.

VERIFICATION AND ACCEPTANCE TESTS

Verify stage ordering, resource lifetime, shader errors, color, scaling, frame and memory budgets, device loss, malformed assets, accessibility, and fallback.

FAILURE STATES AND ANTI-PATTERNS

Testing one GPU; visual snapshots without semantics; leaking resources; blocking the UI; unsafe shader input; and treating average frame rate as absence of deadline misses.

RECOVERY AND RECONSIDERATION

Switch to safe renderer, lower workload, preserve captures and logs, quarantine assets, restore prior pipeline, compensate visible effects, and reopen performance and correctness claims.

COMPLETION CRITERIA

Pipeline, output, budgets, evidence, compatibility, fallback, and lifecycle are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve renderer and shader revisions, captures, device results, decisions, incidents, evidence, and next safe action. Hand off to Graphics, Game, Browser, Accessibility, Testing, Build, and Operations.

RELATED CODEX SECTIONS

139 Graphics Engineering; 138 Game Engineering; 126 Browser Engineering; 247 Accessibility Testing; 239 Performance Testing; 248 Compatibility Testing; 264 Build Engineering; 300 Content Delivery Networks.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 019-020, Sections 083-084 and 107-139, Owner Cockpit Requirements, and Status and Evidence Lexicon. GPU populations, output standards, capture fixtures, accessibility criteria, and performance limits remain unresolved.

141

SECTION 141 - ANIMATION ENGINEERING

CODEX/03_SECTIONS/141_ANIMATION_ENGINEERING.txt

SHA-256 1f98348c56ac6ba150349da03cc73ee12c292949385da39ae088c8c4426b2adb

4,866 BYTES / PRESERVED SOURCE IDENTITY

SECTION 141 - Animation Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of time-varying visual, spatial, or state transitions with explicit timing, interpolation, interaction, accessibility, performance, provenance, and interruption behavior.

REQUIRED OUTCOME

A baseline with animation state, timing, assets, controls, transitions, cancellation, reduced-motion behavior, device support, evidence, and lifecycle.

WHY THIS MATTERS

Animation communicates state and can cause motion harm, confusion, performance loss, or inaccessible interaction when timing and interruption are not designed.

APPLICABILITY

Applies to UI, games, film, visualization, avatars, motion graphics, simulations, and procedural or generated animation.

NON-APPLICABILITY CONDITIONS

May be non-applicable to static output with no temporal behavior, transition, sequence, or motion claim. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires motion purpose, state model, timing, input, assets, platform, reduced-motion, accessibility, performance, safety, and content-rights requirements.

REQUIRED INPUTS

Provide states, triggers, duration, easing, keyframes, procedural rules, interruption, reduced motion, input, device, and acceptance conditions.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve motion meaning, accessibility, safety, content, and release. Humans decide when motion is optional, prohibited, or consequential.

AI RESPONSIBILITIES AND BOUNDARIES

AI may propose sequences, identify timing defects, and generate tests. It must not ignore reduced-motion needs, copy protected style, or ship generated animation without review.

APPLICABLE EXISTING AGENT ROLES

Architect, Motion Designer, User Experience Reviewer, Accessibility specialist, Implementer, Tester, Verifier, Critic, Governance Reviewer, Documentation Writer, Memory Curator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define motion purpose, state, trigger, timing, interruption, and audience.
2. Implement asset and procedural paths with reduced-motion and low-resource behavior.
3. Verify semantics, timing, accessibility, performance, input, cancellation, and recovery.
4. Release, monitor, and retire animation assets and rules.

HUMAN-AI INTERACTION REQUIREMENTS

Show state, motion purpose, source, duration, user setting, evidence, and consequence. Human review is required for accessibility and safety; users must control or bypass nonessential motion.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Model animation state separately from business state; make triggers, cancellation, interruption, time scaling, reduced motion, frame loss, and resource fallback explicit.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show active animation, state, timing, reduced-motion setting, performance, errors, and controls to pause, skip, slow, disable, or restore.

REQUIRED OUTPUTS AND ARTIFACTS

Animation state and timing model, asset and provenance record, accessibility and reduced-motion plan, test fixtures, performance evidence, release record, and fallback guidance.

EVIDENCE REQUIREMENTS

Identify animation, asset, build, state, device, setting, method, result, and limits. Separate timing, semantic, accessibility, performance, and user-outcome evidence.

VERIFICATION AND ACCEPTANCE TESTS

Verify state transitions, interruption, reduced motion, keyboard or alternative input, frame loss, low power, long duration, localization, and safe fallback.

FAILURE STATES AND ANTI-PATTERNS

Motion without purpose; no skip; flashing or vestibular risk; animation blocking state; timing tied to frame rate; and treating visual smoothness as usability.

RECOVERY AND RECONSIDERATION

Disable or reduce motion, preserve state, use a static representation, communicate status, compensate incomplete transitions, and reopen accessibility and safety review.

COMPLETION CRITERIA

Purpose, state, timing, control, accessibility, evidence, fallback, and lifecycle are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve animation and asset revisions, settings, tests, incidents, decisions, evidence, and next safe action. Hand off to Motion, UI, Game, Graphics, Accessibility, Testing, and Operations.

RELATED CODEX SECTIONS

086 Motion Design; 082 UX Design; 083 UI Design; 138 Game Engineering; 139 Graphics Engineering; 247 Accessibility Testing; 249 Usability Testing; 239 Performance Testing.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 075-106 and 107-140, Owner Cockpit Requirements, and Status and Evidence Lexicon. Motion-safety criteria, reduced-motion populations, platform behavior, content rights, and performance budgets remain unresolved.

142

SECTION 142 - AUDIO ENGINEERING

CODEX/03_SECTIONS/142_AUDIO_ENGINEERING.txt

SHA-256 808b35409ecf7683e59b4b159c580da18dd4db38538c04e3a8208da84222efbd

5,163 BYTES / PRESERVED SOURCE IDENTITY

SECTION 142 - Audio Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of recorded, synthesized, spatial, interactive, or streamed sound with explicit content provenance, timing, accessibility, performance, privacy, and user control.

REQUIRED OUTCOME

A sound baseline with sources, assets, mixing, spatialization, timing, device support, captions or alternatives, privacy, evidence, and recovery.

WHY THIS MATTERS

Audio can convey essential state, expose private context, harm hearing, or fail silently. Audio quality alone does not establish comprehension or accessibility.

APPLICABILITY

Applies to games, applications, devices, media, voice interfaces, spatial audio, alerts, synthesis, recording, and audio pipelines.

NON-APPLICABILITY CONDITIONS

May be non-applicable to systems with no audio generation, capture, playback, or audio-dependent outcome. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires audio purpose, assets and rights, timing, device, volume, hearing accessibility, captions, recording consent, privacy, performance, security, and lifecycle policy.

REQUIRED INPUTS

Provide sources, formats, levels, channels, spatial context, triggers, latency, accessibility alternatives, permissions, recording, retention, and failure requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve content, rights, recording, privacy, safety, accessibility, volume, and release. Humans decide whether audio is essential or optional.

AI RESPONSIBILITIES AND BOUNDARIES

AI may generate, classify, mix, transcribe, or test audio. It must preserve consent and provenance, expose uncertainty, and never publish or record without authority.

APPLICABLE EXISTING AGENT ROLES

Audio Engineer, Architect, User Experience Reviewer, Accessibility specialist, Security Auditor, Privacy Reviewer, Implementer, Tester, Verifier, Governance Reviewer, Documentation Writer, Memory Curator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define sound purpose, sources, rights, timing, devices, levels, accessibility, and privacy.
2. Build capture, processing, mixing, spatial, playback, and alternative paths.
3. Verify content, levels, latency, intelligibility, captions, permissions, performance, and recovery.
4. Release, monitor, correct, and retire audio assets and pipelines.

HUMAN-AI INTERACTION REQUIREMENTS

Show source, consent, asset, device, level, spatial state, transcript or alternative, evidence, and consequence. Human approval is required for recording, publication, and safety-impacting alerts.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate capture, processing, storage, mixer, output, spatial context, device, transcript, and consent boundaries. Apply limits, mute, fallback, and data deletion controls.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show audio source, session, levels, devices, consent, latency, errors, captions, incidents, and controls to mute, stop capture, lower level, switch output, or delete recordings.

REQUIRED OUTPUTS AND ARTIFACTS

Audio charter, asset and rights catalog, signal path, consent and privacy record, level and safety plan, accessibility alternatives, test evidence, build package, and retention or retirement record.

EVIDENCE REQUIREMENTS

Identify source, asset, license, build, device, signal settings, consent, method, result, and limits. Separate fidelity, intelligibility, accessibility, safety, privacy, and user outcome evidence.

VERIFICATION AND ACCEPTANCE TESTS

Verify capture consent, levels, mute, device loss, latency, spatial cues, malformed media, captions, alternative signals, privacy, offline, and deletion behavior.

FAILURE STATES AND ANTI-PATTERNS

Recording without consent; unsafe levels; audio-only critical information; unlicensed assets; hidden microphone use; inaccessible alerts; and treating loudness or fidelity as comprehension.

RECOVERY AND RECONSIDERATION

Stop capture and playback, preserve required evidence, delete or quarantine unauthorized recordings, use visual or haptic alternative, compensate affected parties, and reopen consent or safety review.

COMPLETION CRITERIA

Purpose, source, rights, consent, controls, accessibility, evidence, recovery, and lifecycle are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve assets, licenses, consent, signals, builds, incidents, decisions, evidence, and next safe action. Hand off to Game, Media, Voice, Accessibility, Privacy, Security, Testing, and Operations.

RELATED CODEX SECTIONS

087 Sound Design; 088 Voice Interface Design; 138 Game Engineering; 142 Audio Engineering; 143 Video Engineering; 247 Accessibility Testing; 250 Localization Testing; 395 Data Deletion.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 075-106 and 107-141, Owner Cockpit Requirements, and Status and Evidence Lexicon. Actual content rights, consent rules, hearing-safety standards, devices, accessibility alternatives, and retention obligations remain unresolved.

143

SECTION 143 - VIDEO ENGINEERING

CODEX/03_SECTIONS/143_VIDEO_ENGINEERING.txt

SHA-256 6d01824c15d3d474cf644bf6b055cd32330a837a474041edb152160a84caa7ec

5,194 BYTES / PRESERVED SOURCE IDENTITY

SECTION 143 - Video Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of captured, encoded, processed, delivered, decoded, and displayed moving-image content with explicit provenance, quality, accessibility, privacy, security, and lifecycle controls.

REQUIRED OUTCOME

A video baseline with source, rights, codecs, timing, quality, captions, delivery, storage, processing, evidence, recovery, and retirement.

WHY THIS MATTERS

Video can mislead through edits, missing context, stale frames, inaccessible delivery, or provenance loss. Availability and resolution do not prove truth or comprehension.

APPLICABILITY

Applies to capture, editing, encoding, playback, conferencing, surveillance, broadcast, computer vision input, and video delivery systems.

NON-APPLICABILITY CONDITIONS

May be non-applicable to systems with no video capture, processing, delivery, display, or video-dependent outcome. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires purpose, sources, consent, rights, codecs, timing, quality, captions, privacy, security, storage, delivery, accessibility, and retention authority.

REQUIRED INPUTS

Provide source, frame rate, resolution, color, audio relation, edits, metadata, captions, latency, network, device, retention, and failure requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve content, consent, rights, surveillance, publication, accessibility, safety, and retention. Humans decide whether video is fit for evidence or decision use.

AI RESPONSIBILITIES AND BOUNDARIES

AI may transcode, caption, summarize, detect defects, or analyze supplied video. It must preserve provenance, expose uncertainty, and never publish or infer consequential facts without review.

APPLICABLE EXISTING AGENT ROLES

Video Engineer, Architect, Researcher, Privacy Reviewer, Security Auditor, Accessibility specialist, Implementer, Tester, Verifier, Governance Reviewer, Documentation Writer, Memory Curator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define purpose, source, rights, consent, quality, accessibility, pipeline, and lifecycle.
2. Build capture, edit, encode, deliver, playback, caption, and archive paths.
3. Verify integrity, quality, timing, accessibility, privacy, security, and recovery.
4. Operate, monitor, correct, withdraw, and retire video assets and services.

HUMAN-AI INTERACTION REQUIREMENTS

Show source, edit history, model or tool, timestamp, consent, caption state, quality, and evidence. Human approval is required for publication and consequential interpretation; alternative access remains available.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate capture, storage, edit, encode, manifest, delivery, playback, caption, analytics, and archive boundaries. Preserve hashes, time, lineage, access, deletion, and safe fallback.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show asset, source, version, delivery state, latency, quality, captions, permissions, incidents, and controls to stop capture, restrict access, withdraw, restore, or delete.

REQUIRED OUTPUTS AND ARTIFACTS

Video charter, source and rights catalog, media pipeline, edit and provenance record, caption and accessibility plan, quality evidence, delivery manifest, support, and retention package.

EVIDENCE REQUIREMENTS

Identify source, asset, edit, codec, build, timestamp, device, method, result, and limits. Separate media quality, integrity, accessibility, privacy, security, and interpretation evidence.

VERIFICATION AND ACCEPTANCE TESTS

Verify consent, integrity, edit lineage, timestamps, codecs, captions, quality, sync, network loss, authorization, malformed media, retention, deletion, and recovery.

FAILURE STATES AND ANTI-PATTERNS

Unmarked edits; missing captions; capture without consent; insecure manifests; inaccessible controls; codec lock-in; and treating clear video or detection output as truth.

RECOVERY AND RECONSIDERATION

Stop capture or publication, preserve chain of custody, restrict or withdraw content, provide alternate access, compensate affected decisions, and reopen provenance and retention review.

COMPLETION CRITERIA

Purpose, source, rights, consent, provenance, accessibility, evidence, delivery, recovery, and lifecycle are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve asset hashes, source, edits, captions, permissions, decisions, incidents, evidence, and next safe action. Hand off to Media, Streaming, Search, Privacy, Security, Accessibility, Testing, and Operations.

RELATED CODEX SECTIONS

142 Audio Engineering; 143 Video Engineering; 144 Media Engineering; 145 Streaming Engineering; 146 Search Engineering; 218 Privacy Engineering; 247 Accessibility Testing; 392 Data Archival.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 075-106 and 107-142, Owner Cockpit Requirements, and Status and Evidence Lexicon. Actual content rights, consent, chain-of-custody rules, codec populations, accessibility standards, and retention duties remain unresolved.

144

SECTION 144 - MEDIA ENGINEERING

CODEX/03_SECTIONS/144_MEDIA_ENGINEERING.txt

SHA-256 fe6dc7de69fbaa5e7120afeebb0895d36e86e597c5e4b585574c7062d2c90ea7

5,098 BYTES / PRESERVED SOURCE IDENTITY

SECTION 144 - Media Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of systems that create, transform, manage, deliver, and preserve coordinated text, image, audio, video, and interactive media.

REQUIRED OUTCOME

A media baseline with asset identity, rights, provenance, synchronization, formats, accessibility, delivery, quality, security, evidence, and lifecycle.

WHY THIS MATTERS

Media pipelines multiply format, rights, synchronization, and preservation risks. A valid file can still be misleading, inaccessible, stale, or unlawfully used.

APPLICABILITY

Applies to media repositories, production, editing, publishing, digital experiences, asset pipelines, archives, and multi-format delivery.

NON-APPLICABILITY CONDITIONS

May be non-applicable to systems with no media asset, format, delivery, display, archive, or content-dependent outcome. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires asset purpose, owners, rights, consent, formats, metadata, provenance, synchronization, accessibility, privacy, security, quality, delivery, and retention policy.

REQUIRED INPUTS

Provide asset types, sources, versions, transformations, relationships, timing, captions, translations, delivery, access, retention, and failure requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve rights, consent, publication, accessibility, privacy, retention, safety, and acceptance. Humans decide withdrawal and correction for consequential media.

AI RESPONSIBILITIES AND BOUNDARIES

AI may classify, transform, caption, translate, search, and generate media drafts. It must preserve source and rights, label transformation, and never publish or delete without authority.

APPLICABLE EXISTING AGENT ROLES

Architect, Researcher, Media Engineer, Designer, Privacy Reviewer, Security Auditor, Accessibility specialist, Implementer, Tester, Verifier, Governance Reviewer, Documentation Writer, Memory Curator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Inventory assets, owners, rights, sources, formats, relationships, and lifecycle.
2. Design ingest, transform, synchronize, deliver, search, archive, correction, and deletion paths.
3. Verify provenance, quality, accessibility, security, privacy, rights, and recovery.
4. Operate, publish, withdraw, preserve, and retire with traceable decisions.

HUMAN-AI INTERACTION REQUIREMENTS

Show asset, source, transformation, rights, audience, provenance, accessibility, evidence, and state. Human review is required for publication, generation, licensing, and deletion.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate source, derivative, metadata, rights, transformation, delivery, cache, archive, and user presentation. Preserve lineage and format migration without overwriting source.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show asset graph, version, rights, delivery, quality, access, captions, incidents, evidence, and controls to approve, correct, restrict, withdraw, migrate, archive, or delete.

REQUIRED OUTPUTS AND ARTIFACTS

Media charter, asset catalog, rights and consent records, provenance graph, format policy, accessibility plan, pipeline and delivery design, test evidence, and preservation package.

EVIDENCE REQUIREMENTS

Identify asset, source, derivative, rights, version, transformation, tool, configuration, method, result, and limits. Preserve failed, stale, superseded, and withdrawn evidence.

VERIFICATION AND ACCEPTANCE TESTS

Verify ingest, lineage, rights, format conversion, synchronization, captions, search, access, cache, corruption, restoration, migration, withdrawal, and deletion.

FAILURE STATES AND ANTI-PATTERNS

Overwriting masters; missing rights; unlabeled transformations; inaccessible media; format monoculture; unbounded storage; and treating popularity or fidelity as truth.

RECOVERY AND RECONSIDERATION

Quarantine affected assets, preserve source and lineage, restrict distribution, restore a prior derivative, compensate decisions, and reopen rights, provenance, and preservation review.

COMPLETION CRITERIA

Assets, rights, lineage, accessibility, evidence, controls, delivery, preservation, and lifecycle are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve asset graph, source, rights, transformations, decisions, evidence, incidents, and next safe action. Hand off to Content, Video, Audio, Streaming, Search, Privacy, Records, and Operations.

RELATED CODEX SECTIONS

054 Content Architecture; 097 Content Design; 143 Video Engineering; 144 Media Engineering; 145 Streaming Engineering; 146 Search Engineering; 262 Artifact Management; 393 Digital Preservation.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 054, 075-106 and 107-143, Owner Cockpit Requirements, and Status and Evidence Lexicon. Rights, consent, preservation formats, accessibility standards, jurisdictional deletion, and source authority remain unresolved.

145

SECTION 145 - STREAMING ENGINEERING

CODEX/03_SECTIONS/145_STREAMING_ENGINEERING.txt

SHA-256 7d29e43c06a43722fbbaef351909dc9cafc2b5cc2d069e5d52a00029e072a3ac

5,182 BYTES / PRESERVED SOURCE IDENTITY

SECTION 145 - Streaming Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of time-based delivery of audio, video, data, or interactive content across variable networks and clients with explicit latency, quality, access, continuity, and recovery behavior.

REQUIRED OUTCOME

A streaming baseline with source, segments, manifests, timing, authorization, adaptation, buffering, observability, accessibility, evidence, and retirement.

WHY THIS MATTERS

Streaming fails through latency, stalls, drift, stale manifests, unauthorized access, or quality adaptation. Availability metrics alone do not describe user or operational outcome.

APPLICABILITY

Applies to live, on-demand, interactive, event, media, telemetry, and data streaming systems.

NON-APPLICABILITY CONDITIONS

May be non-applicable where content is delivered only as bounded nonstreaming transfers with no timing or continuity claim. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires source, timing, codec, manifests, network, authorization, latency, quality, buffering, captions, recording, privacy, observability, and recovery objectives.

REQUIRED INPUTS

Provide stream type, segments, clocks, bitrate, clients, latency, access, DRM or protection, captions, failover, storage, and user-outcome requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve access, content, latency and quality tradeoffs, recording, privacy, continuity, and release. Humans decide incident communication and stream termination.

AI RESPONSIBILITIES AND BOUNDARIES

AI may detect stalls, optimize adaptation, caption, summarize, and forecast capacity. It must not expose streams, alter access, or trade safety or meaning for engagement without approval.

APPLICABLE EXISTING AGENT ROLES

Architect, Media Engineer, Network Engineer, Security Auditor, Privacy Reviewer, Accessibility specialist, Implementer, Tester, Verifier, Governance Reviewer, Observer, Documentation Writer, Memory Curator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define source, timing, clients, protocol, access, quality, continuity, and lifecycle.

2. Design ingest, packaging, manifests, delivery, adaptation, captions, failover, and recording.
3. Test latency, stalls, loss, drift, authorization, accessibility, scaling, and recovery.
4. Monitor, communicate, correct, and retire streams and contracts.

HUMAN-AI INTERACTION REQUIREMENTS

Show stream identity, source time, playback time, access, quality, buffer, captions, evidence, and consequence. Human approval is required for recording, access, and termination; nonstreaming status remains available.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Model source, clock, encode, segment, manifest, cache, edge, authorization, client, caption, and archive boundaries. Support adaptation, backpressure, failover, replay, and deletion.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show stream, audience, latency, stalls, drift, quality, access, incidents, evidence, and controls to throttle, pause, fail over, restrict, record, or terminate.

REQUIRED OUTPUTS AND ARTIFACTS

Streaming charter, source and rights records, protocol and manifest design, latency and quality budgets, access and privacy assessment, accessibility plan, load evidence, runbook, and retirement package.

EVIDENCE REQUIREMENTS

Identify stream, source, timestamp, segment, client, network, build, method, result, and limits. Separate delivery, quality, accessibility, security, privacy, and user outcome evidence.

VERIFICATION AND ACCEPTANCE TESTS

Verify authorization, manifests, timing, stalls, loss, reconnect, adaptation, captions, failover, load, recording, deletion, and client compatibility.

FAILURE STATES AND ANTI-PATTERNS

Unbounded buffering; stale manifests; unauthorized caches; hiding latency; ignoring captions; recording without consent; and treating stream availability as comprehension.

RECOVERY AND RECONSIDERATION

Pause or restrict stream, preserve source and timeline, fail over, provide alternate delivery, revoke access, compensate commitments, and reopen timing, rights, or capacity decisions.

COMPLETION CRITERIA

Source, timing, access, quality, evidence, accessibility, recovery, ownership, and lifecycle are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve source, manifest, segment, access, traces, incidents, decisions, evidence, and next safe action. Hand off to Media, Video, Network, CDN, Search, Security, Accessibility, Testing, and Operations.

RELATED CODEX SECTIONS

043 Event-Driven Architecture; 143 Video Engineering; 144 Media Engineering; 145 Streaming Engineering; 195 Network Security; 300 Content Delivery Networks; 239 Performance Testing; 322 Availability Management.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 040-044, 075-106 and 107-144, Owner Cockpit Requirements, and Status and Evidence Lexicon. Actual protocols, network populations, rights, latency, retention, and availability objectives remain unresolved.

146

SECTION 146 - SEARCH ENGINEERING

CODEX/03_SECTIONS/146_SEARCH_ENGINEERING.txt

SHA-256 3d4e42789dd0eaab71a5948f4574fea5eace8c8e8ed8e017781e228a2903ab18

5,374 BYTES / PRESERVED SOURCE IDENTITY

SECTION 146 - Search Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of systems that ingest, index, retrieve, rank, filter, explain, and correct information under explicit relevance, authority, access, freshness, provenance, and user-control constraints.

REQUIRED OUTCOME

A search baseline with sources, indexing, schemas, ranking, permissions, freshness, recall and precision evidence, explanations, abuse controls, and retirement.

WHY THIS MATTERS

Search can omit, expose, distort, or stale information. Ranking and relevance metrics can hide minority needs, unauthorized disclosure, or poor task outcomes.

APPLICABILITY

Applies to keyword, semantic, federated, enterprise, media, code, geospatial, log, and AI-assisted search systems.

NON-APPLICABILITY CONDITIONS

May be non-applicable to systems with no retrieval, discovery, ranking, filtering, or search-dependent decision. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires information architecture, source ownership, schemas, permissions, indexing, ranking objectives, freshness, evaluation sets, privacy, security, accessibility, and correction authority.

REQUIRED INPUTS

Provide queries, users, sources, relevance goals, filters, ranking, access, freshness, synonyms, language, citations, latency, feedback, and failure requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve source inclusion, access, ranking objectives, protected populations, evaluation, publication, and acceptance. Humans decide disputes over relevance, omission, and disclosure.

AI RESPONSIBILITIES AND BOUNDARIES

AI may propose ranking, summarize results, classify sources, and generate queries. It must preserve permissions and citations, expose uncertainty, and never equate rank with truth.

APPLICABLE EXISTING AGENT ROLES

Architect, Information Architect, Researcher, Analyst, Security Auditor, Privacy Reviewer, User Experience Reviewer, Implementer, Tester, Verifier, Governance Reviewer, Documentation Writer, Memory Curator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define users, sources, scope, access, vocabulary, relevance, freshness, and lifecycle.

2. Design ingest, index, query, ranking, filtering, explanation, correction, and deletion paths.
3. Evaluate recall, precision, task success, accessibility, privacy, security, latency, and bias.
4. Monitor drift, feedback, abuse, and source change; correct and retire safely.

HUMAN-AI INTERACTION REQUIREMENTS

Show query, user, source, ranking, permission, timestamp, citation, uncertainty, and evidence. Human review is required for access, ranking changes, sensitive sources, and generated results.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate source, index, query, ranker, filter, cache, authorization, result, citation, and feedback boundaries. Enforce access before ranking or caching and preserve deletion propagation.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show index version, freshness, source coverage, query, results, ranking rationale, access decisions, quality metrics, incidents, and controls to reindex, restrict, correct, or roll back.

REQUIRED OUTPUTS AND ARTIFACTS

Search charter, source and permission catalog, schema and index design, ranking policy, evaluation sets, relevance and bias evidence, provenance and correction model, runbook, and retirement record.

EVIDENCE REQUIREMENTS

Identify source, index, query, ranker, user context, permissions, timestamp, method, result, and limits. Separate retrieval, ranking, citation, task success, fairness, and security evidence.

VERIFICATION AND ACCEPTANCE TESTS

Verify access isolation, freshness, deletion, ranking, recall, precision, synonyms, language, adversarial queries, prompt injection, cache, source loss, accessibility, and recovery.

FAILURE STATES AND ANTI-PATTERNS

Ranking by engagement only; indexing unauthorized data; stale caches; missing citations; hiding no-result states; evaluation on convenient queries; and treating relevance as truth.

RECOVERY AND RECONSIDERATION

Restrict or disable affected index or ranker, preserve source and query evidence, reindex from authority, correct results, notify users, compensate decisions, and reopen evaluation and access review.

COMPLETION CRITERIA

Sources, permissions, ranking, freshness, evidence, accessibility, correction, recovery, and lifecycle are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve source and index identity, evaluation sets, queries, decisions, traces, incidents, evidence, and next safe action. Hand off to Information, Knowledge, AI, Data, Privacy, Security, Testing, and Operations.

RELATED CODEX SECTIONS

030 Information Architecture; 055 Knowledge Architecture; 146 Search Engineering; 147 Recommendation Systems Engineering; 171 Natural Language Processing; 218 Privacy Engineering; 246 Security Testing; 391 Data Retention.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 030, 054-057 and 107-145, Owner Cockpit Requirements, and Status and Evidence Lexicon. Source authority, evaluation populations, protected groups, language coverage, freshness objectives, and legal disclosure constraints remain unresolved.

147

SECTION 147 - RECOMMENDATION SYSTEMS ENGINEERING

CODEX/03_SECTIONS/147_RECOMMENDATION_SYSTEMS_ENGINEERING.txt

SHA-256 19b68d440075284fb50de685ca5d43c775cc349001c672a16eb76df3a0b2ff52

5,507 BYTES / PRESERVED SOURCE IDENTITY

SECTION 147 - Recommendation Systems Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of systems that select, rank, personalize, or suggest items, actions, content, or opportunities under explicit relevance, user-control, fairness, privacy, safety, and evidence constraints.

REQUIRED OUTCOME

A recommendation baseline with objectives, candidates, ranking, feedback, permissions, explanations, evaluation, controls, drift, and lifecycle ownership.

WHY THIS MATTERS

Recommendations allocate attention and opportunity. Optimizing clicks or conversions can create manipulation, exclusion, unsafe exposure, feedback loops, and opaque consequential decisions.

APPLICABILITY

Applies to content, products, people, routes, actions, alerts, jobs, treatments, learning, personalization, and automated prioritization systems.

NON-APPLICABILITY CONDITIONS

May be non-applicable to explicit user lookup or deterministic navigation with no ranking, personalization, suggestion, feedback, or allocation effect. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires purpose, users, protected outcomes, candidate sources, permissions, ranking objective, feedback, privacy, safety, fairness, explanation, evaluation, and override authority.

REQUIRED INPUTS

Provide candidates, context, features, constraints, ranking, exploration, feedback, exposure, opt-out, fairness, safety, latency, and lifecycle requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve objectives, protected groups, data use, safety rules, explanations, overrides, experimentation, and acceptance. Humans retain authority over high-impact recommendations and appeals.

AI RESPONSIBILITIES AND BOUNDARIES

AI may analyze candidates and draft ranking changes. It must expose proxy risks, uncertainty, feedback loops, and exclusions; it must not silently personalize consequential choices or suppress appeal.

APPLICABLE EXISTING AGENT ROLES

Researcher, Analyst, Architect, User Experience Reviewer, Privacy Reviewer, Security Auditor, Governance Reviewer, Implementer, Tester, Verifier, Critic, Documentation Writer, Memory Curator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define purpose, audience, candidates, permissions, constraints, and protected outcomes.
2. Design retrieval, ranking, exploration, feedback, explanation, opt-out, override, and safety controls.
3. Evaluate task success, relevance, diversity, fairness, calibration, harm, privacy, and drift.
4. Monitor, audit, correct, and retire models and policies under approval.

HUMAN-AI INTERACTION REQUIREMENTS

Show why an item was shown, source, user controls, context, model or policy, uncertainty, and consequence. Human review is required for high-impact use; nonpersonalized and appeal paths remain available.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate candidate authority, features, ranker, policy filter, personalization, feedback, experiment, explanation, cache, and outcome. Enforce permission and safety before ranking and preserve opt-out.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show model and policy version, audience, exposure, explanation, metrics, drift, protected-group impact, incidents, and controls to pause, narrow, reset, or roll back recommendations.

REQUIRED OUTPUTS AND ARTIFACTS

Recommendation charter, objective and constraint record, candidate and feature catalog, privacy and fairness assessment, evaluation set, experiment record, explanation and appeal design, monitoring, and retirement package.

EVIDENCE REQUIREMENTS

Identify model, policy, data, context, candidate set, audience, exposure, method, result, uncertainty, and limits. Separate ranking metrics from user outcome, fairness, safety, and long-term effects.

VERIFICATION AND ACCEPTANCE TESTS

Verify permissions, opt-out, explanation, adversarial inputs, feedback loops, cold start, drift, protected-group outcomes, unsafe content, exploration, caching, and human override.

FAILURE STATES AND ANTI-PATTERNS

Click-only objectives; hidden personalization; proxy discrimination; filter bubbles; unsafe exploration; no opt-out; experimentation without consent; and treating engagement as benefit.

RECOVERY AND RECONSIDERATION

Pause or narrow ranking, restore prior policy, remove unsafe candidates, preserve exposure evidence, notify affected users, provide appeal or manual review, compensate decisions, and reopen evaluation.

COMPLETION CRITERIA

Objective, constraints, controls, evidence, user choice, fairness, safety, recovery, and lifecycle are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve model, policy, features, candidate sources, exposures, decisions, evidence, incidents, dissent, and next safe action. Hand off to Search, AI, Data, Privacy, Safety, Testing, and Operations.

RELATED CODEX SECTIONS

146 Search Engineering; 147 Recommendation Systems Engineering; 168 Deep Learning Engineering; 180 Model Evaluation; 182 AI Safety; 188 Human-in-the-Loop Operations; 218 Privacy Engineering; 249 Usability Testing.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 030, 055-057, 075-106 and 107-146, Owner Cockpit Requirements, and Status and Evidence Lexicon. Protected-group definitions, impact regimes, feedback data, consent, explanation, and intervention thresholds remain unresolved.

148

SECTION 148 - GEOGRAPHIC INFORMATION SYSTEMS

CODEX/03_SECTIONS/148_GEOGRAPHIC_INFORMATION_SYSTEMS.txt

SHA-256 8661e2c6b959b97b26c676cf23aac21ceb3852ddbe27a04cb944cda987cf308d

5,394 BYTES / PRESERVED SOURCE IDENTITY

SECTION 148 - Geographic Information Systems

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of systems that capture, represent, analyze, visualize, query, exchange, and act on geographic or spatial information with explicit coordinate, temporal, privacy, accuracy, and authority controls.

REQUIRED OUTCOME

A GIS baseline with spatial reference, data lineage, accuracy, access, analysis, visualization, update, interoperability, evidence, and recovery rules.

WHY THIS MATTERS

Spatial error, stale boundaries, misleading maps, and exposed locations can cause physical harm, exclusion, privacy loss, or invalid decisions.

APPLICABILITY

Applies to mapping, geocoding, remote sensing, routing, spatial analytics, cadastral, environmental, logistics, asset, and location-aware systems.

NON-APPLICABILITY CONDITIONS

May be non-applicable to systems with no location, geometry, coordinate, map, spatial query, or geography-dependent outcome. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires spatial purpose, coordinate reference, data ownership, accuracy, temporal validity, privacy, security, interoperability, accessibility, visualization, and decision authority.

REQUIRED INPUTS

Provide geometries, coordinates, projections, units, sources, resolution, time, uncertainty, privacy zones, analysis, routing, users, and failure requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve data collection, location privacy, boundaries, accuracy, public use, safety, and acceptance. Domain authorities retain control over official geographic claims.

AI RESPONSIBILITIES AND BOUNDARIES

AI may geocode, classify imagery, suggest routes, and detect spatial patterns. It must expose uncertainty, preserve source and coordinates, and never infer sensitive location or direct action without authority.

APPLICABLE EXISTING AGENT ROLES

GIS Architect, Domain Expert, Researcher, Analyst, Privacy Reviewer, Security Auditor, Accessibility specialist, Implementer, Tester, Verifier, Governance Reviewer, Documentation Writer, Memory Curator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define spatial scope, reference systems, sources, accuracy, time, users, and authority.
2. Design capture, transform, store, index, analyze, visualize, route, share, and delete paths.
3. Verify coordinates, topology, accuracy, privacy, accessibility, performance, interoperability, and recovery.
4. Monitor source and boundary drift, correct, publish, restrict, and retire layers safely.

HUMAN-AI INTERACTION REQUIREMENTS

Show layer, source, projection, time, accuracy, sensitivity, analysis, uncertainty, and consequence. Human approval is required for official boundaries, sensitive location use, and physical action.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate sensor, source, coordinate, projection, geometry, topology, index, analysis, map, route, cache, and physical effect. Preserve lineage, precision, temporal validity, and privacy masking.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show map and layer versions, source freshness, projection, accuracy, access, analysis, incidents, and controls to restrict, correct, mask, roll back, or retire data.

REQUIRED OUTPUTS AND ARTIFACTS

GIS charter, layer and source catalog, spatial reference and accuracy record, privacy assessment, data lineage, analysis and map design, interoperability tests, runbook, and retirement package.

EVIDENCE REQUIREMENTS

Identify layer, source, coordinate system, timestamp, resolution, method, result, uncertainty, and limits. Separate positional accuracy, semantic validity, visualization, privacy, and decision evidence.

VERIFICATION AND ACCEPTANCE TESTS

Verify coordinate and unit handling, projections, topology, geocoding, routing, stale layers, missing data, privacy masking, accessibility, interoperability, source loss, and recovery.

FAILURE STATES AND ANTI-PATTERNS

Mixing coordinate systems; false precision; stale boundaries; exposing sensitive locations; map visual bias; routing without validation; and treating a map as the territory.

RECOVERY AND RECONSIDERATION

Restrict affected layers, preserve source and lineage, use an authoritative prior layer, correct or withdraw maps, compensate decisions, and reopen accuracy, privacy, and authority review.

COMPLETION CRITERIA

Spatial reference, source, accuracy, privacy, authority, evidence, recovery, and lifecycle are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve layer identity, coordinates, projections, sources, analyses, decisions, incidents, evidence, and next safe action. Hand off to Data, Search, Privacy, Security, Accessibility, Routing, Testing, and Operations.

RELATED CODEX SECTIONS

016 Domain Analysis; 030 Information Architecture; 148 Geographic Information Systems; 157 Data Integration; 163 Data Analysis; 218 Privacy Engineering; 247 Accessibility Testing; 391 Data Retention.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 016, 019-020, 030-031 and 107-147, Owner Cockpit Requirements, and Status and Evidence Lexicon. Official data authorities, reference systems, accuracy standards, sensitive-location rules, and update or retention duties remain unresolved.

149

SECTION 149 - BLOCKCHAIN ENGINEERING

CODEX/03_SECTIONS/149_BLOCKCHAIN_ENGINEERING.txt

SHA-256 c958849b60151fa26a219011b910b4487d0b9302a327b45531b4110f25c38f87

5,522 BYTES / PRESERVED SOURCE IDENTITY

SECTION 149 - Blockchain Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of shared ledgers, blockchain networks, wallets, nodes, consensus, transactions, cryptographic identity, data publication, and lifecycle without confusing immutability with correctness.

REQUIRED OUTCOME

A blockchain baseline with network purpose, consensus, assets, keys, transaction semantics, privacy, governance, observability, recovery, upgrade, and retirement controls.

WHY THIS MATTERS

An immutable record can preserve an error forever. Key loss, consensus failure, opaque fees, privacy leakage, and irreversible transactions require explicit authority and compensation planning.

APPLICABILITY

Applies to permissioned or public ledgers, blockchain infrastructure, wallets, nodes, tokens, proofs, on-chain records, and ledger-connected applications.

NON-APPLICABILITY CONDITIONS

May be non-applicable where a conventional authoritative store satisfies the outcome and no consensus, shared ledger, token, or on-chain integrity claim exists. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires ledger purpose, participants, consensus, threat model, key authority, asset and data model, privacy, fees, finality, governance, legal, recovery, and retirement authority.

REQUIRED INPUTS

Provide transactions, identities, keys, assets, state, consensus, finality, fees, privacy, validators, off-chain links, upgrade, incident, and compensation requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve network purpose, validators, keys, assets, legal and privacy use, upgrades, fees, risk, and acceptance. Humans retain authority over irreversible transactions and governance changes.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect transactions, model fees, draft code, and detect anomalies. It must not hold or request secrets, sign transactions, alter consensus, or declare finality without delegated authority and evidence.

APPLICABLE EXISTING AGENT ROLES

Architect, Cryptography Engineer, Security Auditor, Privacy Reviewer, Analyst, Implementer, Tester, Verifier, Governance Reviewer, Observer, Documentation Writer, Memory Curator, Coordinator, and Red Team. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Define participants, ledger, assets, keys, consensus, finality, privacy, and authority.
2. Design transaction, node, wallet, key recovery, monitoring, upgrade, and off-chain reconciliation paths.
3. Test adversarial consensus, replay, key compromise, reorganization, fees, privacy, and recovery.
4. Operate, govern, upgrade, compensate, and retire under explicit decisions.

HUMAN-AI INTERACTION REQUIREMENTS

Show network, transaction, signer, key scope, fee, finality, privacy, evidence, and consequence. Human approval is required for signing, upgrades, and material transfers; no conversational path bypasses wallet policy.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Separate wallet, key, transaction, mempool, consensus, ledger, oracle, off-chain store, and application boundaries. Define nonce, replay, finality, reorg, fee, privacy, audit, and compensation semantics.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show chain, height, node health, transaction, signer, finality, fees, alerts, evidence, and controls to pause signing, revoke keys, isolate nodes, reconcile, or migrate.

REQUIRED OUTPUTS AND ARTIFACTS

Ledger charter, participant and node catalog, asset and transaction model, key and wallet policy, threat and privacy assessment, contract and upgrade records, test evidence, runbook, and retirement plan.

EVIDENCE REQUIREMENTS

Identify chain, block, transaction, signer, key reference, code, node, input, result, finality, method, date, and limits. Preserve failed, replaced, reverted, and compensated evidence.

VERIFICATION AND ACCEPTANCE TESTS

Verify signatures, permissions, replay, nonce, consensus, reorg, finality, key rotation, node loss, privacy, oracle failure, fee exhaustion, upgrade, and compensation.

FAILURE STATES AND ANTI-PATTERNS

Treating immutability as truth; exposed private data; unaudited contracts; hot keys; unclear finality; ignoring reorgs; and assuming transactions can be destructively undone.

RECOVERY AND RECONSIDERATION

Pause signing, revoke or rotate keys, isolate contracts or nodes, preserve ledger evidence, compensate through an authorized transaction, communicate limits, and reopen governance and security review.

COMPLETION CRITERIA

Purpose, authority, keys, consensus, evidence, privacy, upgrades, compensation, and retirement are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve chain, block, transaction, key references, code, decisions, incidents, evidence, and next safe action. Hand off to Blockchain Architecture, Cryptography, Security, Privacy, Smart Contracts, Testing, and Operations.

RELATED CODEX SECTIONS

149 Blockchain Engineering; 150 Smart Contract Engineering; 203 Cryptography; 204 Key Management; 218 Privacy Engineering; 246 Security Testing; 258 Configuration Management; 289 Change Management.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 019-020, Sections 045, 047-050, 107-148, Owner Cockpit Requirements, and Status and Evidence Lexicon. Jurisdiction, consensus model, asset authority, privacy, custody, finality, fees, and recovery or compensation rules remain unresolved.

150

SECTION 150 - SMART CONTRACT ENGINEERING

CODEX/03_SECTIONS/150_SMART_CONTRACT_ENGINEERING.txt

SHA-256 e42c5f83d6b1f8e65d8a1c4a5b31e2ff9995327d38cb529feb86650a9b7965fc

5,739 BYTES / PRESERVED SOURCE IDENTITY

SECTION 150 - Smart Contract Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Define governed engineering of executable ledger contracts that encode state, rules, permissions, value transfer, events, and external effects under immutable or upgrade-controlled deployment.

REQUIRED OUTCOME

A smart-contract baseline with specification, invariants, authority, keys, interfaces, gas or resource limits, upgrade policy, verification, monitoring, and compensation path.

WHY THIS MATTERS

Contract code can be permanently executable and externally valuable. Bugs, oracle errors, reentrancy, privilege mistakes, and upgrade ambiguity can create irreversible harm.

APPLICABILITY

Applies to on-chain programs, tokens, escrow, DAOs, governance, automated settlement, wallets, or ledger-connected contracts.

NON-APPLICABILITY CONDITIONS

May be non-applicable to ordinary application rules with no ledger execution, cryptographic signing, on-chain state, or irreversible external effect. Record scope, owner, evidence, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

Requires formal purpose, state model, invariants, threat model, language and runtime, interfaces, oracle and key authority, upgrade, audit, deployment, monitoring, legal, and compensation authority.

REQUIRED INPUTS

Provide states, functions, roles, assets, permissions, invariants, events, external calls, oracle data, limits, fees, upgrade, pause, dispute, and retirement requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Humans approve specification, authority, key custody, deployment, audit, upgrade, pause, assets, and acceptance. Humans retain authority over irreversible value and governance effects.

AI RESPONSIBILITIES AND BOUNDARIES

AI may draft code, tests, invariant checks, and audit questions. It must not sign or deploy, claim formal security from tests, or weaken access, pause, upgrade, or compensation controls.

APPLICABLE EXISTING AGENT ROLES

Architect, Cryptography Engineer, Security Auditor, Researcher, Analyst, Implementer, Tester, Verifier, Governance Reviewer, Red Team, Documentation Writer, Memory Curator, and Decision Arbiter Advisory. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Specify state, roles, invariants, calls, assets, events, authority, and external dependencies.
2. Review code, dependencies, oracle, gas, upgrade, pause, and key paths.
3. Verify statically, dynamically, adversarially, and formally where warranted before deployment.
4. Deploy staged, monitor, govern upgrades, respond, compensate, and retire through authorized transactions.

HUMAN-AI INTERACTION REQUIREMENTS

Show contract address, version, code hash, signer, role, call, state, value, oracle, gas, evidence, and consequence. Human approval is required for deployment, signing, upgrades, and material calls; all decisions remain attributable.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Represent specification, code, compiler, bytecode, state, storage, roles, keys, oracle, external calls, events, upgrade proxy, pause, and compensation as linked versioned artifacts. Model reentrancy, replay, failure, and partial effects.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

Show contract, network, code, roles, balances, calls, events, alerts, oracle freshness, pause state, evidence, and controls to pause, revoke, upgrade, migrate, or compensate within authority.

REQUIRED OUTPUTS AND ARTIFACTS

Contract specification, state and invariant model, source and bytecode provenance, threat assessment, audit and formal evidence, deployment and key records, monitoring, incident runbook, and compensation or retirement plan.

EVIDENCE REQUIREMENTS

Identify network, contract, address, source, compiler, bytecode hash, call, signer, state, value, oracle, method, result, and limitations. Audit, test, formal, runtime, and owner-acceptance evidence remain distinct.

VERIFICATION AND ACCEPTANCE TESTS

Verify access control, invariants, reentrancy, arithmetic, replay, oracle failure, denial of service, upgrade, pause, gas, key rotation, event integrity, deployment identity, and compensation paths.

FAILURE STATES AND ANTI-PATTERNS

Unaudited deployment; proxy confusion; privileged upgrade; unsafe external call; missing pause; oracle trust; hidden admin key; assuming tests prove safety; and proposing destructive rollback for immutable state.

RECOVERY AND RECONSIDERATION

Pause or narrow authorized calls, preserve chain evidence, revoke or rotate keys, migrate or compensate through linked transactions, communicate residual loss, and reopen specification, audit, and governance decisions.

COMPLETION CRITERIA

Specification, code, authority, invariants, evidence, deployment, monitoring, compensation, and retirement are explicit. Keep CONTENT-COMPLETE until independent verification and owner acceptance.

HANDOFF AND CONTINUITY REQUIREMENTS

Preserve specification, source, compiler, bytecode, address, keys by reference, transactions, audits, decisions, incidents, evidence, and next safe action. Hand off to Blockchain, Cryptography, Security, Key Management, Testing, Operations, and Governance.

RELATED CODEX SECTIONS

048 Governance Architecture; 149 Blockchain Engineering; 150 Smart Contract Engineering; 203 Cryptography; 204 Key Management; 246 Security Testing; 251 Compliance Testing; 258 Configuration Management; 289 Change Management.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Based on the Canonical Section Contract, shared governance, Sections 019-020, Sections 045, 048-050, 107-149, Owner Cockpit Requirements, and Status and Evidence Lexicon. Contract language, chain, audit depth, formal-method need, asset jurisdiction, key custody, upgrade authority, and compensation feasibility remain unresolved.